VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Faculty of Computer Science and Engineering

REPORT

Specialized Project

Computer Science Project

Developing Academic AI Warning System for HCMUT Students

Specialization: Software Engineering

Committee: Committee 4CC – Specialized Project
Supervisor 1: Assoc. Prof. Dr. Bui Hoai Thang
Supervisor 2: MEng. Bui Cong Tuan

—o0o—

Student 1: Ngo Thai Minh Tien (2252809)
Student 2: Nguyen Viet Hoang (2252235)

HO CHI MINH CITY, JUNE 2026

Declaration of Originality

We hereby declare that the Specialized Project entitled **"Developing Academic AI Warning System for HCMUT Students"** is our own independent research, conducted under the supervision of **Assoc. Prof. Dr. Bui Hoai Thang** and **Ph.D. Bui Cong Tuan**. All content, analysis, design, and implementation presented in this report are honest and have not been copied from or based upon any other comparable project.

We further commit that the use of Artificial Intelligence (AI) tools during the development of this project has been strictly limited to syntactic reference, sample data generation, and performance optimization. All core technological solutions, system architectures, machine-learning models, and algorithms presented in this work are the intellectual products of the authors. We accept full responsibility for the originality of the content and pledge to comply with any disciplinary measures imposed by the Faculty and the University should any form of academic misconduct be discovered.

Ho Chi Minh City, June 2026

Ngo Thai Minh Tien

Nguyen Viet Hoang

Acknowledgements

First and foremost, we would like to express our deepest gratitude to **Assoc. Prof. Dr. Bui Hoai Thang** and **Ph.D. Bui Cong Tuan**, who have generously dedicated their time, expertise, and patience throughout the entire course of this Specialized Project. Their insightful guidance, constructive feedback, and steadfast support have not only shaped the technical direction of our work, but also helped us grow significantly in both professional skills and research mindset. The valuable lessons we have learned from our supervisors will continue to benefit us long after the completion of this project.

We are also profoundly grateful to the faculty members and staff of the **Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology – Vietnam National University Ho Chi Minh City**, who have provided us with a stimulating academic environment and the foundational knowledge required to undertake a project of this scope. The cumulative knowledge gained through their dedicated teaching across our years of study has proven indispensable in transforming the initial concept of this system into a working solution.

Finally, we wish to extend our heartfelt thanks to our families and friends for their unwavering encouragement throughout this journey. Their patience and emotional support have helped us overcome the numerous challenges encountered during the design, development, and evaluation of the system. Without this collective support network, the timely completion of this project would not have been possible.

Sincerely,

Ngo Thai Minh Tien

Nguyen Viet Hoang

Abstract

Academic underperformance and untimely dismissal remain pressing concerns at the Ho Chi Minh City University of Technology (HCMUT), where students must navigate a credit-based curriculum with strict GPA and progress requirements. While the existing administrative procedures are capable of identifying at-risk students, they typically do so reactively, after the warning thresholds have already been crossed. This project, entitled **"Developing Academic AI Warning System for HCMUT Students"**, addresses this limitation by designing and implementing an integrated, proactive early-warning platform that simultaneously serves two stakeholder groups: (i) *students*, by providing personalized risk forecasts, course-level performance predictions, what-if GPA simulations, and an AI assistant capable of answering questions about academic regulations; and (ii) *academic administrators*, by offering centralized dashboards for cohort-wide monitoring, bulk data ingestion, warning approval workflows, and event notifications.

From a technical standpoint, the system follows a modern client–server architecture coupled with a service-oriented backend. The frontend is implemented in **Next.js 14** with the `shadcn/ui` component library, delivering a responsive single-page experience for both students and administrators. The backend is built on **FastAPI** and **PostgreSQL**, exposing a RESTful API consumed by the web client. Two intelligent subsystems form the core of the platform: an **XGBoost-based risk prediction pipeline** enriched with SHAP-based explanations, which produces per-student dropout probabilities and identifies the most influential academic features, and a **Retrieval-Augmented Generation (RAG) chatbot** powered by Google Gemini that grounds its responses in the official HCMUT academic regulations through dense-vector retrieval over PDF documents. Authentication, role-based access control, batch prediction jobs, and notification dispatch are implemented as dedicated service modules to ensure separation of concerns and future scalability.

The resulting platform constitutes a complete prototype that covers the full academic-warning workflow, from raw grade ingestion (including a custom myBK parser) to risk assessment, personalized study-plan recommendations, and student-facing notifications. The system has been designed with extensibility in mind: the modular architecture allows additional faculties, new prediction features, or external data sources to be integrated with minimal effort, paving the way for broader deployment at HCMUT and, potentially, at other higher-education institutions facing similar challenges in student-success monitoring.

Table of Contents

1 Project Overview

1.1 Introduction

1.1.1 Motivation

Ho Chi Minh City University of Technology (HCMUT, Vietnam National University HCMC) places its students under a dual pressure imposed by the credit-based academic system: maintaining a minimum annual cumulative GPA while progressing through the credit accumulation schedule. In early 2026, VNU-HCMC reported that only approximately **40.6%** of HCMUT students graduated on time in 2025 — the **lowest** rate across all member institutions — driven by four main causes: incomplete coursework, failure to meet the B1 English graduation standard, individual psychological factors, and financial difficulties. The first two of these are amenable to early intervention if students are equipped with proactive monitoring tools.

Nationally, academic warnings and forced withdrawals have likewise become widespread concerns. The Ministry of Education and Training has codified dropout-rate thresholds (below 10% annually, below 15% for first-year) as a quality-accreditation criterion through Circular 01/2024/TT-BGDĐT, creating strong institutional motivation for proactive academic risk-monitoring tools.

Meanwhile, existing solutions leave substantial gaps. HCMUT's `myBK` portal aggregates approximately 13 online services but remains a transactional administrative gateway: no risk prediction, no virtual assistant for regulation lookup, and warnings are issued only *after* thresholds are crossed. The Academic Affairs end-of-semester review remains a largely manual process that scales poorly. The recent maturation of interpretable machine learning and large language models opens up the possibility of a new generation of tooling — simultaneously delivering personalized risk forecasting, natural-language regulation Q&A, and an end-to-end automated workflow for staff. This is the direct motivation for the present project, titled **"Development of an AI-Powered Academic Warning System for Students at Ho Chi Minh City University of Technology"**.

1.1.2 Project Objectives

The project pursues three mutually-reinforcing objectives. **For students**, the system provides a web platform for self-monitoring of GPA, ingesting myBK grades, viewing academic risk scores with SHAP explanations, simulating "what-if" GPA scenarios, receiving personalized study plans, managing schedules, and asking a regulation chatbot. **For Academic Affairs Office staff**, it automates workflows traditionally done manually: an overview dashboard, bulk Excel import, batch risk prediction, AI-suggested warning approval, document management for the chatbot, and periodic report exports. **For research and applied work**, the project demonstrates the feasibility of integrating interpretable machine learning (XGBoost + SHAP), hyperparameter tuning (Optuna), and Retrieval-Augmented Generation with Google Gemini into the Vietnamese higher-education domain.

1.1.3 Social Benefits

For students, the system supports better academic decisions by making future risk visible in advance, reducing the proportion of students who receive warnings or are forced to withdraw. The 24/7 virtual assistant answers regulation questions — information typically scattered across dense documents — without requiring office hours.

For the university, the system reduces Academic Affairs Office workload by automating warning computation and identifying students needing early intervention. Aggregated data can serve as input for policy analysis and directly supports compliance with MOET Circular 01/2024 on institutional quality standards.

For society, the project provides empirical evidence that advanced AI can be applied to a concrete Vietnamese education problem, with source code and architecture reusable for other universities facing similar challenges.

1.1.4 Expected Results

- **Product:** A full-stack web application packaged with Docker Compose, with 10 student pages, 7 admin pages, approximately 60 API endpoints, and a PostgreSQL database with pgvector.
- **ML model:** XGBoost achieving $F_1 \geq 0.7$ and AUC-ROC ≥ 0.85 on the held-out test set, each prediction accompanied by accessible SHAP explanations.
- **Chatbot:** Average response latency under 5 seconds per turn, with source citations (document name, page number).
- **Performance:** Main page load under 3 seconds; data-query APIs under 500 ms; batch prediction for 1,000 students under 60 seconds.

- **Security:** bcrypt-hashed passwords, JWT authentication, role-based access control across two roles, API error rate below 1%.
- **Testing:** At least 60% automated test coverage for core components.

1.1.5 Scope and Limitations

1.1.5.1 Scope of work. The project covers the full software stack: a frontend built with Next.js 14, TypeScript, `shadcn/ui` and Tailwind CSS as a role-aware SPA; a FastAPI asynchronous backend on Python 3.11 with 13 router groups, 11 business services and 11 data entities; PostgreSQL 16 with pgvector for semantic vector retrieval; an AI subsystem comprising XGBoost (with Optuna and SHAP) trained on 1,000 synthetic student records and a RAG chatbot using Google Gemini 1.5 Flash with Google's embedding model; all packaged with Docker Compose and using APScheduler for in-process scheduled jobs.

1.1.5.2 Out of scope. The system does **not** integrate directly with myBK (using a copy-paste mechanism in place of an API), does **not** support SMS or mobile push notifications (in-app and email only), does **not** implement multi-tenant architecture (single institution only), does **not** handle special academic cases (major transfers, leaves of absence, double-degree programs), does **not** train on HCMUT's real student data (uses synthetic data due to access restrictions), and is **not yet** deployed at production scale (the current artifact is a production-ready prototype suitable for evaluation and demo).

1.2 Overall Analysis

Existing student management and academic-support systems show a sharp divide between two regions. **Domestic** solutions operate primarily as transactional administrative portals that issue warnings only *after* thresholds are crossed. **International** solutions have matured into *student success management* platforms capable of ML-based risk prediction and integrated virtual assistants. This section surveys both groups, provides a comparison, and derives the inheritance direction for the project.

1.2.1 Domestic Systems

HCMUT's `myBK` portal is the official Single Sign-On entry point administered by the Academic Affairs Office, aggregating approximately 13 online services from course registration and grade lookup to BKPay and BK-LMS. However, `myBK` only *displays* warning status (Level 1, 2, dismissal) after end-of-semester review by staff, has no ML-based risk prediction, no regulation chatbot for enrolled students, and notifications are push-style rather than proactive forecasts.

Edusoft.NET by Anh Quan Technology (AQTech) is a commercial academic management suite deployed at numerous Vietnamese universities, providing traditional SIS modules (records, training plans, course registration, tuition, scholarships, rewards/discipline). Per publicly available product materials, Edusoft.NET can compute warnings via configured rules but **does not advertise** ML-based prediction features, has no virtual assistant, and is an ERP-style SIS relying on operational staff to interpret data.

UEH AI Chatbot of the University of Economics HCMC (launched April 2025) is currently the most prominent LLM-based chatbot officially deployed at a Vietnamese university. Its initial scope focuses on admissions consulting and career guidance for prospective applicants, available 24/7. Expansion to support enrolled students and act as a virtual TA for lecturers was announced as *planned*, not yet generally available. No public information exists on academic risk prediction or model explainability.

Other portals. FPT University's `FAP/myFAP` has a strict attendance policy that creates implicit early-warning signals but lacks ML prediction or regulation chatbots. HUST's portals (`ctt`, `qldt`, `dk-sis`) and VNU Hanoi's portals all operate on administrative models similar to HCMUT, with no public evidence of AI features.

Vietnamese academic research has produced meaningful work on ML-based academic outcome prediction — for example, Sang et al. (2020) at Can Tho University using multi-layer neural networks, or Nguyen Van Thuy (2023) at the Banking Academy comparing multiple models (Random Forest, XGBoost, CatBoost). However, these results have **not been integrated** into operational SIS at surveyed universities and remain at the academic-publication stage.

1.2.2 International Systems

The international market, particularly North America, has developed a mature segment called *Student Success Management*. Six representative platforms surveyed are:

Civitas Learning (Austin, Texas) provides predictive risk scores for student persistence, completion, and engagement. Its distinctive feature is building models *per institution* rather than using a single shared model, though the specific algorithms are not publicly disclosed. No student-facing chatbot.

EAB Navigate360 (Washington, D.C.) is one of the largest Student Success Management vendors in the United States, with over 850 institutions using it. The standout feature is the **AI Assistant** (Knowledge Agent + Web Agent) supporting over 90 languages. EAB claims to have deployed over 200 custom-built predictive models, but algorithm details and explanation methods are not disclosed.

Watermark Student Success & Engagement (formerly Aviso Retention) provides predictive models for persistence, advisor collaboration tools, and notably **SMS text messaging** for student outreach with empirically high response rates, alongside a dedicated student mobile app.

Starfish (acquired by EAB in 2021) is a campus-wide *early-alert + connect* platform, characterized by a "kudos/flags" framework raised by faculty and deep integration into faculty workflows.

Anthology Illuminate (Anthology + Blackboard after the 2021 merger) is a data-lake analytics platform aggregating data from multiple Blackboard products, producing Student Risk Reports from combined LMS and SIS data.

D2L Brightspace Student Success System (S3) is a ML-based predictive analytics suite identifying struggling learners from engagement data in the Brightspace LMS, strong on learning-telemetry granularity.

1.2.2.1 Common patterns and gaps.

The survey reveals four noteworthy points: (1) *no platform supports Vietnamese* at the primary UI level (EAB's chatbot supports 90+ languages but only at the conversational layer); (2) *no vendor publicly documents per-prediction explanation mechanisms* (SHAP, LIME) in marketing materials — a defensible gap for the present project; (3) *academic structure is not suited to Vietnam*, with every platform encoding US/Canadian assumptions about credits and regulations; (4) *pricing is not public, enterprise-only sales models* with typical annual contracts in the six-figure US-dollar range.

1.2.3 General Comparison

Table 1: *Comparison of academic management systems and the proposal*

Criterion	Domestic systems	UEH AI Chatbot	International	This project
ML-based risk prediction	No	No	Yes (model undisclosed)	Yes (XGBoost, transparent)
ML result explainability	N/A	N/A	Not disclosed	Yes (SHAP)
Academic-regulation chatbot	No	Admissions only	Limited	Yes (Gemini + HCMUT RAG)
"What-if" GPA simulator	No	No	No	Yes
Vietnamese at application level	Yes	Yes	No	Yes
HCMUT regulation integration	Partial	No	No	Yes
Deployment cost	Per contract	N/A	6-figure USD/year	Low (self-host)

1.2.4 Inheritance and Development Direction

The project inherits two proven pillars from international platforms: *the predictive-analytics paradigm* with per-student risk scores (Civitas, EAB Navigate360), and *the early-alert + intervention workflow philosophy* that routes warning signals into actual workflows (Starfish, EAB).

Simultaneously, the project adds three components either missing or undisclosed in international platforms: *per-prediction explainability via SHAP* displaying specific contributing factors per student; *a RAG chatbot* grounded in HCMUT's own regulation documents with explicit source citations; and *a what-if GPA simulator* allowing students to self-assess the impact of academic decisions.

For the HCMUT context, the project also builds components no international vendor offers out of the box: a myBK transcript parser, a warning engine directly encoding MOET Circular 08/2021/TT-BGDĐT and HCMUT regulations, and a fully Vietnamese interface. Compared to domestic systems, the project shifts from *reactive* to *proactive* warning, from *manual* to *semi-automated workflow with human approval*, and from *transactional administrative portal* to *decision-support platform*.

1.2.5 Conclusion

The survey reveals a clear gap: international systems are mature in predictive analytics but ill-suited to the Vietnamese context in language, regulations and cost; domestic systems remain at the transactional-portal stage with no modern AI integration. No system — domestic or international — simultaneously provides explainable risk prediction, regulation chatbot, GPA simulation and automated administrator workflow within a single integrated solution designed for HCMUT. This is precisely the gap the project aims to fill.

1.3 Stakeholders and Needs

The system serves two main user groups with distinct roles and needs: **students** (end users, large scale) and **Academic Affairs Office staff** (administrators, smaller in number but with elevated privileges).

1.3.1 Students

Students are the central stakeholder group, comprising undergraduate students enrolled at HCMUT across all years. They need a holistic, comprehensible, up-to-date view of their academic situation — GPA per semester and cumulative, total credits earned, list of failed courses, current warning level, and trend over time — presented visually with charts and color coding so signals requiring attention are easy to identify.

Unlike traditional reactive approaches, today's students want to *know in advance* about potential risks: an overall academic risk score with category classification, the specific factors contributing to that score, and pass/fail predictions for each currently-enrolled course. They also need decision-support tools such as a "what-if" GPA simulator, suggestions for retake-priority courses, and recommended credit loads for the upcoming semester.

For information needs, students want a virtual assistant that understands natural Vietnamese questions and answers accurately about academic regulations — typically scattered across multiple documents (MOET Circular 08/2021, HCMUT regulations, faculty rules, scholarships, graduation conditions...) — with explicit source citations. They also need a unified tool to manage timetables, exam schedules and personal events across multiple views (week, month, list), along with multi-channel notifications (in-app + email) that can be customized. Finally, as a generation accustomed to modern web applications, students demand a friendly, fast interface that works equally well on desktop and smartphone, with step-by-step wizards for complex features.

1.3.2 Administrators (Academic Affairs Office Staff)

Administrators are Academic Affairs Office staff at HCMUT, responsible for issuing official warnings, managing student data, updating regulations, creating academic events and exporting reports. They need an overview dashboard providing university-wide and per-faculty metrics — student distribution by warning level and by AI risk level, top list of highest-risk students for priority intervention, course-failure rate per course and per semester — updated in real time or at minimum per login session.

For data operations, administrators need a bulk Excel import tool with row-level validation and specific error reporting, along with the ability to manually trigger university-wide risk-prediction tasks outside of the automated schedule (e.g., after ingesting a new semester's data). They also need RAG document management: upload regulation files (PDF, DOCX, TXT, MD), update versions when regulations are amended, and deactivate outdated documents.

Unlike a fully-automated model, AI-suggested warnings must be *reviewed and approved* by administrators before being sent to students. This ensures prediction quality is controlled, allows for contextual additions the model cannot see (e.g., student on academic leave), and maintains human accountability for decisions affecting student welfare. Finally, administrators need configuration capabilities for business parameters (default warning-suggestion threshold of 0.6), targeted academic event creation (university-wide / faculty / cohort), and periodic report exports in common formats (Excel for analysis, PDF for archival).

1.4 System Requirements

Building on the context and stakeholder analysis, this section consolidates system requirements into two groups: functional requirements ("what the system does") and non-functional requirements ("how well the system does it").

1.4.1 Functional Requirements

1.4.1.1 For students. Students can **manage their account** (register, log in, update profile, recover password) and **view a personal dashboard** displaying cumulative GPA, credits, warning level, latest AI risk score and trend chart. For grade management, students can **import grades automatically** by pasting their myBK transcript (the

system parses, previews, and persists after confirmation) or **enter grades manually** with common weight templates. For the AI subsystem, students can **view risk predictions** with SHAP explanations and per-course pass/fail forecasts, **simulate GPA** with hypothetical scores (results not persisted), **query the RAG chatbot** with streaming responses and source citations, and **view a study plan** suggesting retake-priority courses and an appropriate credit load. Finally, students can **manage warnings** (view history, mark as resolved), **manage schedules** (import timetable and exam schedule from myBK, create personal events, view by week/month/list), and **manage notifications** (mark as read, configure email reception).

1.4.1.2 For administrators. Administrators can **view the admin dashboard** with university-wide aggregate metrics, **manage students** (search, filter by faculty/warning level/risk, view detailed profiles), and **bulk-import data from Excel** (students and grades, with validation and downloadable templates). For business operations, administrators can **manually trigger batch prediction**, **approve pending warnings** suggested by AI or **create manual warnings** for special cases, **configure the AI threshold** for warning suggestions, and **manage the RAG document repository** (upload/deactivate/delete, with ZIP and OCR support for scanned PDFs). Administrators also **create and manage academic events** with target audiences (university-wide / faculty / cohort, mandatory or optional), and **export reports** in Excel or PDF formats.

1.4.2 Non-Functional Requirements

1.4.2.1 Performance. Main page load under 3 seconds; data-query APIs under 500 ms; risk-prediction API for a single student under 1 second (including feature extraction, model inference, and SHAP explanation); RAG chatbot first-token latency under 3 seconds and full response under 8 seconds; batch prediction for 1,000 students under 60 seconds.

1.4.2.2 Reliability. API error rate below 1% under normal conditions; critical transactions (grade saving, warning creation, notification dispatch) are wrapped in database transactions ensuring integrity; chatbot has a fallback chain (Gemini → HuggingFace → Local → Extractive); email notification failures do not affect primary API responses.

1.4.2.3 Security. Passwords hashed with bcrypt (no plaintext storage); session authentication via JWT with configurable expiry (default 24 hours); RBAC across two roles (student accesses own data, administrator accesses all university data); `/admin/*` APIs require the admin role, `/students/me/*` APIs automatically filter by current user; database queries go through SQLAlchemy ORM with bound parameters; CORS is centrally configured to allow only registered origins.

1.4.2.4 Usability. Fully Vietnamese interface with the ability to switch to English; responsive across screen sizes from mobile to desktop; complex features (myBK import, event creation) guided step-by-step (wizard); onboarding tour for new users; toast notifications for operation results; loading states (skeleton, spinner) while awaiting data.

1.4.2.5 Scalability and Maintainability. Asynchronous backend architecture for non-blocking concurrent processing; modular monolith with clearly separated layers (API → Service → ORM → DB) for easy maintenance and future microservices extraction; entire system packaged with Docker Compose for single-command deployment; Alembic-based schema migration; ML model loaded as a singleton with feature-version validation.

1.4.2.6 Explainability and Testability. Every risk prediction is accompanied by SHAP explanation rendered accessibly for students (not technical labels); every chatbot answer includes clear source citations (document name, page number, excerpt); core components (warning engine, GPA calculator, myBK parser) are designed as pure functions, easy to unit-test; at least 60% test coverage for core components, plus integration tests for critical API endpoints.

1.4.3 Conclusion

The requirements above collectively describe a system that is functionally rich, operationally high-quality, and respectful of the specifics of the Vietnamese higher-education context. Clearly defining both groups of requirements — functional and non-functional — forms an important basis for the design decisions in subsequent chapters, particularly the Client-Server architecture with an asynchronous backend (Chapter **??**), the database design combining transactional and vector data (Chapter **??**), and the choice of AI technologies enabling result explanation (XGBoost + SHAP, RAG with citation).

2 System Model

2.1 Use Case Diagram

The AI-powered academic warning system is designed to serve three categories of actors: **Student**, **Administrator** (Academic Affairs Office staff), and the **Scheduler** (a system actor representing automated periodic tasks). Use cases are grouped into four functional categories that reflect the distinct user journeys of each actor.

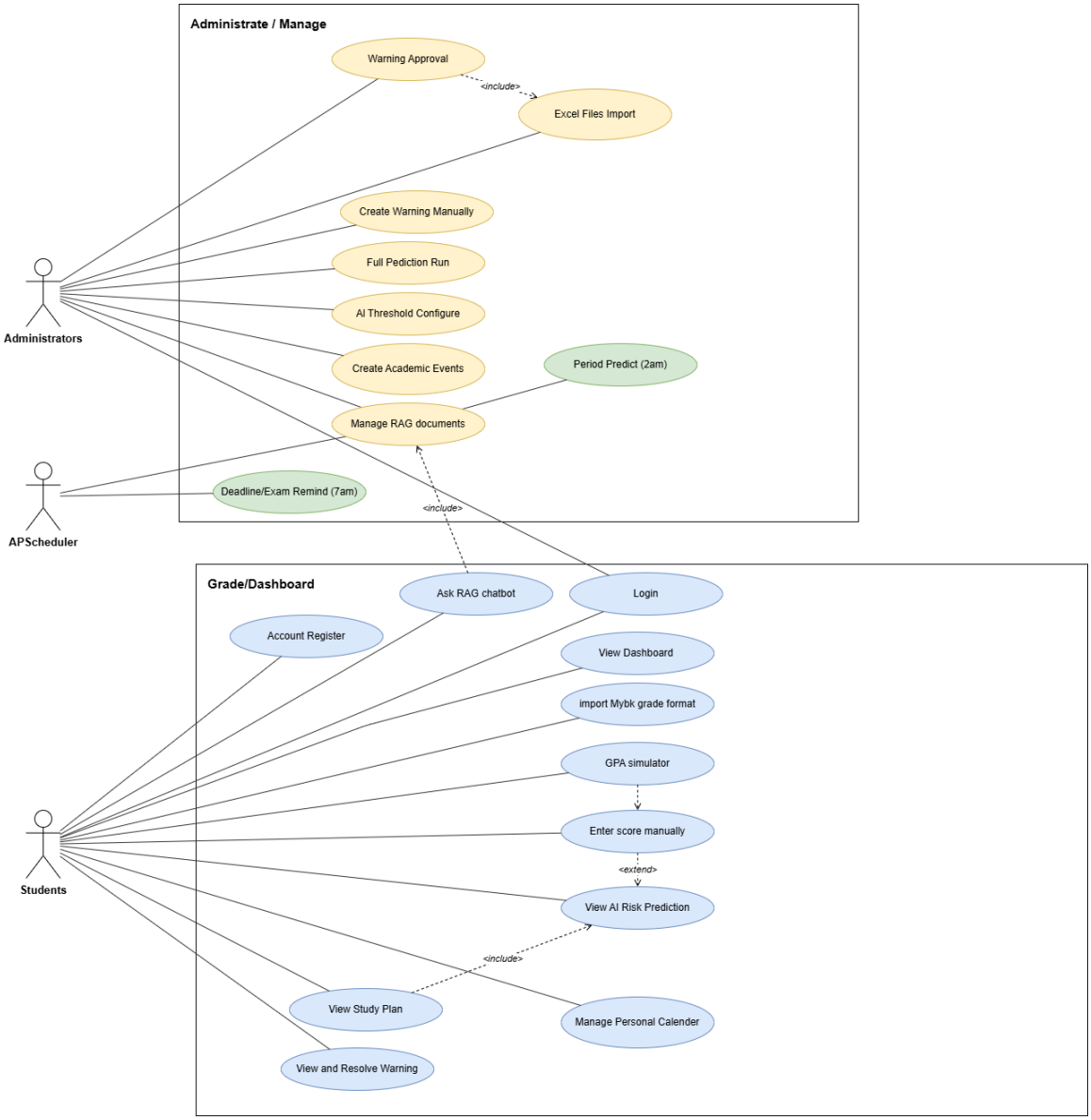

Figure 1: *Overall Use Case Diagram*

For the **Student**, the journey begins with *Authentication* (Register Account, Login) and continues into the *Academic Self-Monitoring* group: viewing the personal dashboard with GPA and warning status, importing grades from myBK via copy-paste, manually managing course grades, and resolving warning notices. The *AI Support* group is the centerpiece of student value: viewing the AI risk prediction (which extends into showing SHAP-based contributing factors), running the GPA What-if Simulator, and querying the RAG chatbot for regulation Q&A. The *Planning & Schedule* group covers viewing the personalized study plan, managing the timetable and exam schedule (with myBK import), creating personal events, and managing notifications.

For the **Administrator**, use cases concentrate on *Bulk Operations* (importing students and grades from Excel, triggering batch risk prediction) and *Warning Workflow* (reviewing AI-suggested warnings pending approval, creating manual warnings, configuring the AI suggestion threshold). The *Knowledge & Content* group covers managing the RAG document repository (upload, deactivate) and creating academic events targeted at specific audiences (university-wide, faculty, cohort).

The **Scheduler** actor encapsulates two recurring jobs that run inside the application process via APScheduler: the daily batch prediction task that refreshes risk scores across the active student body, and the deadline-reminder

job that scans mandatory events and exam schedules to dispatch notifications 24 hours in advance. These jobs operate without human intervention but reuse the same business services invoked by the Administrator's manual triggers.

Three relationships link the use cases. «include» expresses mandatory reuse: *GPA Simulator* includes *View Risk Prediction* because every simulation re-invokes the same prediction pipeline on hypothetical features; *View Study Plan* includes *View Risk Prediction* because retake recommendations rely on the same risk model; and *Ask RAG Chatbot* includes *Manage RAG Documents* since chatbot quality depends on the corpus the administrator maintains. «extend» expresses optional behaviour: *Approve Pending Warning* can extend *Bulk Import Excel* when an import provides enough new data to trigger AI suggestions worth reviewing.

2.2 Use Case Specifications

The following sub-sections specify fifteen representative use cases covering the full functional scope of the system. Each specification follows a seven-row template (Name, Actors, Description, Trigger, Pre-conditions, Post-conditions, Normal Flow, Alternative Flow, Exception Flow) consistent with standard requirements-engineering practice.

2.2.1 Usecase 1 — Register Student Account

Name	Register student account
Actors	Student (new)
Description	A prospective user creates a new account by providing email, password, and basic student information (MSSV, full name, faculty, major, cohort). The system validates the input, hashes the password, and persists both `User` and `Student` records.
Trigger	The user clicks "Register" on the login page.
Pre-conditions	The user has no existing account; email and MSSV are unique in the system.
Post-conditions	A new `User` (role = student) linked one-to-one with a `Student` record exists in the database, password stored as a bcrypt hash.
Normal Flow	1. The user opens the registration page. 2. The system displays the form (email, password, MSSV, full name, faculty, major, cohort). 3. The user enters all required fields and submits. 4. The system validates input format (email regex, password strength rules). 5. The system checks email and MSSV uniqueness in the database. 6. The system hashes the password with bcrypt and persists the `User` + `Student` records inside a transaction. 7. The system returns a JWT token and redirects the user to the student dashboard.
Alternative Flow	4a. Password fails strength rules → the system highlights specific rules violated and asks the user to retry.
Exception Flow	5a. Email or MSSV already exists → the system returns a `409 Conflict` and suggests login or password recovery. 6a. Database write fails → the system rolls back the transaction and returns "Cannot complete registration, please try again."

2.2.2 Usecase 2 — Login

Name	Login
Actors	Student, Administrator
Description	Authenticates an existing user, issues a JWT session token, and routes the client to the appropriate dashboard based on role.
Trigger	The user submits the login form on the public landing page.
Pre-conditions	The user has a registered, active account.
Post-conditions	A valid JWT (default 24-hour expiry) is stored in the client; the user is redirected to `/student/dashboard` or `/admin/dashboard` depending on role.

Normal Flow	1. The user opens the login page. 2. The user enters email and password and submits. 3. The system looks up the `User` by email. 4. The system verifies the password against the stored bcrypt hash. 5. The system signs and returns a JWT containing the user ID and role. 6. The client stores the token and redirects based on role.
Alternative Flow	2a. The user clicks "Forgot password" → proceeds to the password-recovery sub-flow (separate use case).
Exception Flow	3a. Email not found → `401 Unauthorized` "Invalid credentials." 4a. Password mismatch → `401 Unauthorized` "Invalid credentials." (Identical message to prevent account enumeration.)

2.2.3 Usecase 3 — View Personal Dashboard

Name	View personal dashboard
Actors	Student
Description	Displays the student's current academic snapshot: cumulative GPA, credits earned, credits in progress, failed-course count, current warning level, latest AI risk score, and a GPA trend chart across semesters.
Trigger	The student navigates to `/student/dashboard`.
Pre-conditions	The student is authenticated.
Post-conditions	The dashboard view is rendered with up-to-date GPA, credits-in-progress, failed-course count, and the current warning level (synchronised on every dashboard load). The AI risk panel links to the dedicated predictions endpoint, which manages its own freshness logic.
Normal Flow	1. The student loads the dashboard route. 2. The system fetches the student profile, an enrollment summary, the latest `Warning` record, and synchronises the warning level. 3. The system computes derived metrics (semester GPA series, credit progress, failed-course count). 4. The dashboard renders summary cards, the GPA trend chart, and a personalised greeting reflecting warning level and GPA.
Alternative Flow	2a. The student clicks the AI risk card → navigates to `/student/predictions`, where freshness checking and re-inference take place.
Exception Flow	2b. Database read fails → the page shows a non-blocking error state and a retry control.

2.2.4 Usecase 4 — Import Grades from myBK

Name	Import grades from myBK (paste & parse)
Actors	Student
Description	The student copies their transcript from the HCMUT myBK portal and pastes it into the system, which automatically parses the content into `Course` and `Enrollment` records and updates derived GPA and warning state.
Trigger	The student clicks "Import from myBK" on the Grades page.
Pre-conditions	The student is authenticated and possesses a transcript exported from myBK.
Post-conditions	New or updated `Enrollment` records are persisted with `source = "mybk_paste"` and `is_finalized = true`; cumulative GPA, credits earned, and warning level are recalculated.

Normal Flow	1. The student opens the import wizard. 2. The student pastes the raw transcript text and submits. 3. The parser detects the semester header pattern (e.g., "Năm học 2025-2026 / Học kỳ 1") and validates each row against the course-code regex. 4. The system displays a preview of all parsed courses with semester, grade, and credit information. 5. The student reviews the preview and confirms. 6. The system upserts `Course` and `Enrollment` records inside a transaction, recomputes cumulative GPA and credits earned, and synchronises the warning level. 7. The system confirms success and returns the student to the updated dashboard; the next visit to `/student/predictions` will recompute the AI risk score based on the new enrollment state.
Alternative Flow	2a. The student enables "Dry run" mode → the system performs steps 3–4 only, no persistence.
Exception Flow	3a. The pasted content does not match any known myBK transcript pattern → "Cannot recognize transcript format. Please copy the entire transcript from myBK." 6a. An enrollment row references a course that already has `is_finalized = true` for the same semester → the row is skipped and reported in the result summary.

2.2.5 Usecase 5 — Manage Grades Manually

Name	Manage course grades manually
Actors	Student
Description	The student creates, edits, or deletes a course enrollment by entering the course code, semester, component scores (midterm, lab, other, final) and their weights. The system computes the total score, derives the grade letter, and re-evaluates GPA.
Trigger	The student opens the Grades page and clicks "Add course" or selects an existing enrollment.
Pre-conditions	The student is authenticated; for edit/delete the enrollment must not be finalized via myBK paste.
Post-conditions	The `Enrollment` record is persisted with `source = "manual"`; cumulative GPA and warning level are recalculated in the same transaction; any existing `Prediction` record is invalidated so the next visit to the predictions page recomputes the risk score.
Normal Flow	1. The student opens the grade form. 2. The form supports common weight presets in the client (for example 30% midterm + 70% final) or accepts custom values. 3. The student enters component scores and submits. 4. The system validates that the four weight values sum to exactly 1.0 and that all scores are within [0, 10]. 5. The system computes the weighted total, derives the grade letter, and persists the enrollment. 6. Derived metrics (GPA, credits, warning state) are synchronised and the existing prediction record (if any) is invalidated within the same transaction.
Alternative Flow	3a. The student updates only the attendance rate without changing scores → steps 4–6 still execute, because attendance feeds the AI feature pipeline.
Exception Flow	4a. The weights do not sum exactly to 1.0 → the schema validator rejects the submission with a clear error. 4b. The student attempts to edit a finalized enrollment (myBK source) → the API returns `409 Conflict` explaining that finalised data is locked.

2.2.6 Usecase 6 — View AI Risk Prediction

Name	View AI risk prediction

Actors	Student
Description	The student requests the latest XGBoost-based academic risk prediction, which returns an overall risk score (0–100%), a risk level (low/medium/high/critical), the contributing factors derived from SHAP values, and per-course pass/fail probabilities for courses currently in progress.
Trigger	The student opens `/student/predictions` or clicks the risk card on the dashboard.
Pre-conditions	The XGBoost model is loaded; the student has at least one enrollment record for feature extraction.
Post-conditions	A new `Prediction` record is persisted if the previous prediction is older than the latest enrollment update; otherwise the cached prediction is returned.
Normal Flow	1. The student opens the predictions page. 2. The system checks the freshness of the most recent `Prediction` against the timestamp of the latest enrollment change. 3. If stale or absent, the system extracts the 11-feature vector (GPA deficits, low-GPA streak, unresolved failed courses, attendance risk, etc.) and invokes `PredictionService`. 4. The XGBoost model returns `predict_proba` and the SHAP explainer returns per-feature contributions. 5. The system applies the early-warning calibration layer (rule-based boost for product alignment) and persists the prediction. 6. The page renders a radial risk gauge, a bar chart of contributing factors with human-readable Vietnamese labels, a table of per-course pass probabilities, and a 30-day risk-history chart.
Alternative Flow	2a. A fresh prediction already exists (no enrollment changes since) → the system serves the cached record and skips inference. 6a. The student clicks "Refresh" → the system bypasses freshness checking and forces re-inference.
Exception Flow	3a. The model is not loaded (training has not produced a model file) → `503 Service Unavailable` "AI service initializing." 4a. Feature extraction fails due to missing data → the page shows "Insufficient academic history for prediction" with guidance to import myBK grades.

2.2.7 Usecase 7 — Simulate GPA (What-if)

Name	Simulate GPA (what-if scenarios)
Actors	Student
Description	The student enters hypothetical scores for one or more courses (currently enrolled or planned) and the system returns the projected GPA, projected warning level, and projected AI risk score without persisting any changes.
Trigger	The student clicks "Simulator" on the predictions page.
Pre-conditions	The student is authenticated; the XGBoost model is loaded.
Post-conditions	No database mutation; simulation result is returned only to the client.
Normal Flow	1. The student selects courses (existing enrollments or fictive course codes for planning) and inputs hypothetical component scores. 2. The system merges the hypothetical inputs with the student's actual enrollment data into a temporary in-memory view. 3. The system recomputes cumulative GPA and re-evaluates warning level under the simulated state. 4. The system re-extracts features from the simulated state and invokes the XGBoost model, returning a new risk score and updated SHAP contributions. 5. The page presents a side-by-side comparison: current vs. simulated GPA, current vs. simulated warning level, current vs. simulated risk.
Alternative Flow	1a. The student adjusts hypothetical scores on the simulator panel multiple times → each adjustment triggers a fresh server call but no record is persisted between runs.
Exception Flow	4a. The simulated state is inconsistent (e.g., negative score or score above 10) → the system returns the validation error inline without invoking the model.

2.2.8 Usecase 8 — Ask RAG Chatbot

Name	Ask the RAG chatbot
Actors	Student
Description	The student poses a natural-Vietnamese question about academic regulations (warning thresholds, retake rules, graduation conditions, etc.). The system retrieves the most relevant chunks from the HCMUT regulation corpus via hybrid (vector + keyword) search and asks Google Gemini to compose an answer, which is streamed back token-by-token with explicit source citations.
Trigger	The student submits a message in the chatbot interface.
Pre-conditions	The student is authenticated; the vector store contains at least one active document; the configured chat provider (Gemini by default) is reachable.
Post-conditions	The user message and the assistant response (with citations JSON) are appended to `ChatMessage` history.
Normal Flow	1. The student types a question and submits. 2. The system embeds the query using the configured embedding provider (defaulting to a deterministic hash provider, or Google's `embedding-001` when an API key is configured). 3. The system performs hybrid retrieval: cosine similarity over pgvector plus BM25-like keyword scoring with stopword filtering. 4. Top-K (default 5) document chunks are retrieved, deduplicated, and capped at 3 chunks per source file. 5. The system assembles the prompt: system instructions + retrieved chunks + recent chat-history turns + the student's personal context (name, MSSV, GPA, current warning level). 6. The system calls the configured chat provider (Gemini by default) with streaming enabled and forwards each token to the client over Server-Sent Events. 7. The system extracts citation metadata (document name, page number, snippet) and persists both the user and assistant messages.
Alternative Flow	6a. The configured chat provider is unreachable → the system falls back to the extractive provider, which composes a response from the retrieved chunks themselves.
Exception Flow	3a. Hybrid search returns no chunks → the LLM still receives the prompt with empty context and is instructed to acknowledge that the knowledge base lacks relevant information; the client renders the response without citations.

2.2.9 Usecase 9 — View Study Plan

Name	View personalized study plan
Actors	Student
Description	The system computes and presents a recommended credit load range for the upcoming semester, a prioritized list of courses to retake, and suggested elective courses to improve cumulative GPA. Recommendations are derived from the student's current GPA, warning level, and unresolved failed courses.
Trigger	The student navigates to `/student/study-plan`.
Pre-conditions	The student is authenticated and has at least one completed semester of enrollment data.
Post-conditions	The page renders the computed plan; no database mutation.

Normal Flow	1. The student opens the study plan page. 2. The system loads the student's enrollment history and applies the highest-wins rule for retaken courses. 3. The system categorizes courses into unresolved-failed and low-grade-passed sets. 4. The recommender computes a credit-load range based on GPA and warning level (e.g., GPA $< 1.5 \rightarrow$ 12–17 credits). 5. The recommender produces a retake-priority list ranked by impact on GPA and the credit weight of each course. 6. The page renders the credit-load card, retake list with priority badges, and elective suggestions.
Alternative Flow	2a. The student has no completed semester data $\rightarrow$ the page shows guidance to import grades first; recommendations are deferred.
Exception Flow	4a. The recommender encounters an unknown warning level $\rightarrow$ falls back to default "standard load" (15–21 credits).

2.2.10 Usecase 10 — View and Resolve Warning

Name	View and resolve academic warning
Actors	Student
Description	The student reviews their warning history, filters by status (unresolved/resolved), opens the detail of a specific warning, and marks it as resolved (acknowledging the notice or after meeting with an academic advisor).
Trigger	The student opens `/student/warnings`.
Pre-conditions	The student is authenticated.
Post-conditions	The selected warning's `is_resolved` flag is updated; the warning history view is refreshed.
Normal Flow	1. The student opens the warnings page. 2. The system fetches all `Warning` records for the student (newest first, capped at 50). 3. The student selects a specific warning to view details (level, semester, reason, GPA at warning time, AI risk score if any). 4. The student clicks "Mark as resolved." 5. The system updates the warning record and refreshes the list.
Alternative Flow	4a. The student undoes the resolution $\rightarrow$ the system toggles `is_resolved` back to false.
Exception Flow	2a. No warning records exist $\rightarrow$ the page shows "You currently have no academic warnings."

2.2.11 Usecase 11 — Manage Schedule

Name	Manage class timetable, exam schedule, and personal events
Actors	Student
Description	The student imports their class timetable and exam schedule from myBK (paste & parse), creates ad-hoc personal events, and views all entries across week, month, and list layouts.
Trigger	The student opens `/student/schedule` or clicks an import action.
Pre-conditions	The student is authenticated.
Post-conditions	The relevant JSON columns on the `Student` record (`timetable_json`, `exam_schedule_json`, `personal_events_json`) are updated; the calendar view reflects the changes immediately.

Normal Flow	1. The student opens the schedule page. 2. The system loads existing timetable, exam, and personal-event JSON payloads. 3. The student chooses an action: import timetable, import exam schedule, or create a personal event. 4. For imports: the relevant parser (`tkb_parser` or `exam_schedule_parser`) extracts session/exam data and the system stores the structured payload. 5. For personal events: the system appends to the `personal_events_json` array with title, date/time, and reminder settings. 6. The calendar re-renders to show the updated state.
Alternative Flow	3a. The student re-imports a timetable for the same semester → the system overwrites the previous `timetable_json` payload entirely, ensuring sessions stay in sync with myBK without per-session deletion.
Exception Flow	4a. The pasted timetable does not match the expected pattern → the import is rejected with a hint about the supported source format.

2.2.12 Usecase 12 — Approve Pending Warning

Name	Approve pending academic warning
Actors	Administrator
Description	The administrator reviews warnings the system has suggested (either from regulation-based rules or AI risk scores exceeding the threshold), inspects the supporting evidence (GPA, history, risk factors), and approves the warning to be officially issued and dispatched to the student via notification and email.
Trigger	The administrator navigates to `/admin/warnings` and selects a pending entry.
Pre-conditions	The administrator is authenticated with the admin role; at least one warning is in the pending state.
Post-conditions	The warning's status changes to approved; a `Notification` record is created and an email is dispatched (fire-and-forget) to the student.
Normal Flow	1. The administrator opens the pending-warnings list. 2. The system displays the queue: AI-suggested warnings (risk above threshold) plus regulation-based warnings awaiting review. 3. The administrator selects a specific warning to inspect the detail panel (student info, semester GPA, cumulative GPA, AI risk factors, prior warning history). 4. The administrator clicks "Approve and send." 5. The system updates the warning status, creates a `Notification`, and queues the email through `email_service.fire_and_forget`. 6. The interface confirms "Warning sent to student."
Alternative Flow	3a. The administrator creates a manual warning outside the AI queue → proceeds directly to the warning-issuance step with the administrator-entered reason via a dedicated manual-creation endpoint. 4a. The administrator leaves a suggestion in the pending queue (no approval) → no warning is issued; the suggestion remains visible for later review or re-evaluation on the next batch run.
Exception Flow	5a. Email dispatch fails (SMTP error) → the warning and `Notification` are still committed; the failure is logged but does not block the API response.

2.2.13 Usecase 13 — Bulk Import Excel Data

Name	Bulk import students or grades from Excel
Actors	Administrator
Description	The administrator uploads an Excel file (XLSX) containing either a list of new students or a batch of grades. The system validates each row, reports row-level errors, and commits valid rows in a single transaction. The administrator can subsequently invoke the batch-warning evaluation endpoint to refresh warning states for affected semesters.
Trigger	The administrator uploads a file on `/admin/import`.

Pre-conditions	The administrator is authenticated with the admin role; the uploaded file matches the published template schema.
Post-conditions	Valid rows result in new or updated `User+Student` records (for student imports) or `Course+Enrollment` records (for grade imports); the import history (most recent runs) is updated.
Normal Flow	1. The administrator selects the import type (Students or Grades) and uploads the file. 2. The system parses the workbook and validates each row against the schema (required columns, format, uniqueness for MSSV/email). 3. The system displays a preview with per-row pass/fail status. 4. The administrator confirms. 5. The system commits the valid rows in a single transaction and records the run in the import history. 6. The interface shows the import summary (created, updated, failed counts). 7. The administrator may optionally trigger the batch warning-evaluation endpoint to refresh warning states using the freshly-imported grades.
Alternative Flow	1a. The administrator first downloads the Excel template from `/admin/import/templates/...` to ensure the correct schema.
Exception Flow	2a. The file is not a valid XLSX or exceeds the size limit → `400 Bad Request` with the specific reason. 5a. A unique-constraint violation occurs at commit time → the transaction is rolled back and the failing row is reported.

2.2.14 Usecase 14 — Manage RAG Documents

Name	Manage the RAG document repository
Actors	Administrator
Description	The administrator uploads, lists, deactivates, or deletes documents that the RAG chatbot uses as its knowledge base. Uploaded files (PDF, DOCX, TXT, MD; ZIP archives are unpacked) are automatically text-extracted (with OCR fallback for scanned PDFs), chunked, embedded via the configured embedding provider, and stored in pgvector.
Trigger	The administrator visits `/admin/documents` or uploads a new file.
Pre-conditions	The administrator is authenticated with the admin role.
Post-conditions	New `Document` rows (one per chunk) are persisted with their 768-dimension embedding vector; chatbot queries immediately benefit from the new content.
Normal Flow	1. The administrator opens the document management page. 2. The system displays the existing documents grouped by source file with active/inactive status. 3. The administrator uploads one or more files (or a ZIP). 4. For each file: the system extracts text (falling back to OCR at 220 DPI for image-based PDFs), normalizes whitespace, and splits into overlapping chunks (default 800 characters, 120-character overlap). 5. The system embeds each chunk with the configured embedding provider (Gemini by default) and inserts a `Document` row per chunk with metadata (filename, source_file, chunk_index, page_number). 6. The interface confirms successful ingestion and shows the new chunk count.
Alternative Flow	2a. The administrator toggles an existing document to inactive → its chunks are excluded from future retrieval without being deleted.
Exception Flow	4a. The PDF contains neither extractable text nor OCR-able pages → the file is rejected with a descriptive message. 5a. The embedding provider is unreachable → the system falls back to the hash-based embedding provider and logs a warning.

2.2.15 Usecase 15 — Create Academic Event

Name	Create and manage academic events

Actors	Administrator
Description	The administrator publishes academic events (exam dates, registration deadlines, evaluation periods, university activities) targeted at specific audiences (all students, a particular faculty, or a particular cohort), optionally flagged as mandatory. Mandatory events trigger automatic 24-hour reminder notifications to the target audience via the scheduler.
Trigger	The administrator opens the event editor on `/admin/events`.
Pre-conditions	The administrator is authenticated with the admin role.
Post-conditions	A new `Event` record is persisted; matching students see the event in their calendar; if mandatory, the next scheduler tick will pick it up for reminder dispatch within the 23–25 hour window.
Normal Flow	1. The administrator opens the event editor. 2. The administrator enters event metadata: title, description, type (exam/submission/activity/evaluation), start/end time, target audience (all/faculty/cohort), and mandatory flag. 3. The system validates the input (start time before end time, target value required if audience is faculty- or cohort-specific). 4. The system persists the event with the current administrator as the creator. 5. The interface confirms creation and lists the event in the upcoming-events panel.
Alternative Flow	2a. The administrator edits or deletes an existing event $\rightarrow$ the system applies the same validation rules and updates the record.
Exception Flow	3a. The target audience is "faculty_specific" but no faculty name is provided $\rightarrow$ the form rejects the submission with a clear field-level error.

2.3 Activity Diagrams

This part presents twelve activity diagrams covering the user-facing flows and the automated background jobs implemented in the system. Each diagram is drawn with two or three swim-lanes (the actor, the backend, and an external service where relevant) and uses the standard UML activity notation: a filled start circle, decision diamonds with explicit branch labels, rounded rectangles for activities, and a bullseye end node. Descriptions accompanying simpler authentication and CRUD flows are kept short; the three AI-specific flows (risk prediction, GPA simulator, RAG chatbot) and the scheduler-driven jobs receive fuller treatment because they involve multi-stage pipelines that are not obvious from the use-case specifications alone.

2.3.1 1. Register Student Account

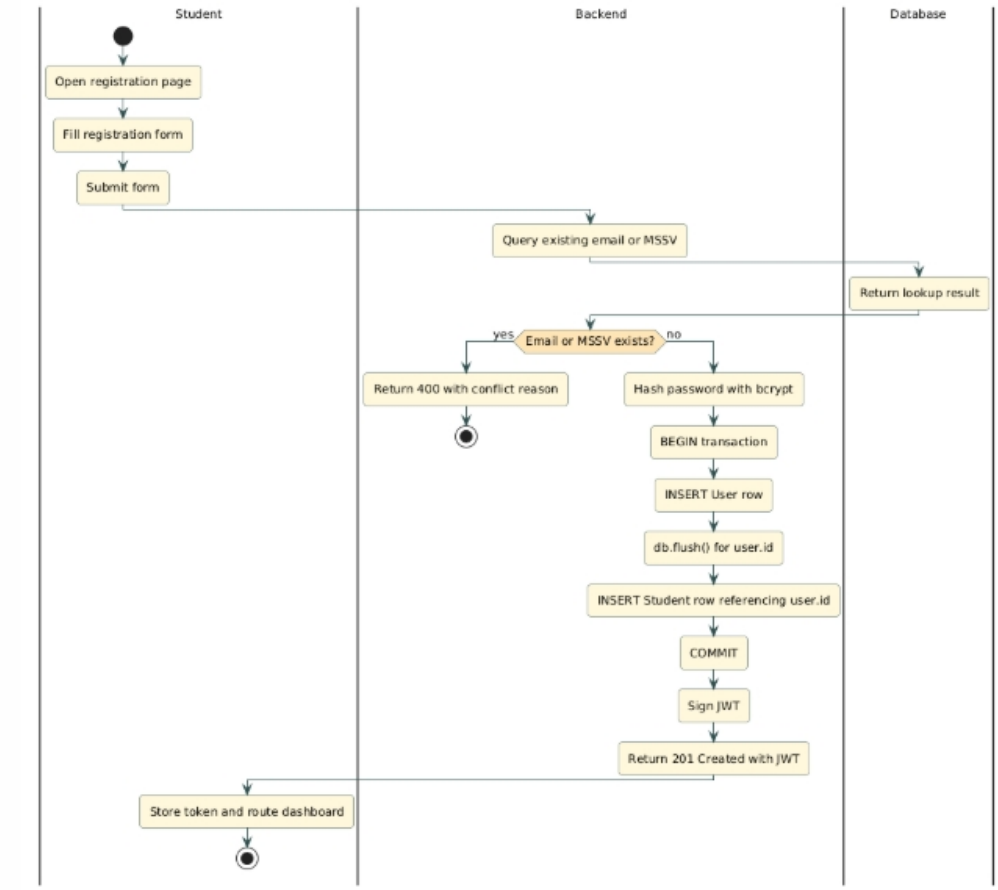

Figure 2: *Activity diagram — Register student account*

The student opens the registration page, fills in the form (email, password, MSSV, full name, faculty, major, cohort), and submits. The backend checks email and MSSV uniqueness; on conflict it returns `400 Bad Request` with the specific reason. Otherwise it hashes the password with bcrypt, persists the `User` and `Student` rows in the same transaction (with `db.flush` between them to obtain `user.id` for the foreign key), issues a JWT, and returns it as `201 Created`. The frontend stores the token and routes the new student to the dashboard.

2.3.2 2. Login

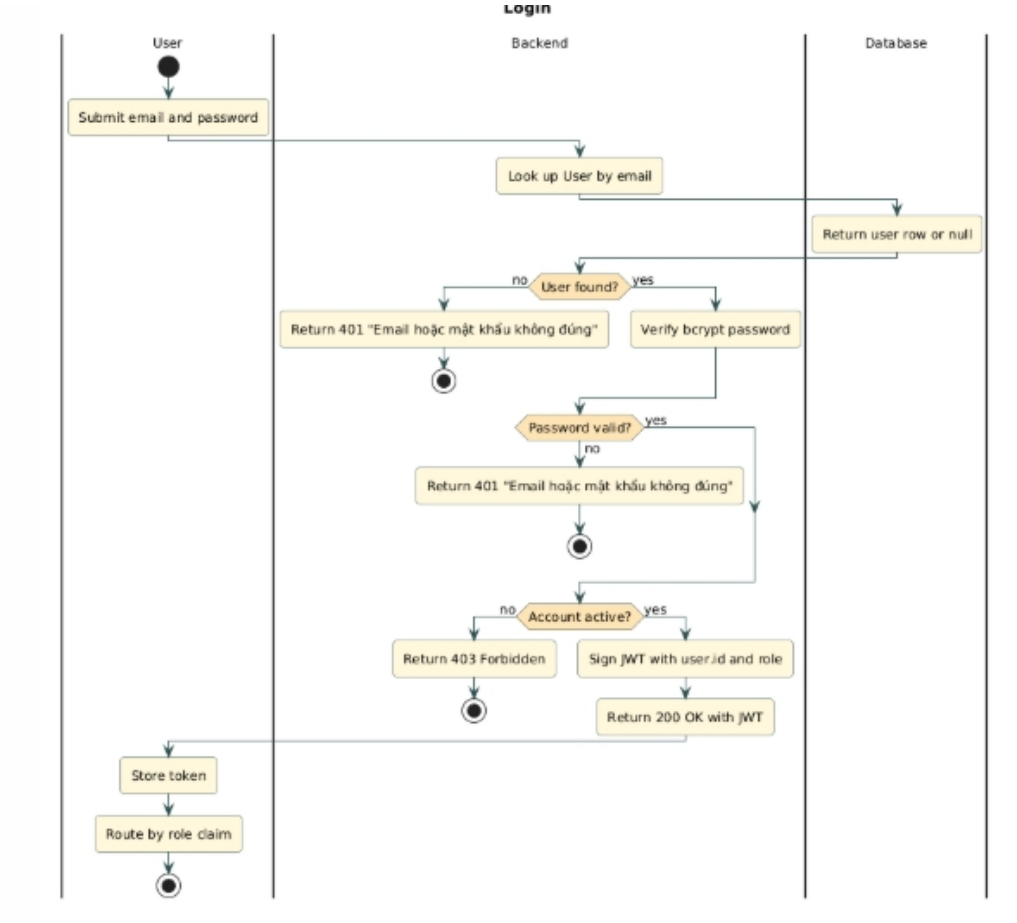

Figure 3: *Activity diagram — Login*

The user submits email and password. The backend looks up the `User` by email and verifies the password against the bcrypt hash. Three outcomes are possible: unknown email or password mismatch returns `401 Unauthorized` with the identical message "Email hoặc mật khẩu không đúng" to prevent account enumeration; an inactive account returns `403 Forbidden`; on success the backend signs a JWT carrying the user's ID and role and returns it. The frontend stores the token and routes the user to either `/student/dashboard` or `/admin/dashboard` according to the role claim.

2.3.3 3. Import Grades from myBK

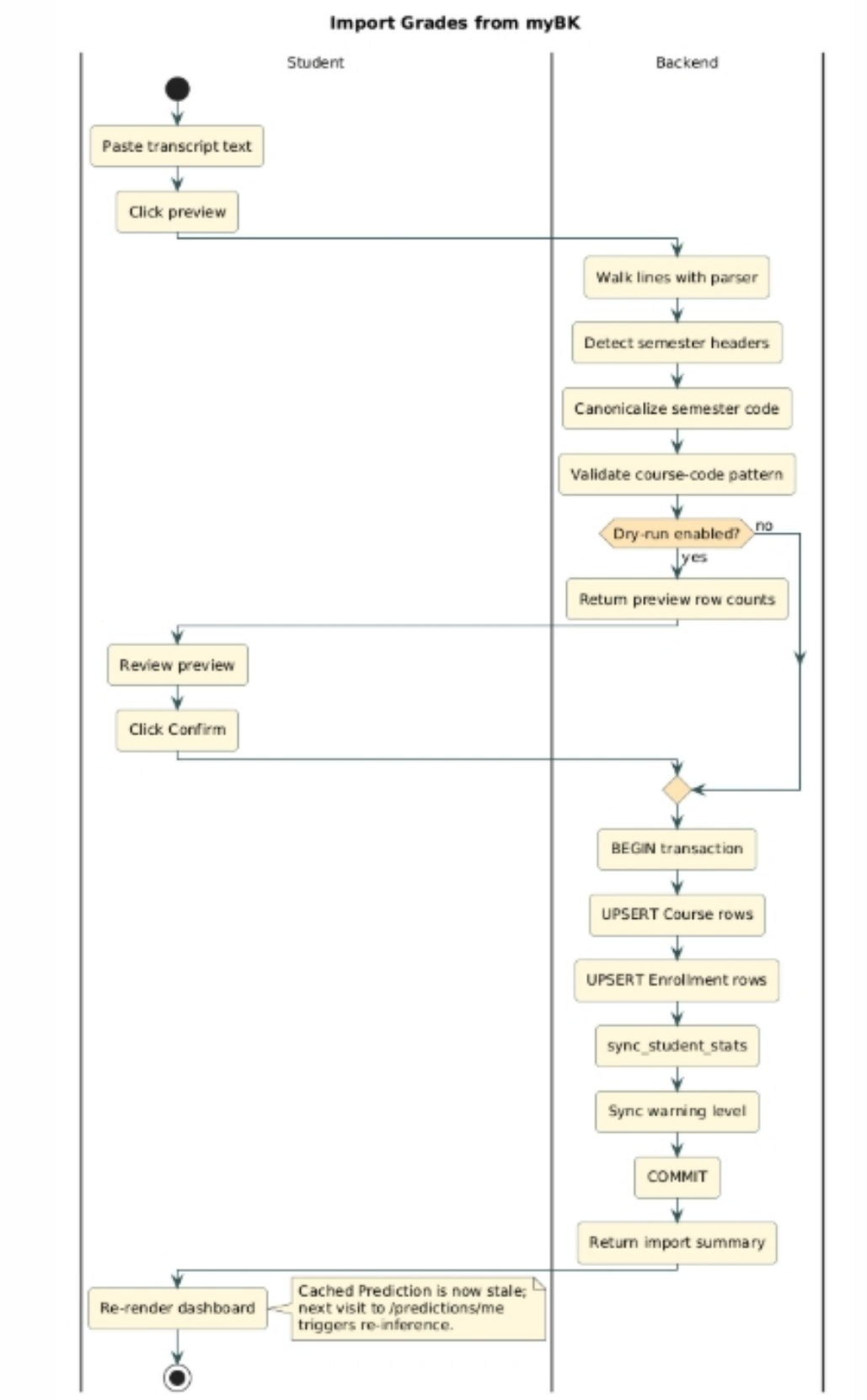

Figure 4: *Activity diagram — Import grades from myBK*

The student pastes the transcript text into the import wizard. The parser walks the lines, detecting semester headers via a regular expression that turns “Năm học 2025-2026 / Học kỳ 1” into the canonical semester code `251`, then validates each row against the course-code pattern. Rows that do not match are skipped and listed in the

result summary. When the user enables the *dry-run* toggle, the flow stops at the preview step. On confirmation, the backend upserts `Course` and `Enrollment` rows inside a single transaction, marks every newly-created enrollment with `source = "mybk_paste"` and `is_finalized = true`, and synchronises the student's cumulative GPA, credits earned, and warning level. The next visit to the predictions page will trigger a fresh AI inference because the cached `Prediction` record is now older than the most recent enrollment.

2.3.4 4. AI Risk Prediction

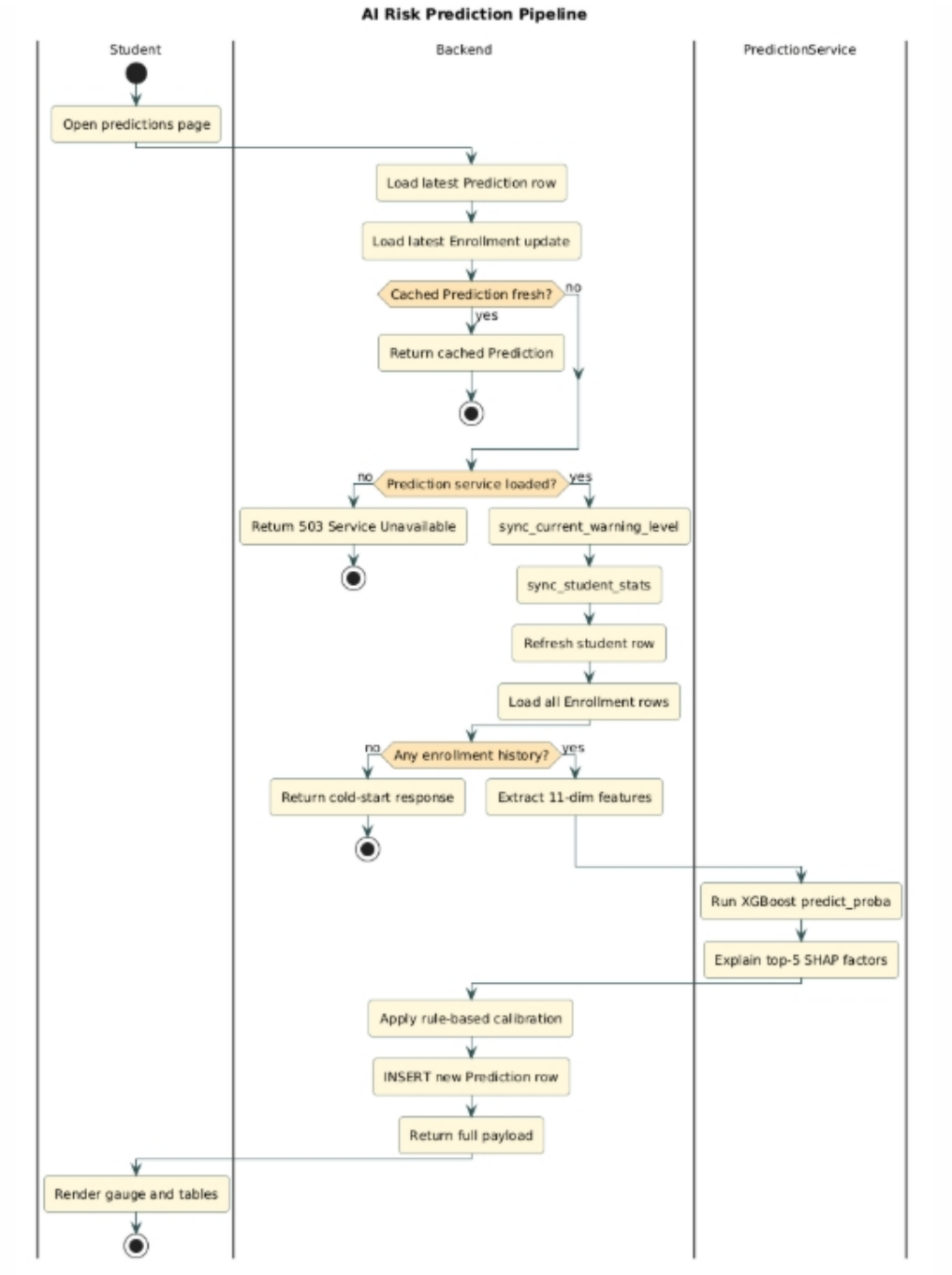

Figure 5: *Activity diagram — AI risk prediction pipeline*

The risk-prediction flow is the most layered student-facing activity. When the student opens the predictions page, the backend first checks freshness by comparing the timestamp of the most recent `Prediction` record against the latest `Enrollment` update. If a fresh prediction already exists the cached result is returned and no inference takes place. Otherwise the pipeline runs end-to-end:

1. Sync the current warning level so the feature extractor sees up-to-date status.

2. Sync `gpa_cumulative` and `credits_earned` via `sync_student_stats`, then refresh the in-memory student row.
3. Load all enrollments (with their courses eagerly loaded) and bail out with a "cold-start" response if the student has no enrollment history at all.
4. Extract the 11-feature vector through `extract_features` and call `predict_proba` on the XGBoost model.
5. Pass the same feature vector to the `RiskExplainer`, which produces the top-five SHAP contributions, filters out low-impact or contradictory signals, normalises the surviving impacts to sum to 100%, and labels each with a Vietnamese phrase.
6. Apply the rule-based early-warning calibration layer that boosts the raw probability when established product rules indicate the score should be higher (for example when the cumulative GPA is below 1.2).
7. Persist a new `Prediction` row carrying the risk score, the discrete risk level, the SHAP factor list and the per-course pass probabilities, then return the response.

If the model file has not yet been trained (no compatible model on disk, or the feature names disagree with the current code's `FEATURE_NAMES`), the endpoint short-circuits with `503 Service Unavailable`.

2.3.5 5. GPA What-if Simulator

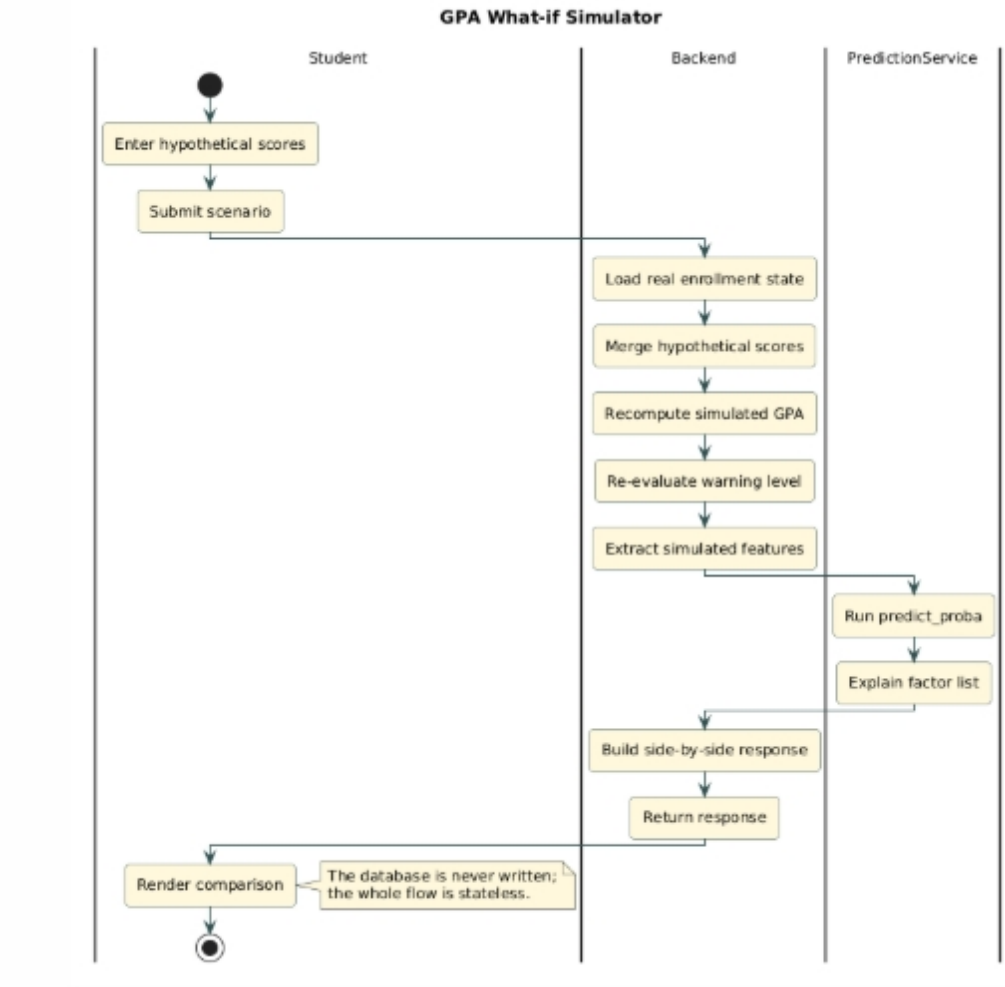

Figure 6: *Activity diagram — GPA what-if simulator*

The simulator reuses the prediction pipeline against a temporary in-memory state. The student supplies hypothetical scores for one or more courses (existing or planned); the backend merges those inputs with the real enrollment data into a transient view, recomputes the simulated cumulative GPA, re-evaluates the warning level under the simulated state, extracts a fresh feature vector, and runs `predict_proba` plus `RiskExplainer` on it. The response carries a side-by-side comparison of current and simulated GPA, warning level, and risk score. Because the entire flow writes nothing to the database, the student can iterate quickly through scenarios.

2.3.6 6. RAG Chatbot Q&A

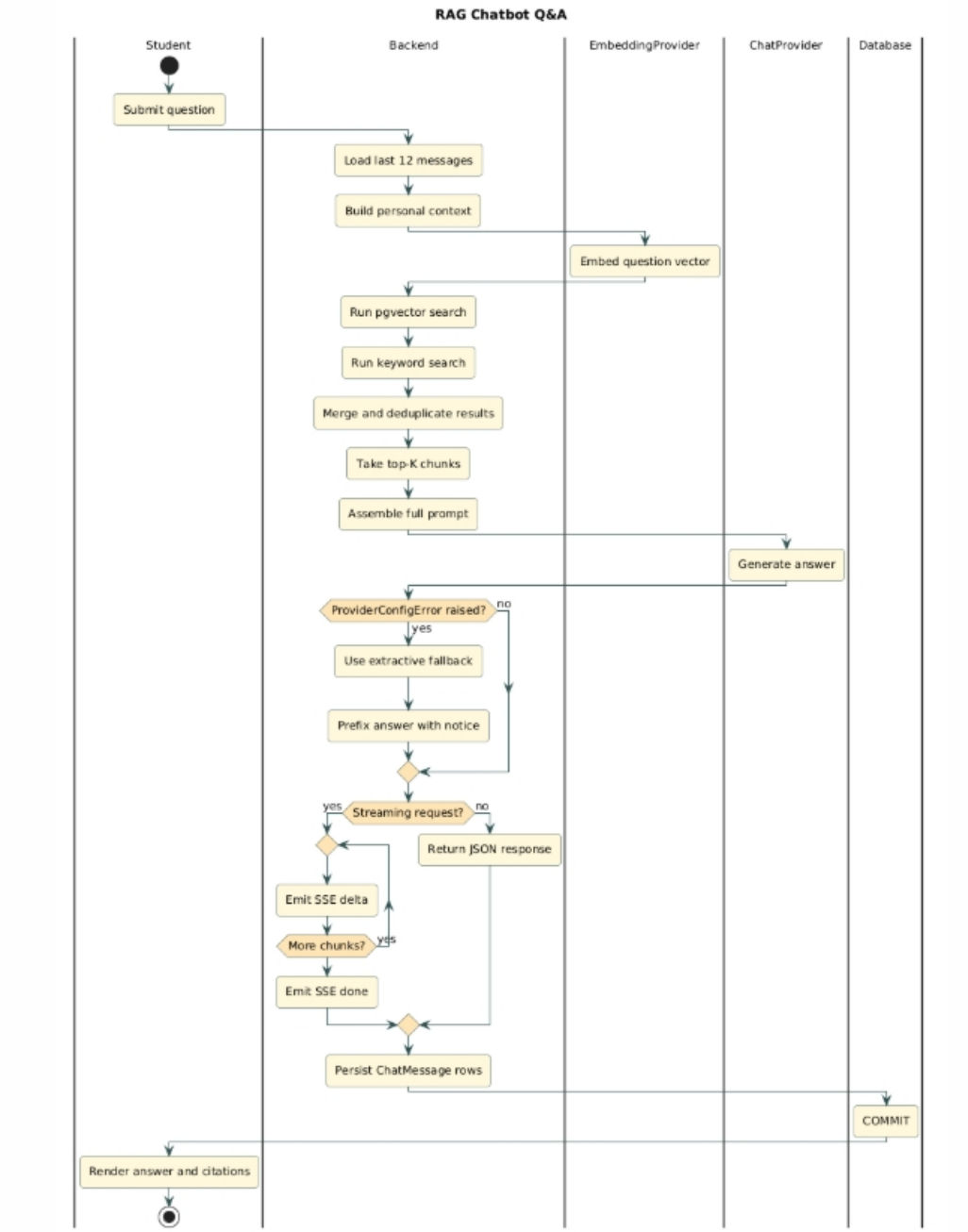

Figure 7: *Activity diagram — RAG chatbot question answering*

The chatbot activity orchestrates four sub-systems — conversation memory, retrieval, prompt assembly, and the chat provider — behind a single user-visible message exchange. After the student submits a question, the backend loads the twelve most recent `ChatMessage` rows for context, builds the personal-context string from the student's profile (name, MSSV, GPA, warning level, failed-course summary), and runs a hybrid retrieval against the `Document` table. The hybrid retrieval combines pgvector cosine similarity with a BM25-style keyword scorer using stopword filtering, returns the top-K results (default 5), deduplicates them by document, and caps any single source file to three chunks so a long document cannot monopolise the context window. The citations are formatted into a numbered list and concatenated with the system instructions, the recent history, and the personal context into a single prompt. The active chat provider — Gemini by default, with the extractive provider as the universal fallback when `ProviderConfigError` is raised — generates the answer. For streaming requests the backend forwards each chunk emitted by the provider as a `data: {"type":"delta",...}` Server-Sent Event, then emits a `"done"` event carrying the citation array and the provider name. Whether streamed or not, the final user message and assistant response (with citations JSON) are appended to `ChatMessage` history within the same database session.

2.3.7 7. View and Resolve Warning

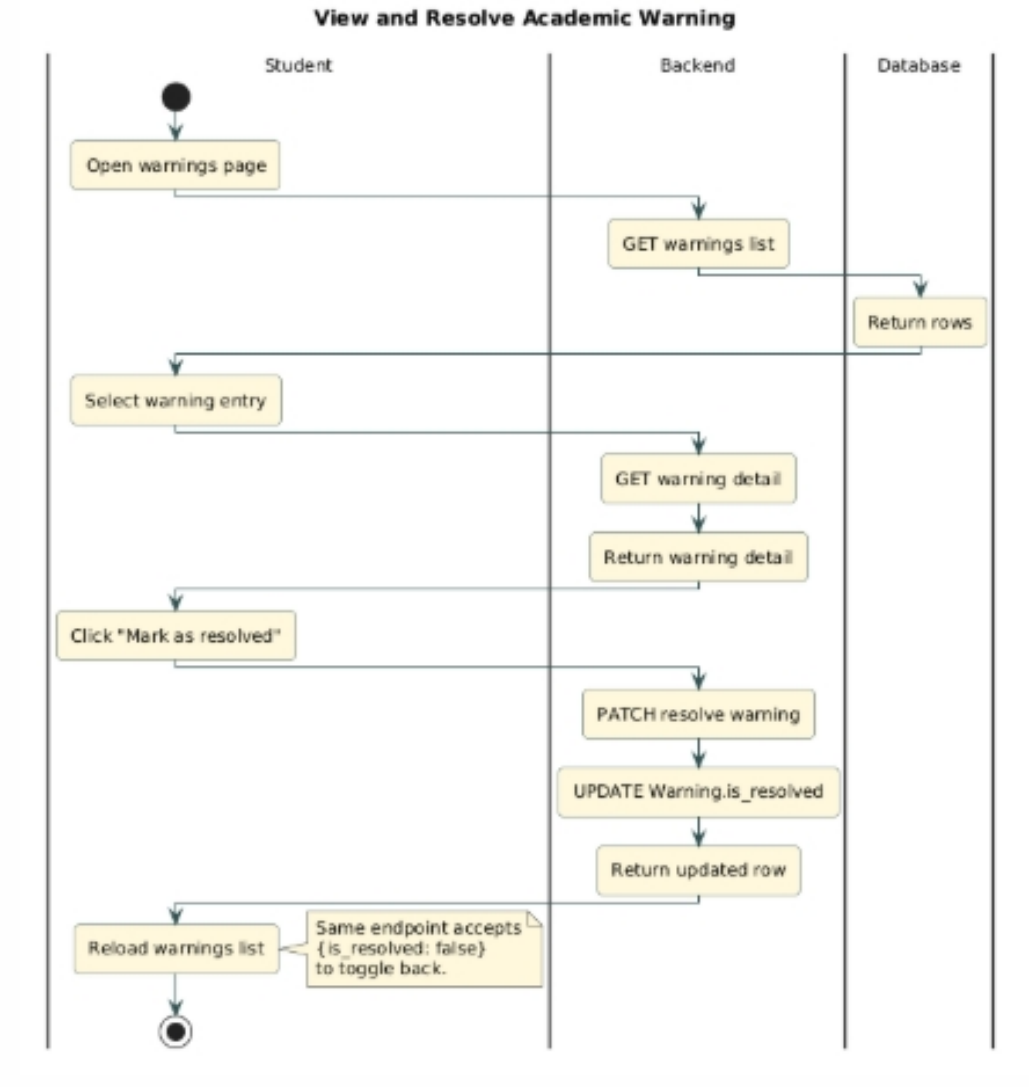

Figure 8: *Activity diagram — View and resolve academic warning*

The student opens the warnings page; the backend loads the up to fifty most recent `Warning` rows for the student, newest first. The student selects a warning to view its details (level, semester, reason, GPA at warning time, AI risk score when applicable) and clicks *Mark as resolved*; the `PATCH /warnings/me/{id}/resolve` endpoint flips the `is_resolved` flag, and the list is reloaded. Because the endpoint accepts any boolean value, the same path supports toggling back to unresolved.

2.3.8 8. Manage Schedule (Timetable / Exams / Personal Events)

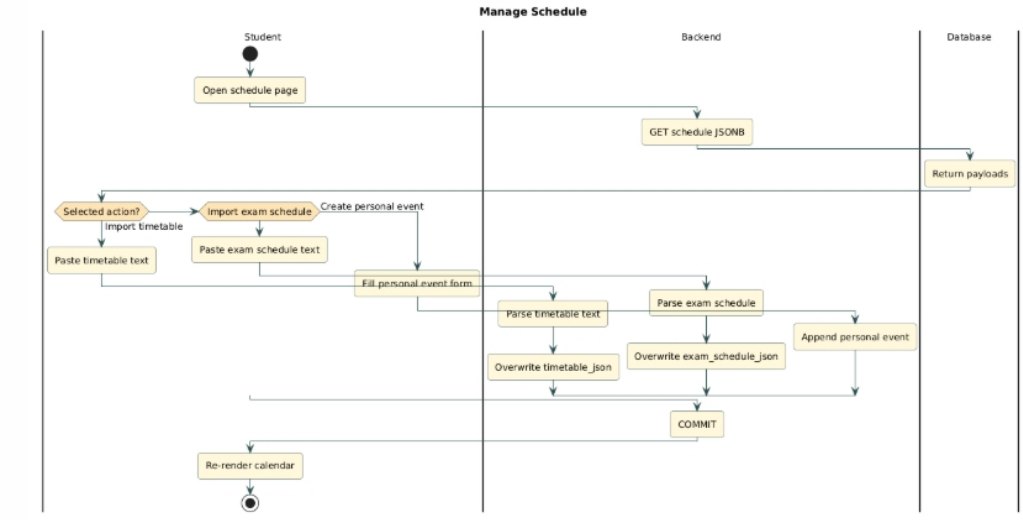

Figure 9: *Activity diagram — Manage class timetable, exam schedule and personal events*

The schedule page loads the three JSONB payloads on the student record (`timetable_json`, `exam_schedule_json`, `personal_events_json`) and lets the student choose an action. Importing a timetable or exam schedule routes the pasted text through `tkb_parser.parse` or `exam_schedule_parser.parse` respectively; the parsed structured payload overwrites the corresponding JSON column entirely, so re-importing for the same semester naturally replaces stale sessions. Creating a personal event appends a new entry to `personal_events_json`. The calendar component re-renders immediately after the update.

2.3.9 9. Admin — Bulk Import from Excel

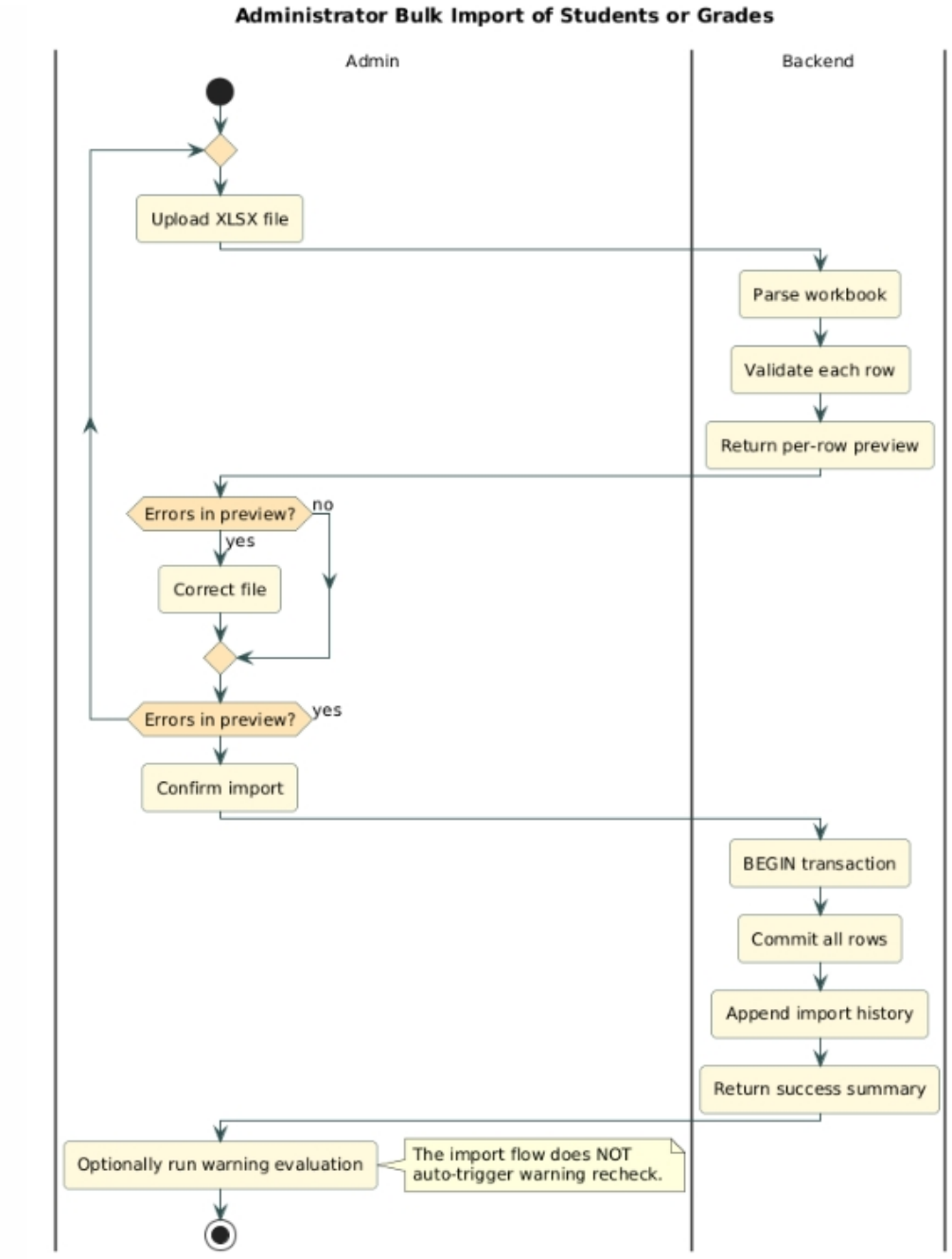

Figure 10: *Activity diagram — Administrator bulk import of students or grades*

The administrator uploads an XLSX file under either the *students* or *grades* import type. The backend parses the workbook through `import_service`, validates every row against the schema (required columns, MSSV/email format, uniqueness), and returns a per-row preview with pass/fail status. If validation reveals errors the administrator corrects the file and re-uploads; otherwise the rows are committed in a single transaction and the run is appended to the in-process import-history list. The administrator can subsequently invoke the batch warning-evaluation endpoint to refresh warning states using the freshly-imported grades — the import flow itself does not trigger that re-check automatically.

2.3.10 10. Admin — Batch Risk Prediction

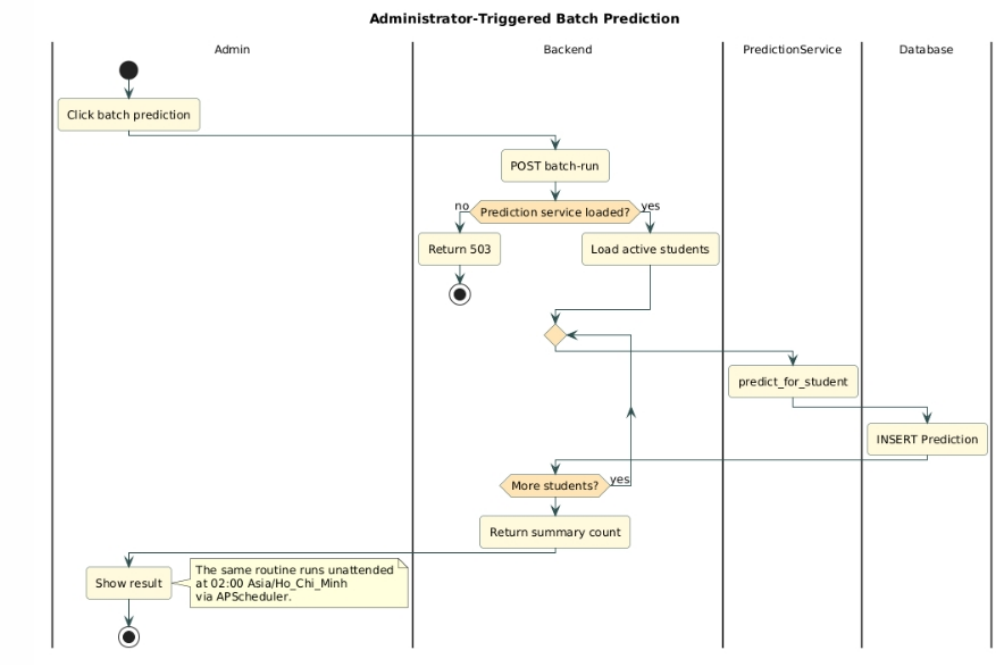

Figure 11: *Activity diagram — Administrator-triggered batch prediction*

The administrator clicks “Run batch prediction” on the warnings page. The backend checks that `prediction_service.is_loaded` is true (returning `503` otherwise), then iterates over every active student, calling `predict_for_student` for each one and persisting a fresh `Prediction` row. The same logic runs unattended every day at 02:00 (Asia/Ho_Chi_Minh) via the `predictions_batch_daily` job described in subsection 2.3.12. The administrator endpoint exists primarily to refresh predictions on demand after a large grade import without waiting for the next scheduled run.

2.3.11 11. Admin — Approve Pending Warning

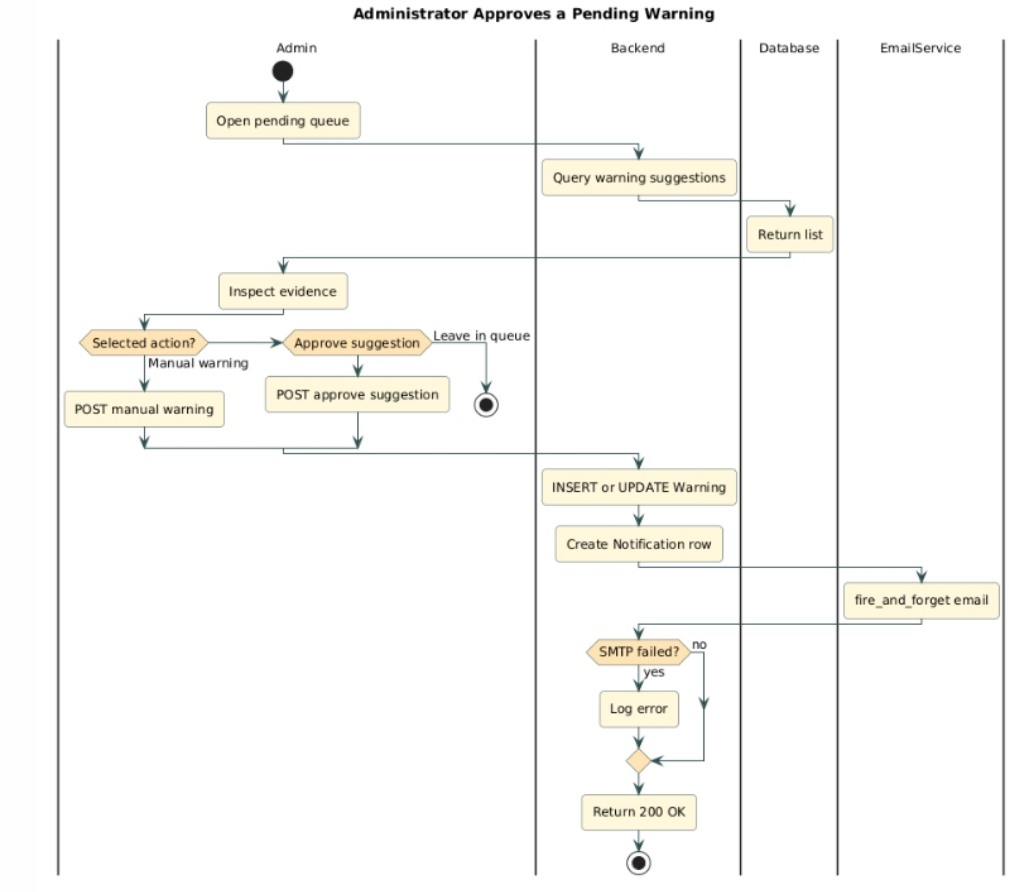

Figure 12: *Activity diagram — Administrator approves a pending warning*

The administrator opens the pending-warnings queue, which mixes regulation-based suggestions emitted by `warning_engine` and AI-driven suggestions whose risk score exceeds the configurable threshold. After inspecting the supporting evidence (semester GPA, cumulative GPA, AI risk factors, prior warning history), the administrator either issues a manual warning via the dedicated `POST /admin/warnings/manual` endpoint, approves a suggestion via `POST /admin/warnings/approve`, or leaves the suggestion in the queue for the next review. On approval the system creates a `Notification` for the student and dispatches the corresponding email through `email_service.fire_and_forget`; SMTP failures are logged but do not block the API response.

2.3.12 12. Notification Scheduler (Cron)

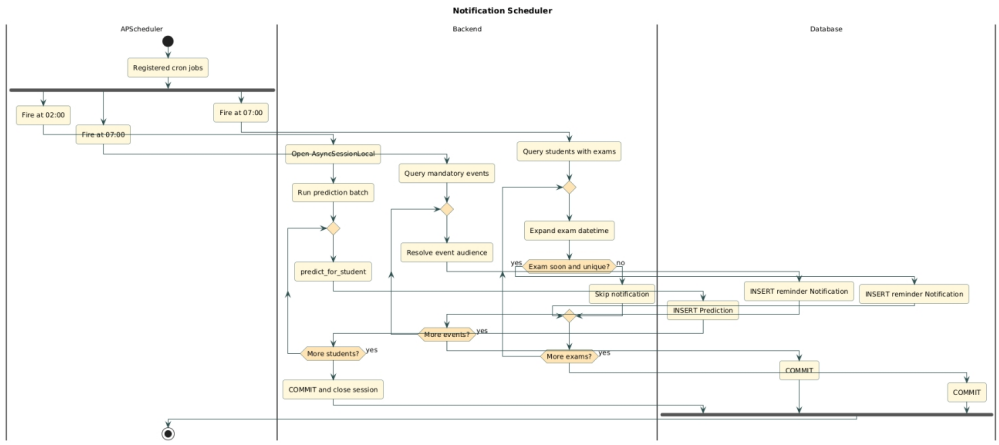

Figure 13: *Activity diagram — Three in-process scheduler jobs*

The scheduler runs in the same process as the API server using APScheduler with the Asia/Ho_Chi_Minh timezone, and registers three cron jobs. The `predictions_batch_daily` job fires at 02:00 and calls `prediction_service.predict_batch` for every non-synthetic student. The `deadline_reminders_daily` job fires at 07:00, scans the `events` table for mandatory events whose `start_time` falls within a 23–25 hour window from now, resolves the matching students according to the event's `target_audience` (`all`, `faculty_specific`, or `cohort_specific` keyed off `target_value`), and creates a `Notification` of type `reminder` for each one. The `exam_reminders_daily` job also fires at 07:00; it iterates over all students with a non-null `exam_schedule_json`, expands each exam entry into a UTC datetime, and creates a `reminder` notification for exams falling within the next 25 hours, deduplicating against notifications with the same title that have already been sent in the last 23 hours.

2.4 Sequence Diagrams

This part presents eight sequence diagrams that complement the activity views by showing the temporal interactions among lifelines (the actor, the frontend client, the backend API, the database, the AI service classes, the vector store, and external providers). Each diagram uses the standard UML notation: stick-figure actor, lifeline rectangles with dashed timelines, activation bars on the receiving lifeline while a call is in progress, solid arrows for synchronous calls, dashed arrows for returns, and `alt`/`opt`/`loop` fragments to express conditional branches. As with the activity diagrams, simple flows are described briefly and the AI-heavy interactions (prediction, simulator, RAG chatbot, batch jobs) are described in more detail because their distributed step ordering is not immediately apparent from reading the code.

2.4.1 1. Authentication (Register + Login)

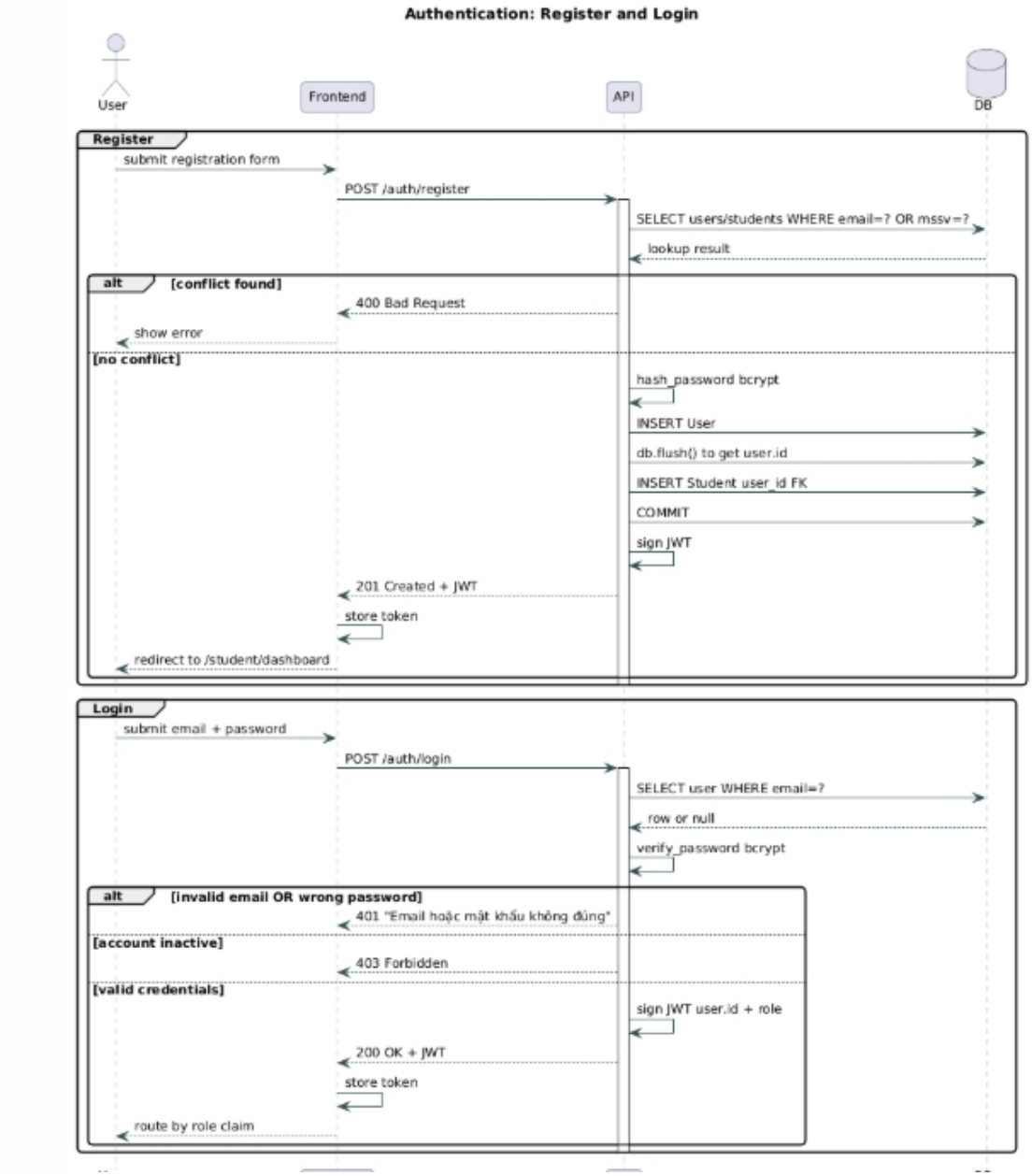

Figure 14: *Sequence diagram — Register and Login*

Lifelines: User, Frontend, API, Database.

The Register branch begins with the user submitting the registration form. The API queries the `users` and `students` tables to check for email and MSSV collisions; on conflict it responds with `400 Bad Request` and the flow ends. Otherwise the API calls `hash_password`, inserts the `User` row, flushes to obtain the new ID, inserts the `Student` row referencing that ID, commits, signs a JWT and returns it as `201 Created`. The Login branch is shorter: lookup by email, `verify_password` against the stored bcrypt hash, then either return `401` for invalid

credentials (with an identical message for unknown email and wrong password), 403 for an inactive account, or 200 with a fresh JWT. The frontend persists the token and routes the user based on the role claim it contains.

2.4.2 2. Import myBK Grades and Cache Invalidation

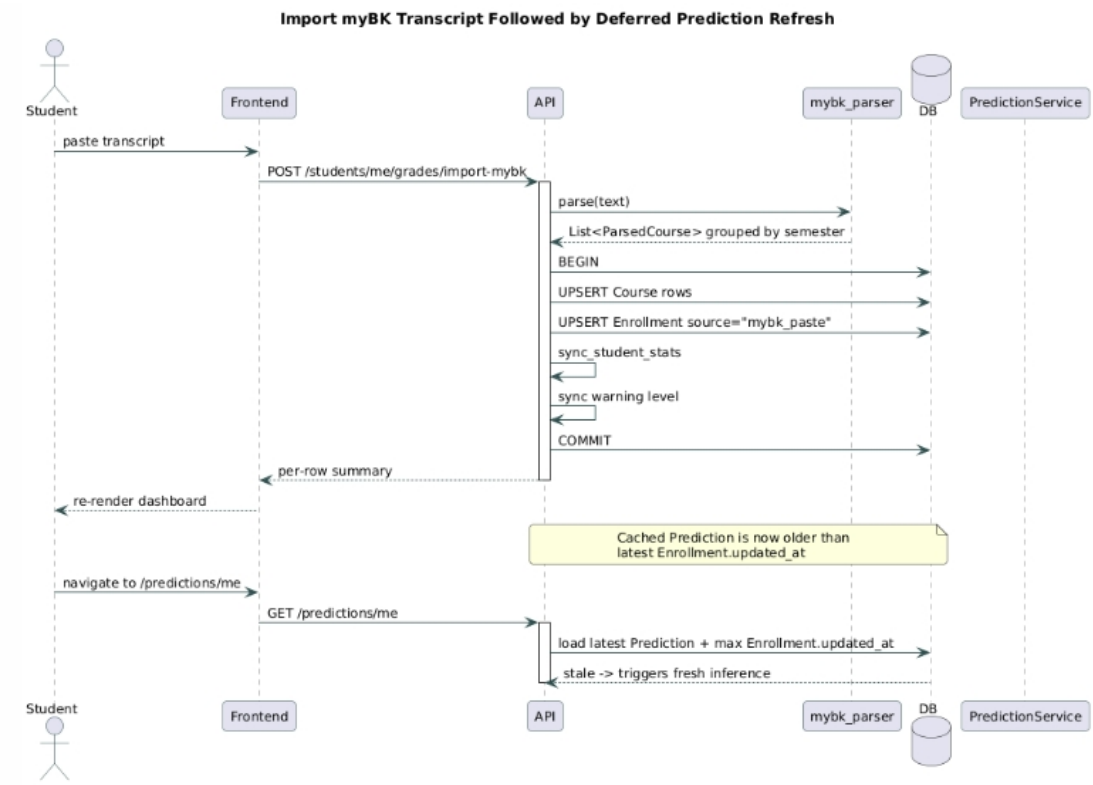

Figure 15: *Sequence diagram — Import myBK transcript followed by deferred prediction refresh*

Lifelines: Student, Frontend, API, Parser, Database, PredictionService.

The student pastes the transcript and the frontend POSTs it to `/students/me/grades/import-mybk`. The API delegates to the `mybk_parser` module to extract a list of `ParsedCourse` records grouped by semester. Inside a single transaction the API upserts the corresponding `Course` rows (creating placeholders for unseen codes), upserts the `Enrollment` rows with `source = "mybk_paste"` and `is_finalized = true`, recomputes derived statistics via `sync_student_stats`, and synchronises the warning level. The API commits and returns a per-row summary. The frontend re-renders the dashboard. Crucially, the cached `Prediction` record is now older than the latest `Enrollment.updated_at`; the next visit to `/predictions/me` detects this staleness and triggers a fresh inference (the dedicated sequence in subsection 2.4.3).

2.4.3 3. AI Risk Prediction

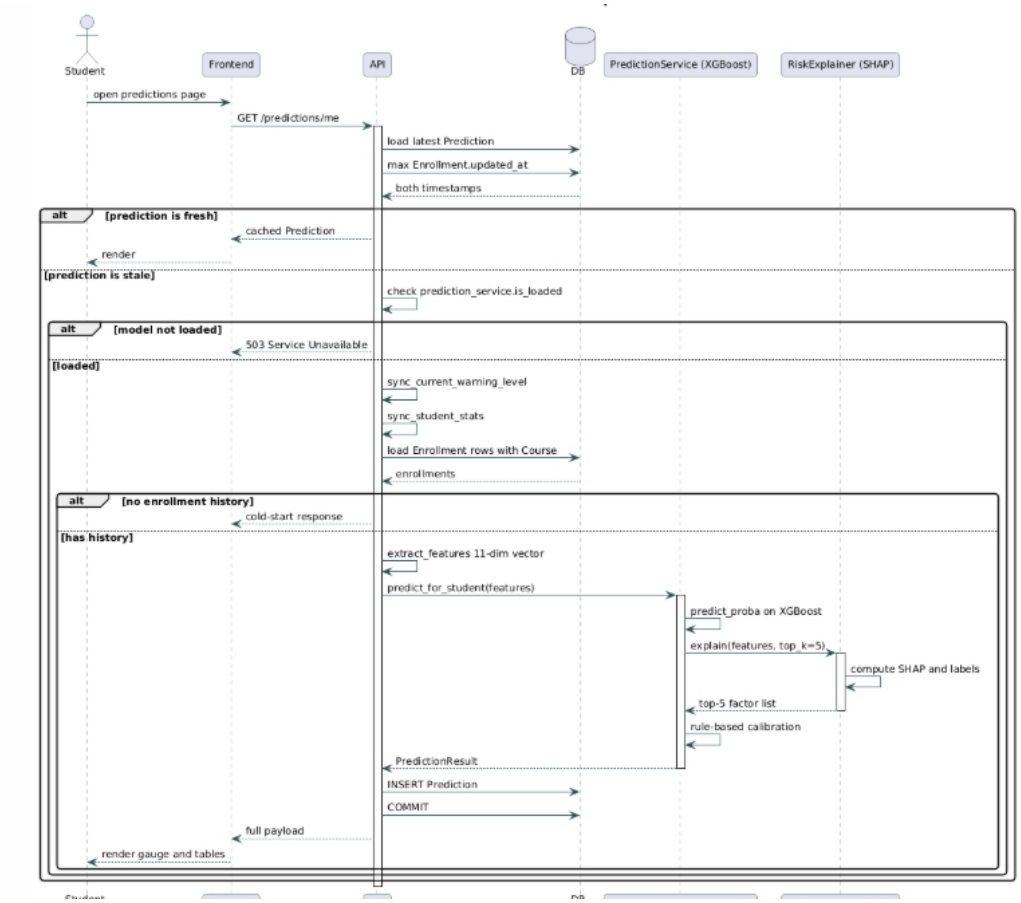

Figure 16: *Sequence diagram — End-to-end risk prediction pipeline*

Lifelines: Student, Frontend, API, Database, PredictionService (XGBoost), RiskExplainer (SHAP).

The student opens the predictions page; the frontend issues `GET /predictions/me`. The API loads the most recent `Prediction` row and compares its `created_at` against the maximum `Enrollment.updated_at` for the student. An `alt` fragment splits two branches: when the cached prediction is fresh the API returns it immediately and the pipeline terminates; otherwise the staleness branch executes the full inference loop. In the staleness branch the API first calls `warning_engine.sync_current_warning_level` and `sync_student_stats` to make sure the student's GPA, credits and warning level are current before features are extracted (this prevents the model from seeing a stale cold-start state). The API then fetches all enrollments with their courses eagerly loaded, calls `extract_features` to assemble the 11-dimensional feature vector, and invokes `PredictionService.predict_for_student`. Internally, `predict_for_student` runs `predict_proba` on the XGBoost classifier and passes the same vector to `RiskExplainer.explain`, which returns the top-five normalised SHAP contributions with Vietnamese labels. The rule-based early-warning calibration layer then adjusts the raw probability when domain conditions warrant it (for example boosting the score to at least 0.78 when the cumulative GPA is below 1.2). The API inserts a new `Prediction` row carrying the calibrated risk score, the risk level, the SHAP factor list and the per-course pass probabilities, then returns the full payload to the frontend, which renders the radial gauge, factor list, and per-course table. If at any point the model is not loaded (no compatible model file, or feature names disagree with the current `FEATURE_NAMES`) the API short-circuits with `503 Service Unavailable`.

2.4.4 4. GPA What-if Simulator

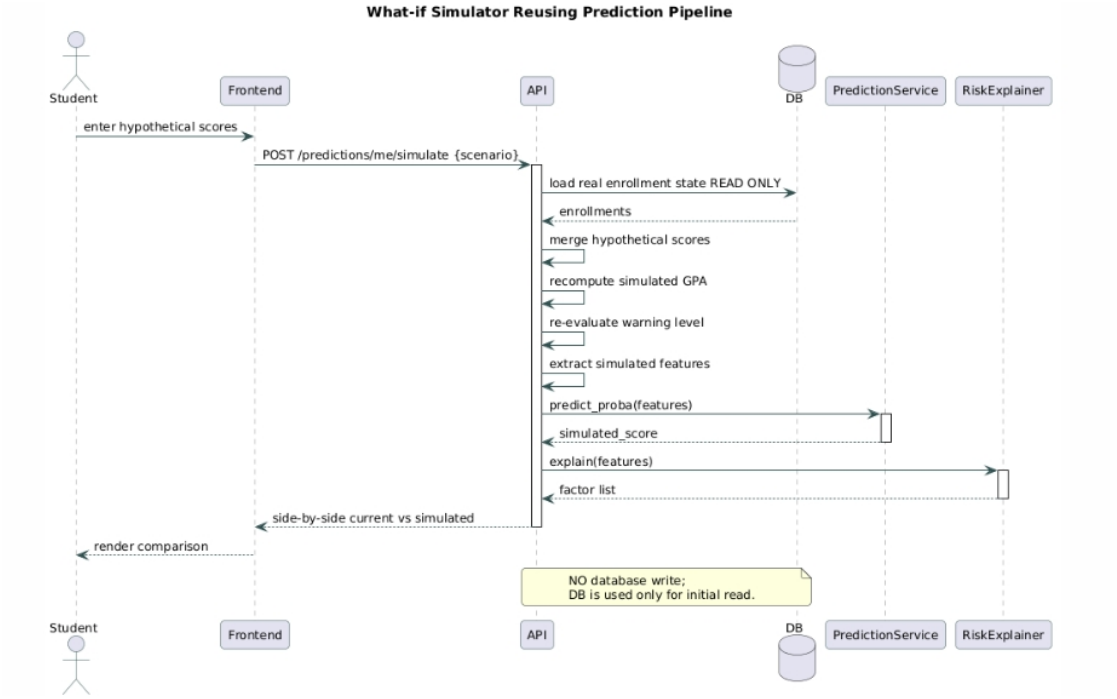

Figure 17: *Sequence diagram — What-if simulator reusing the prediction pipeline without persistence*

Lifelines: Student, Frontend, API, PredictionService, RiskExplainer.

The student submits a hypothetical scenario (a list of courses and assumed scores) via `POST /predictions/me/simulate`. The API loads the student's current enrollment state from the database, merges the hypothetical scores into a transient in-memory view, recomputes GPA and warning level on that view, re-extracts features, and calls the same `PredictionService` and `RiskExplainer` as the production pipeline. The response is a side-by-side comparison of current and simulated metrics. No `Prediction` row is written — the database lifeline appears in the diagram only for the initial read, never for a write.

2.4.5 5. RAG Chatbot Q&A

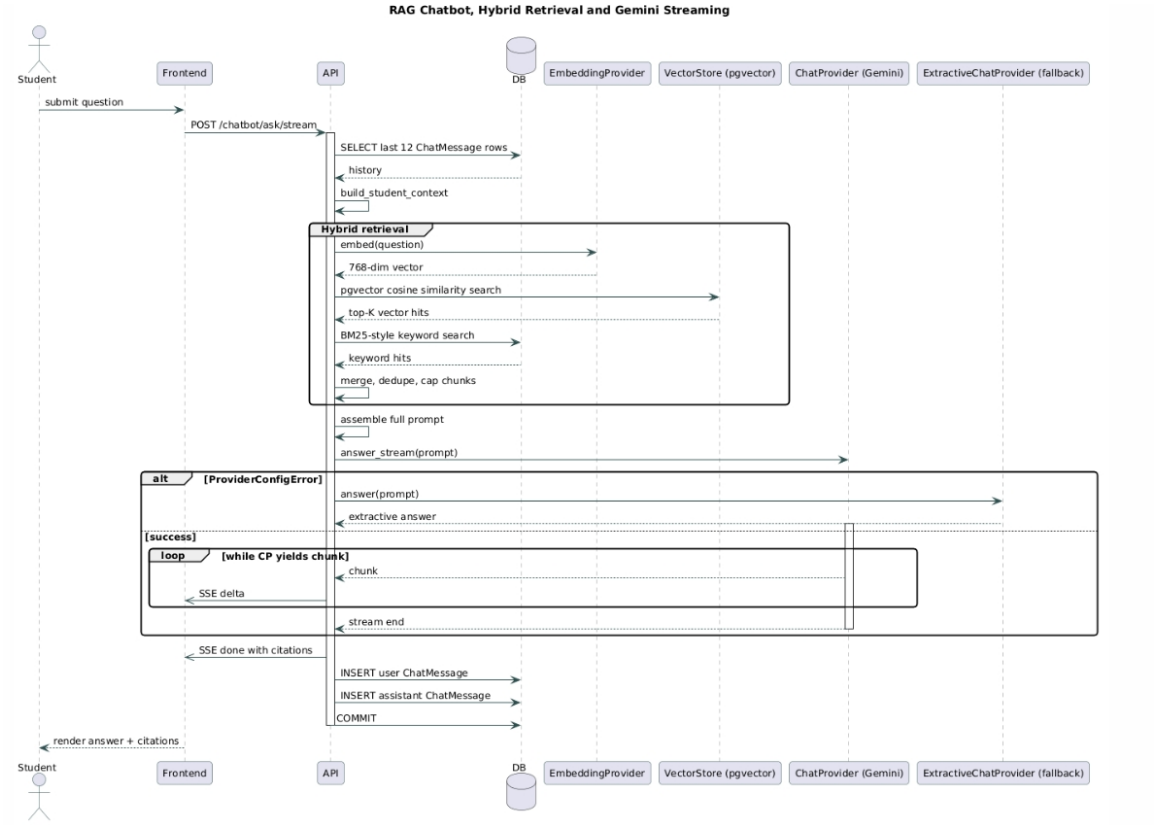

Figure 18: *Sequence diagram — RAG chatbot, hybrid retrieval and Gemini streaming with fallback*

Lifelines: Student, Frontend, API, Database, EmbeddingProvider, VectorStore (pgvector), ChatProvider (Gemini), ExtractiveChatProvider (fallback).

The student submits a question. The frontend POSTs it to `/chatbot/ask` (non-streaming) or `/chatbot/ask/stream` (Server-Sent Events). The API first loads the twelve most recent `ChatMessage` rows for conversation history, then calls `build_student_context` to assemble the personal profile string (name, MSSV, GPA, warning level, failed-course summary). Retrieval then begins: the active `EmbeddingProvider` embeds the question text, the API runs hybrid search against pgvector (cosine similarity over the `embedding` column combined with a BM25-style keyword scorer applied to `content` with stopword filtering), and the merged hit list is deduplicated by document and capped at three chunks per source file. Each retained hit becomes a `ChatCitation` carrying the document filename, page number, chunk index, similarity score and a query-centred snippet. The API formats the citations into a numbered context block and calls the active `ChatProvider.answer` (or `answer_stream`). An `alt` fragment handles `ProviderConfigError`: when the primary provider raises (missing API key, rate limit, network failure), the API instantiates `ExtractiveChatProvider` and retries with the same inputs, prefixing the answer with a notice. For streaming, the API forwards every chunk as a `data: {"type":"delta",...}` SSE event, then emits a closing `"done"` event carrying the citation array and the provider name. Finally the API persists two `ChatMessage` rows (user and assistant, the assistant row including the citations JSON) and commits.

2.4.6 6. View and Resolve Warning

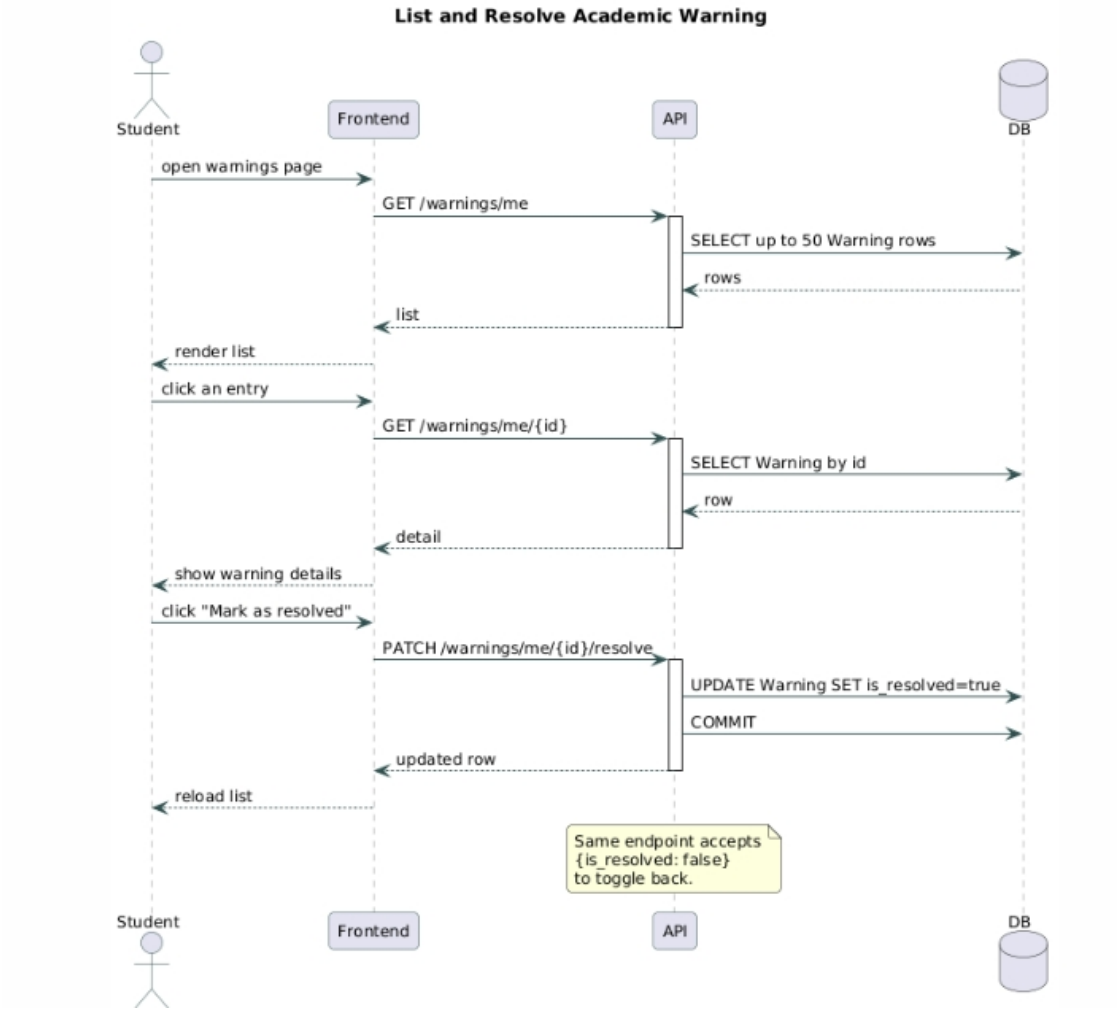

Figure 19: *Sequence diagram — List and resolve academic warning*

Lifelines: Student, Frontend, API, Database.

The frontend issues `GET /warnings/me` on page load; the API returns up to fifty `Warning` rows ordered by `created_at` descending. The student selects an entry; the frontend may issue `GET /warnings/me/{id}` for the detail view, then `PATCH /warnings/me/{id}/resolve` with `{is_resolved: true}` when the student clicks the resolve button. Because the payload accepts an arbitrary boolean, the same endpoint reverts the state to unresolved when invoked with `false`.

2.4.7 7. Admin Batch Prediction (Manual and Scheduled)

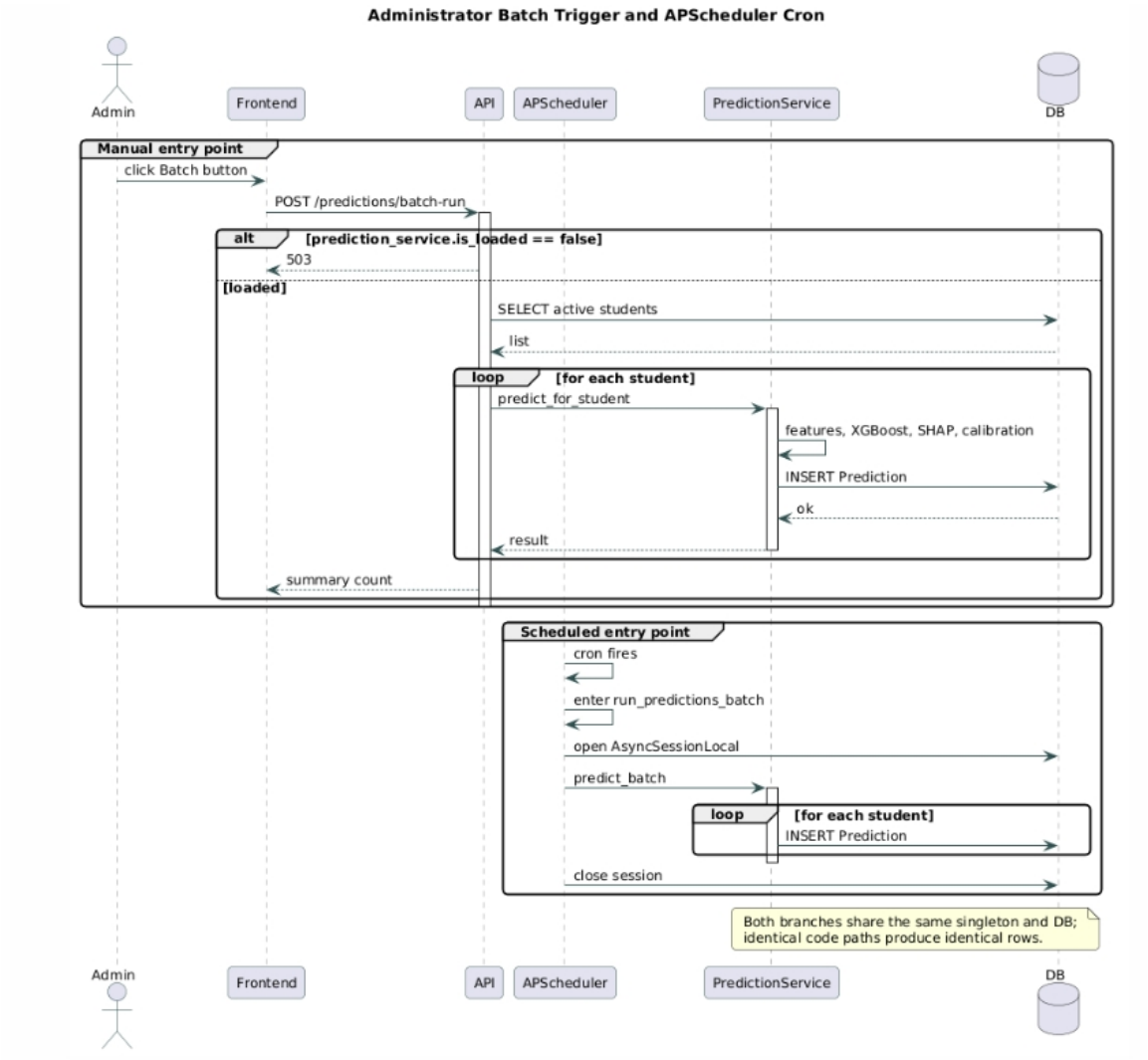

Figure 20: *Sequence diagram — Administrator batch trigger and APScheduler 02:00 cron, sharing one service*

Lifelines: Administrator, Frontend, API, APScheduler, PredictionService, Database.

The diagram shows two entry points calling the same `predict_batch` routine. The administrative branch begins with the administrator pressing the batch button; the frontend POSTs to `/predictions/batch-run`. The API first guards with `prediction_service.is_loaded` (returning `503` when no model is loaded). On success the API enters a loop over every active student, calling `predict_for_student`, which itself fans out into the feature extraction, XGBoost inference, SHAP explanation, calibration and persistence sub-steps described in subsection 2.4.3. A summary count is returned when the loop completes. The scheduled branch shows the APScheduler cron firing at 02:00 (Asia/Ho_Chi_Minh), entering `run_predictions_batch`, which opens its own `AsyncSessionLocal` and invokes the same `predict_batch` method. The two branches share the database and the model singleton, so a manual run uses identical code paths and produces identical `Prediction` rows.

2.4.8 8. Admin Upload RAG Document

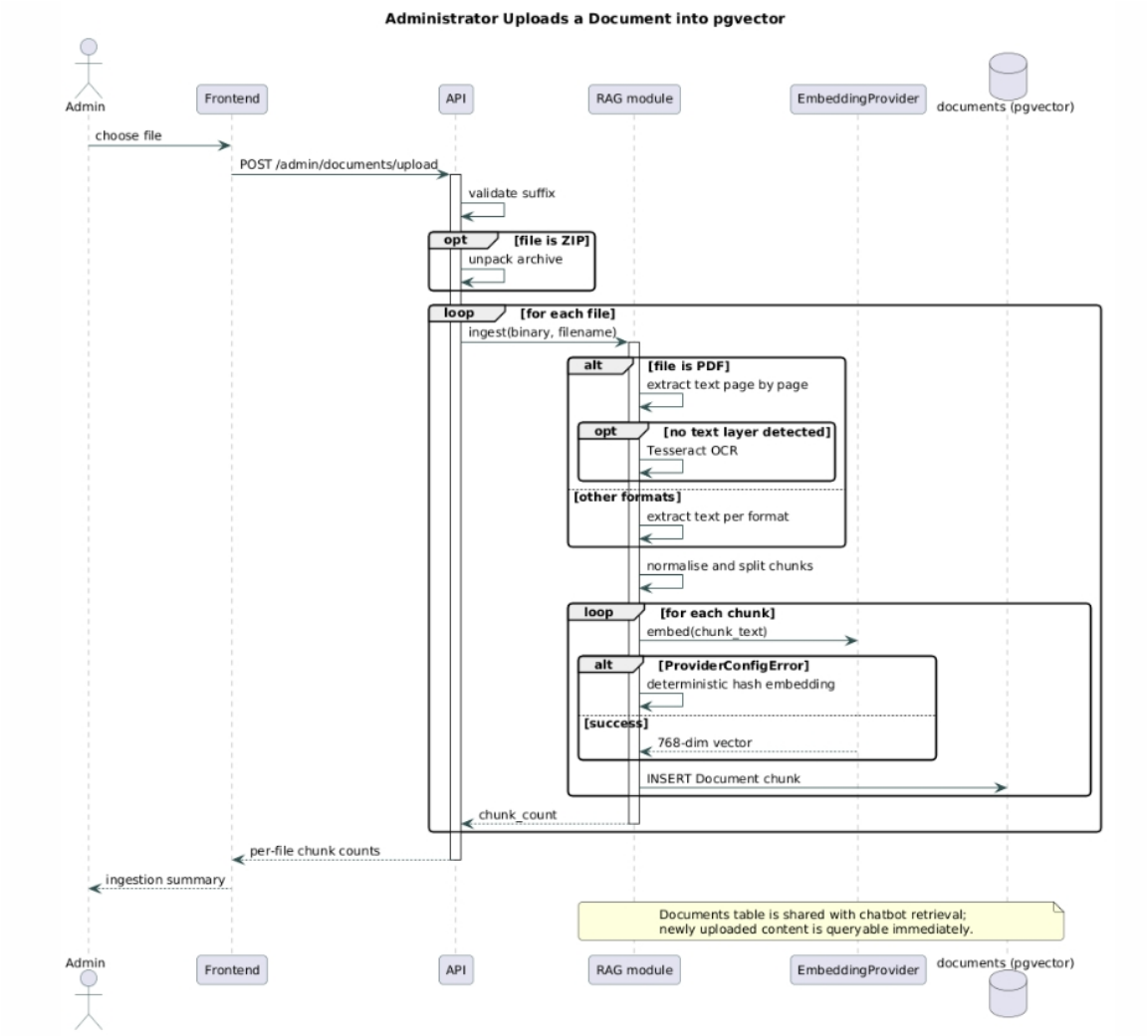

Figure 21: *Sequence diagram — Administrator uploads a document and the system ingests it into pgvector*

Lifelines: Administrator, Frontend, API, RAG module, EmbeddingProvider, Database (`documents` with pgvector).

The administrator uploads a file (PDF, DOCX, TXT, MD, or a ZIP archive). The API validates the suffix against `SUPPORTED_DOCUMENT_SUFFIXES`, unpacks the ZIP if present, and routes the binary content to the RAG ingestion helper. For PDFs the helper extracts text page by page, falling back to Tesseract OCR (at 220 DPI in Vietnamese + English) when no text layer is found. The extracted text is normalised and split into overlapping chunks (default 800 characters, 120-character overlap). A `loop` fragment then iterates over the chunks: each chunk is embedded by the active `EmbeddingProvider` (which falls back to the deterministic hash provider on `ProviderConfigError`, never blocking the upload), and a `Document` row is inserted with the chunk text, the 768-dimensional vector, the chunk index, the page number when available, and the metadata JSON. After the loop the API returns the chunk count to the frontend, and subsequent chatbot queries can retrieve from the newly ingested content immediately because the same `documents` table backs the vector search in subsection 2.4.5.

2.5 Database Design

The persistence layer is built on PostgreSQL 16 with the `pgvector` extension enabled, allowing both relational records and 768-dimensional embedding vectors to coexist in a single database. The schema comprises **ten** entities that together cover authentication, academic records, AI predictions, communication, and the RAG knowledge base. This section presents the conceptual model as an Entity-Relationship Diagram (ERD) and then details the implementation as a relational schema.

2.5.1 Entity-Relationship Diagram

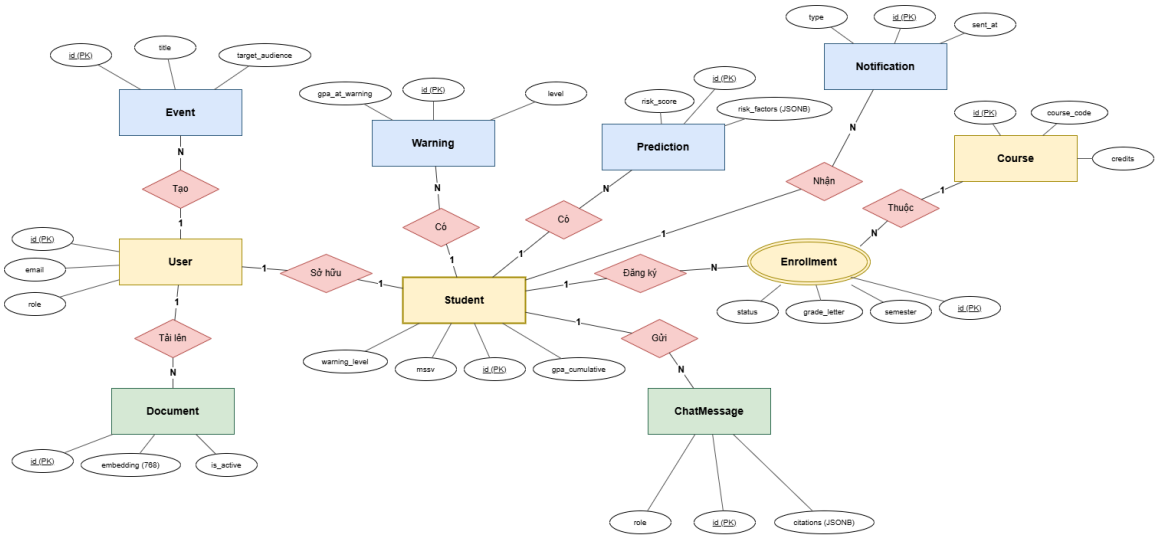

Figure 22: *Entity-Relationship Diagram (Chen notation)*

2.5.1.1 Entities. The model is organised around the `Student` entity as the operational centre, with all academic-history, AI-output, and communication entities attaching to it directly.

- **User** — Authentication identity. Stores email, hashed password, role (`student` or `admin`), and an opt-in flag for email notifications.
- **Student** — Academic profile in a one-to-one relationship with `User`. Carries the cumulative GPA, credits earned, current warning level, and three JSONB columns (`timetable_json`, `exam_schedule_json`, `personal_events_json`) that hold the schedule data introduced in milestone M9.
- **Course** — Course catalogue, keyed by HCMUT course code.
- **Enrollment** — Associative entity between `Student` and `Course` per semester. Holds the four component scores (midterm, lab, other, final) with their corresponding weights, the computed total score and letter grade, attendance rate, status, and provenance flags (`is_finalized`, `source`).
- **Warning** — Academic-warning record. Captures the warning level (1, 2, or 3 for forced withdrawal), the GPA at the moment of warning, an optional AI risk score, and a foreign-key link to the `Notification` that was dispatched.
- **Prediction** — AI risk forecast per student per semester. Stores the risk score, the discrete risk level, and two JSONB payloads: `risk_factors` (SHAP contributions) and `predicted_courses` (per-course pass/fail probabilities).
- **Notification** — In-app and email notification record. Differentiates four types (warning, reminder, event, system) and tracks both `sent_at` (in-app) and `email_sent_at` (email).
- **Event** — Academic events published by administrators. Carries a target audience selector (university-wide, faculty-specific, or cohort-specific) and a mandatory flag that controls reminder dispatch.
- **ChatMessage** — One row per user or assistant turn in the RAG chatbot. Persists the role string, raw content, and the JSONB `citations` array attached to assistant responses.
- **Document** — Chunked RAG knowledge base. Each row is one chunk of a source file with its 768-dimension embedding stored in a `pgvector` column, the chunk index within the file, the page number if available, and an `is_active` flag for soft deactivation.

2.5.1.2 Principal relationships. Each `User` either has at most one associated `Student` record (one-to-one) or, when in the administrator role, is the creator of zero or more `Event` and `Document` rows. A `Student` owns many `Enrollment`, `Warning`, `Prediction`, `Notification`, and `ChatMessage` records, all of which are cascade-deleted when the student record is removed. Each `Enrollment` also references exactly one `Course`, forming the many-to-many tie between `Student` and `Course` as a strong associative entity (with grade data as relationship attributes). A `Warning` optionally references the `Notification` it triggered through `notification_id`, allowing the dispatch record to be linked but not requiring it (the foreign key is set to `NULL` if the notification is later removed).

2.5.2 Relational Schema

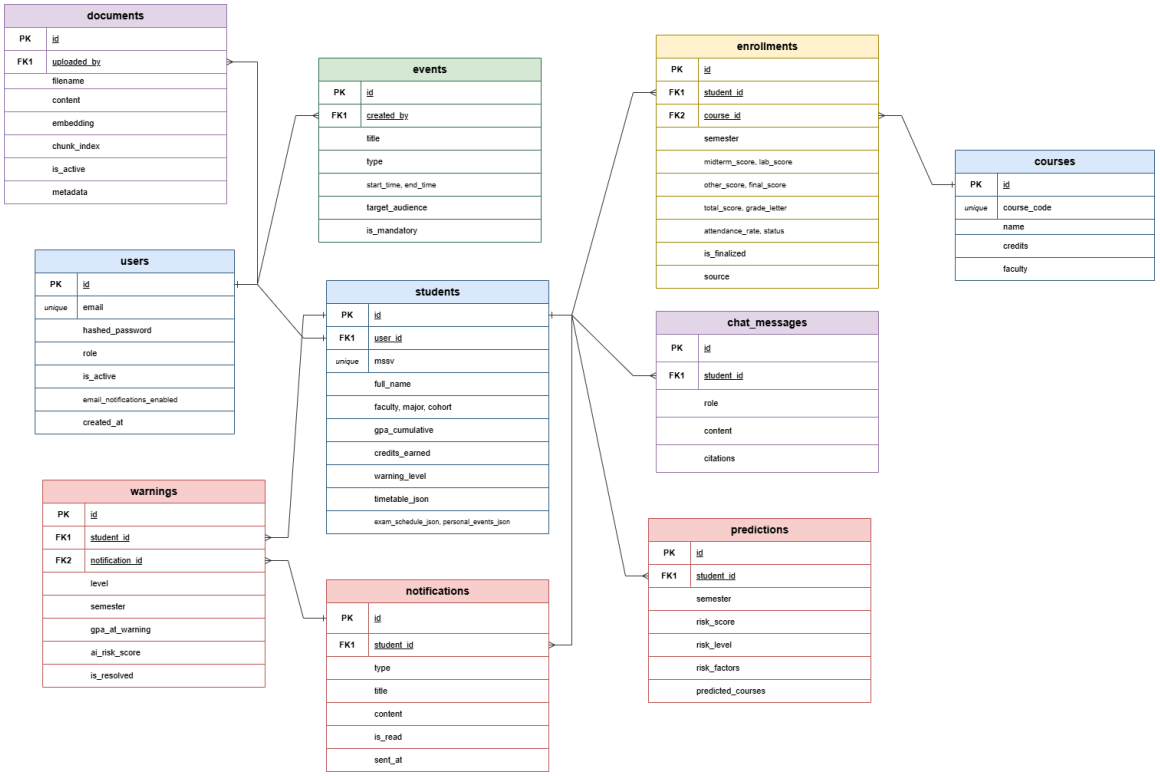

Figure 23: *Relational schema with primary keys, foreign keys, and crow's-foot cardinality*

The relational schema implements the conceptual model with UUID primary keys throughout, explicit indexes on every foreign-key column that participates in frequent joins, and enum types created as native PostgreSQL types. The tables, with their key columns and constraints as defined in the SQLAlchemy ORM, are described below.

2.5.2.1 `users`. Primary key `id` (UUID). Unique-indexed columns: `email` (VARCHAR(255)). Stores `hashed_password`, the `role` enum (`userrole`: `student`, `admin`), and two boolean flags — `is_active` (account status) and `email_notifications_enabled` (opt-in for email notifications, default true).

2.5.2.2 `students`. Primary key `id` (UUID). Foreign key `user_id` (UUID, references `users.id` with `ON DELETE CASCADE`, declared `UNIQUE` to enforce the one-to-one tie). Unique-indexed: `mssv` (VARCHAR(20)). Other columns: `full_name`, `faculty`, `major` (VARCHAR(255) each), `cohort` (INTEGER), `gpa_cumulative` (FLOAT, default 0.0), `credits_earned` (INTEGER, default 0), `warning_level` (INTEGER, default 0), and three nullable JSONB columns (`timetable_json`, `exam_schedule_json`, `personal_events_json`).

2.5.2.3 `courses`. Primary key `id` (UUID). Unique-indexed: `course_code` (VARCHAR(20)). Other columns: `name` (VARCHAR(255)), `credits` (INTEGER, NOT NULL), `faculty` (VARCHAR(255)).

2.5.2.4 `enrollments`. Primary key `id` (UUID). Foreign keys: `student_id` (references `students.id`, `ON DELETE CASCADE`, indexed) and `course_id` (references `courses.id`, `ON DELETE CASCADE`). Table-level uniqueness: `uq_enrollment` on `(student_id, course_id, semester)` prevents duplicate registration of the same course by the same student in the same semester. Nullable score columns: `midterm_score`, `lab_score`, `other_score`, `final_score` (FLOAT). Weight columns with explicit defaults: `midterm_weight`

(0.3), `lab_weight` (0.0), `other_weight` (0.0), `final_weight` (0.7). Derived columns: `total_score` and `grade_letter` (VARCHAR(3)), both nullable. Status enum (`enrollmentstatus`: `enrolled`, `passed`, `failed`, `withdrawn`, `exempt`, default `enrolled`). Metadata: `attendance_rate` (FLOAT, nullable), `is_finalized` (BOOLEAN, default false), `source` (VARCHAR(20), default `"manual"`).

2.5.2.5 `warnings`**.** Primary key `id` (UUID). Foreign keys: `student_id` (`students.id`, CASCADE, indexed) and `notification_id` (`notifications.id`, `ON DELETE SET NULL`, nullable). Columns: `level` (INTEGER), `semester` (VARCHAR(10)), `reason` (TEXT), `gpa_at_warning` (FLOAT), `ai_risk_score` (FLOAT, nullable), `is_resolved` (BOOLEAN, default false), `sent_at` (TIMESTAMPTZ, nullable), `created_by` (`warningcreatedby` enum: `system`, `admin`, default `system`).

2.5.2.6 `predictions`**.** Primary key `id` (UUID). Foreign key: `student_id` (CASCADE, indexed). Columns: `semester`, `risk_score` (FLOAT), `risk_level` (`risklevel` enum: `low`, `medium`, `high`, `critical`). Two JSONB columns: `risk_factors` (per-feature SHAP-based contributions) and `predicted_courses` (per-course pass-probability list).

2.5.2.7 `notifications`**.** Primary key `id` (UUID). Foreign key: `student_id` (CASCADE, indexed). Columns: `type` (`notificationtype` enum: `warning`, `reminder`, `event`, `system`), `title` (VARCHAR(255)), `content` (TEXT), `is_read` (BOOLEAN, default false), `sent_at` (TIMESTAMPTZ, NOT NULL), `email_sent_at` (TIMESTAMPTZ, nullable — only set after a successful SMTP dispatch).

2.5.2.8 `events`**.** Primary key `id` (UUID). Optional foreign key: `created_by` (`users.id`, `ON DELETE SET NULL`). Columns: `title` (VARCHAR(255)), `description` (TEXT, nullable), `event_type` (`eventtype` enum: `exam`, `submission`, `activity`, `evaluation`), `target_audience` (`targetaudience` enum: `all`, `faculty_specific`, `cohort_specific`, default `all`), `target_value` (VARCHAR(255), nullable — holds the faculty name or cohort year when the audience is specific), `start_time` (TIMESTAMPTZ, NOT NULL), `end_time` (TIMESTAMPTZ, nullable), `is_mandatory` (BOOLEAN, default false).

2.5.2.9 `chat_messages`**.** Primary key `id` (UUID). Foreign key: `student_id` (CASCADE, indexed). Columns: `role` (VARCHAR(20) — typically `"user"` or `"assistant"`), `content` (TEXT), `citations` (JSONB, nullable — a list of citation objects produced by the RAG layer).

2.5.2.10 `documents`**.** Primary key `id` (UUID). Optional foreign key: `uploaded_by` (`users.id`, `ON DELETE SET NULL`). Columns: `filename` (VARCHAR(255)), `content` (TEXT, the chunked text body), `embedding` (`Vector(768)` from the `pgvector` extension, nullable), `source_file` (VARCHAR(255), used for grouping chunks back to their originating file), `chunk_index` (INTEGER, the chunk's position within the source file), `page_number` (INTEGER, nullable, populated when the source is a PDF), `metadata_json` (JSONB, mapped from the SQL column literally named `metadata`), `is_active` (BOOLEAN, default true — toggling this excludes the chunk from retrieval without hard-deleting it).

2.5.2.11 Schema evolution. The schema is managed through Alembic migrations and has been extended in five distinct revisions: the initial schema (`e55f5d666040`), the grade-component fields and `exempt` status added for milestone M3 (`331730d3032d`), the RAG chatbot tables and document page numbers added for M5 (`9a5d2e7c4b19`), the email-tracking columns and warning-to-notification linkage added for M6 (`7f4a8b1c2d3e`), and the schedule JSONB columns on `students` added for M9 (`8c1f3a7e9d4b`). Each revision is forward and backward compatible at the column level and can be applied or rolled back independently.

2.6 Class Diagram

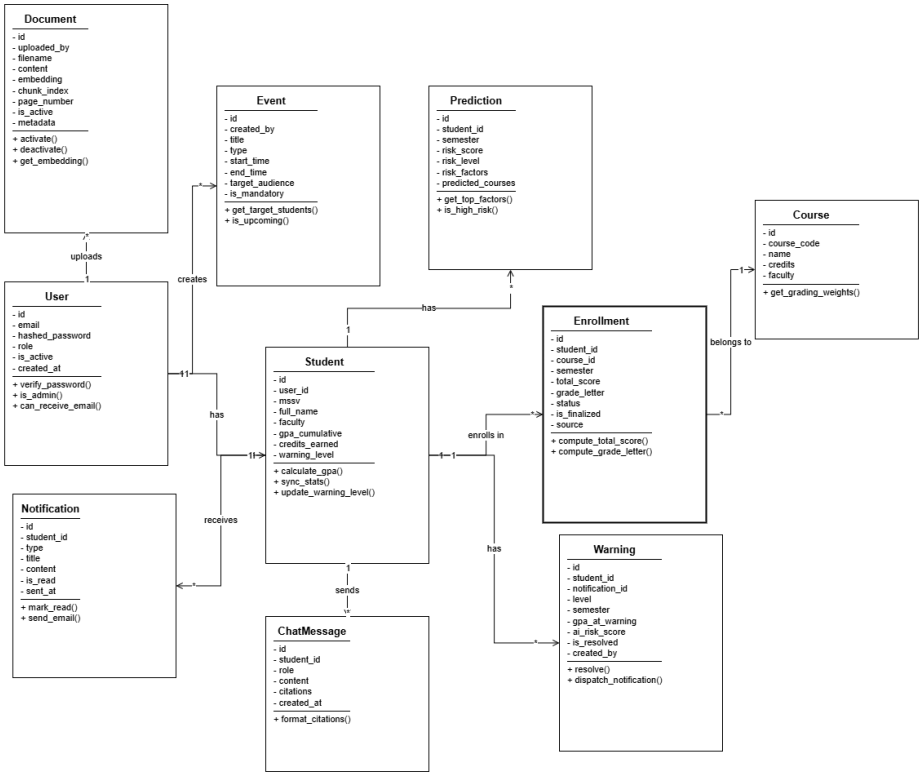

Figure 24: *Class diagram of the domain entities*

The class structure of the system is deliberately mixed: **domain data** and the **AI subsystem** are organised as object-oriented classes (with inheritance and polymorphism where it pays off), while the **business services** are organised as Python modules of pure functions backed by `@dataclass` value objects. This subsection describes each group in turn, mapping the diagram above to the actual code organisation.

2.6.1 Domain Entity Classes

2.6.1.1 User and Student form a one-to-one composition. `User` owns three relationships: `student` (one-to-one, `uselist=False`), `created_events` and `uploaded_documents`. The `Student` side cascades deletes via `all, delete-orphan` to five children: `enrollments`, `warnings`, `predictions`, `notifications`, and `chat_messages`, so removing a student record purges its entire history.

2.6.1.2 Enrollment is the associative class connecting `Student` and `Course`, carrying both the four component scores and their weights as first-class attributes. The single non-trivial table-level invariant — the `(student_id, course_id, semester)` unique constraint — is declared in `__table_args__` as `uq_enrollment`.

2.6.1.3 Warning, Prediction, Notification, ChatMessage are all student-scoped child entities. `Warning` additionally holds a nullable `notification_id` foreign key back to `Notification`, recording which dispatched notification was triggered for each warning approval.

2.6.1.4 Event and Document are administrator-scoped: each carries an optional `created_by` / `uploaded_by` foreign key to `User` with `ON DELETE SET NULL`, so removing an administrator does not orphan the events or documents they previously authored.

2.6.1.5 Enumerations. The model layer also defines seven Python enum classes used as database enum types: `UserRole` (`student`, `admin`); `EnrollmentStatus` (`enrolled`, `passed`, `failed`, `withdrawn`, `exempt`); `WarningCreatedBy` (`system`, `admin`); `RiskLevel` (`low`, `medium`, `high`, `critical`); `NotificationType` (`warning`, `reminder`, `event`, `system`); `EventType` (`exam`, `submission`, `activity`, `evaluation`); and `TargetAudience` (`all`, `faculty_specific`, `cohort_specific`).

2.6.2 AI Service Classes

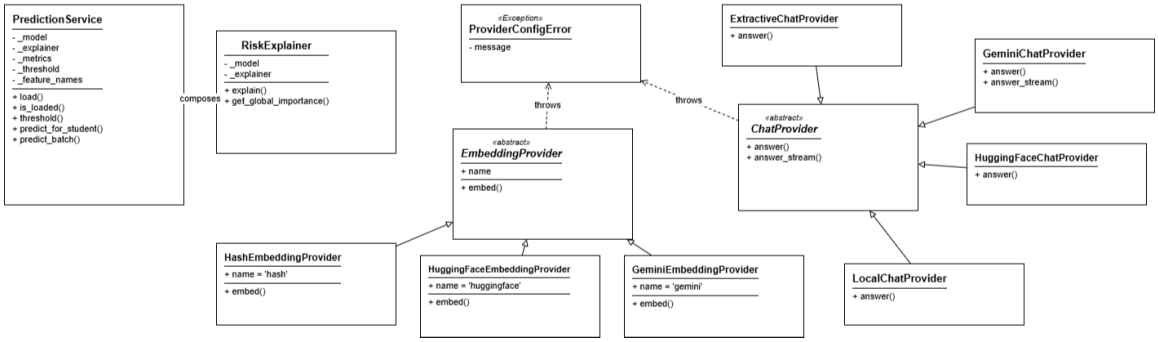

Figure 25: *Class diagram of the AI service classes*

The AI subsystem contains the only first-class service objects in the codebase. They use inheritance and the abstract-base-class pattern to support runtime selection of the active provider.

2.6.2.1 `PredictionService` (in `app.ai.prediction.model`) is a singleton holding the loaded XGBoost classifier, a `RiskExplainer` instance, and the metrics dictionary persisted alongside the trained model. Its public surface is `load()` (return `True` if a compatible model was loaded from disk; reject the model if its feature names disagree with the current code's `FEATURE_NAMES`), the properties `is_loaded` and `threshold` (the decision threshold persisted in the training metrics, defaulting to 0.5), and `predict_for_student(student, db, save=True)` which returns a fresh `Prediction` record or `None` when the student has too little data to score (cold-start). The service is instantiated once during application startup and reused for every inference call.

2.6.2.2 `RiskExplainer` (in `app.ai.prediction.explainer`) wraps a SHAP `TreeExplainer` around the same XGBoost model. Its `explain(features, top_k=5)` method computes per-feature SHAP values, filters out low-impact or contradictory contributions, normalises the surviving values so their absolute impacts sum to 100%, and returns each contribution along with a human-readable Vietnamese label and a direction indicator (+ increases risk, − decreases risk). `get_global_importance()` exposes the model's static gain-based importances when no per-student features are needed.

2.6.2.3 Embedding provider hierarchy. The abstract class `EmbeddingProvider` declares a class attribute `name` and a single asynchronous method `embed(text, task_type)` → `list[float]`. Three concrete implementations exist: `HashEmbeddingProvider` (deterministic Blake2b token hashing into 768 buckets, used as the dependency-free default), `GeminiEmbeddingProvider` (calls Google's `embed_content` with the configured `GEMINI_EMBEDDING_MODEL`), and `HuggingFaceEmbeddingProvider` (calls the Hugging Face Inference API). The module-level factory `get_embedding_provider()` selects the concrete class based on the `EMBEDDING_PROVIDER` setting.

2.6.2.4 Chat provider hierarchy. The abstract class `ChatProvider` declares `answer(question, retrieved_context, student_context, history)` → `str` as the single required method, plus a default `answer_stream(...)` that chunks the full answer into 24-word groups for clients that need streaming. Four concrete subclasses exist: `ExtractiveChatProvider` (composes responses directly from retrieved chunks and the parsed student context, without an external LLM, so the chatbot never returns nothing), `GeminiChatProvider` (overrides `answer_stream` to use Gemini's native streaming SDK and includes a fallback list of compatible Gemini models), `HuggingFaceChatProvider` (calls a hosted text-generation model on the Hugging Face Inference API), and `LocalChatProvider` (POSTs an OpenAI-compatible `/chat/completions` request to a configurable local endpoint such as Ollama). `get_chat_provider()` resolves the active subclass from `CHAT_PROVIDER`. A shared `ProviderConfigError` exception class signals misconfiguration uniformly across both hierarchies.

2.6.3 Business Services as Function Modules

The remaining business logic lives in `app.services.*` as collections of pure functions and `@dataclass` value objects rather than as service classes. This choice keeps the call sites trivially testable in isolation and avoids the "services holding only static methods" anti-pattern. Each module owns a coherent set of operations and the data carriers those operations exchange:

- `warning_engine` — the HCMUT regulation engine. Pure decision functions (`check_regulation_warning`, `check_ai_early_warning`) return a `WarningDecision` dataclass; database orchestrators (`sync_current_warning_level`, `evaluate_and_persist`, `batch_check_warnings`) wrap these with `Warning` and `Notification` persistence and produce a `WarningOutcome` dataclass for the API layer.
- `gpa_calculator` — HCMUT four-point scaling rules. Functions such as `score_to_grade_letter`, `grade_letter_to_gpa_point`, `calculate_semester_gpa`, and `calculate_gpa_trend` return an `EnrollmentGrade` value object or scalar values; they have no state and no I/O.
- `grade_aggregator` — the "highest wins" retake rule and student-stat synchronisation (`sync_student_stats`, `count_unresolved_failed`, `effective_enrollments_per_course`).
- `recommender` and `study_plan` — the credit-load and retake-priority recommender. Returns a `CreditLoad` dataclass and a list of `RetakeSuggestion` dataclasses; `study_plan.build_study_plan(db, student)` composes both.
- `notification` — a single `create(...)` entry point that persists a `Notification` row and, depending on the type and the user's opt-in flag, dispatches an email through `email_service.fire_and_forget`. Helpers (`list_for_student`, `unread_count`, `mark_read`, `update_email_preference`) cover the remaining CRUD operations.
- `email_service` — non-blocking SMTP dispatcher with a fire-and-forget coroutine wrapper so notification creation never blocks the request response.
- `mybk_parser`, `tkb_parser`, `exam_schedule_parser` — transcript, timetable, and exam-schedule parsers. Each `parse(text)` function returns a `ParsedTranscript` / `ParsedTimetable` / `ParsedExamSchedule` dataclass made up of `ParsedCourse`, `TimetableSession`, or `ExamEntry` sub-records.
- `import_service` — bulk-import functions for students and grades from Excel, producing an import-result summary and the downloadable templates.
- `event_manager` — CRUD helpers over the `Event` table, plus the audience-filtering logic used by both the admin and student event endpoints.

2.6.4 Supporting Value Objects

A small number of frozen `@dataclass` declarations carry intermediate data between layers without owning behaviour. They are first-class classes in the diagram even though they have no methods beyond the auto-generated ones. The notable members are `FeatureResult` (the 11-feature vector returned by `extract_features`), `SearchHit` (a single hybrid-retrieval result with its similarity scores and the originating document chunk), `ParsedPage` and `DocumentChunk` (PDF parsing intermediaries inside the RAG indexing pipeline), and the parser result dataclasses listed in the previous subsection. Together they make the boundaries between modules type-safe without requiring a heavier object-oriented design.

3 Architecture Design

3.1 Technology Stack

The system is built as a containerised three-tier web application with a Python asynchronous backend, a TypeScript single-page frontend, and a PostgreSQL database augmented with the `pgvector` extension. The chosen libraries are listed below per layer, with a short justification of the role each one plays. Some Python dependencies declared in `pyproject.toml` — notably `celery`, `redis`, `openai` and `llama-index` — are kept for forward compatibility but are not exercised by the current runtime; the active stack is the one described in this section.

3.1.1 Frontend Layer

3.1.1.1 Next.js 14 with App Router. The user interface is a Next.js 14.2 (`next dev -turbo` in development) application using the App Router convention. Pages are organised by role under `frontend/app/auth/`, `frontend/app/student/` and `frontend/app/admin/`, each segment owning its own layout and middleware-driven access control. React 18 is the rendering library and TypeScript 5 enforces type-safety across components, hooks and API contracts.

3.1.1.2 shadcn/ui on Tailwind CSS. The component library is `shadcn/ui` (v4.5, copy-into-project pattern rather than installable widgets) on top of Tailwind CSS 3.4 for styling. Primitive accessibility behaviour is provided by `@radix-ui/react-dialog` and `@base-ui/react`; iconography uses `lucide-react`. The combination yields a small, fully owned component layer with no opaque vendor styles.

3.1.1.3 Data fetching and forms. Network I/O is wrapped in an Axios instance that automatically attaches the JWT from the auth store and triggers a logout on `401` responses. Server-state caching uses `@tanstack/react-query 5`. Forms are built with `react-hook-form 7` validated by `zod 4` schemas declared in the same file as each form for tight coupling between UI and validation.

3.1.1.4 Visualisation and feedback. Charts in the dashboard, the prediction page (radial risk gauge, factor bars) and the history view are produced with `recharts 3.8`. The chatbot transcript renders Markdown with `react-markdown 10` + `remark-gfm 4` so tables and lists from the model appear correctly. Toast notifications use `sonner 2`, and the onboarding walkthrough for first-time students is built with `react-joyride 3`.

3.1.1.5 State management. Client-side authentication and ephemeral UI state are kept in `zustand 5` stores. The auth store persists to `localStorage` and writes a `SameSite=Lax` cookie so middleware-protected routes can read the role on the server side during SSR.

3.1.2 Backend Layer

3.1.2.1 FastAPI 0.111 and Uvicorn. The HTTP server is FastAPI on Uvicorn (with the `standard` extras for HTTP/2 and watchgod-based hot reload). Pydantic 2.7 powers request/response models and the application settings, with `pydantic-settings` loading configuration from environment variables and the `.env` file. The application root `app/main.py` wires the lifespan handler (`bootstrap_admin`, model loading, scheduler start-up), the CORS middleware, and the `/api/v1` router prefix.

3.1.2.2 SQLAlchemy 2.0 with async + asyncpg. The ORM layer is SQLAlchemy 2.0.29 using the async `AsyncSession` pattern over the `asyncpg` driver. Each model in `app/models/` maps to a single PostgreSQL table; relationships are declared bidirectionally so that orchestrator services in `app/services/` can traverse the object graph without explicit joins. Schema migrations are managed by Alembic 1.13, with five revisions on disk covering the initial schema and the four follow-up milestones (M3 enrollment fields, M5 RAG, M6 email tracking, M9 schedule JSONB columns).

3.1.2.3 PostgreSQL 16 with pgvector. The database runs from the `pgvector/pgvector:pg16` Docker image, which bundles PostgreSQL 16 with the `pgvector 0.2` extension. The same database holds both relational records and 768-dimensional embedding vectors on the `documents` table, removing the need for a separate vector store.

3.1.2.4 APScheduler in-process. Recurring jobs (daily batch prediction, deadline reminders, exam reminders) are scheduled with APScheduler running inside the same process as the API server, registered through `setup_scheduler()` during application start-up. This replaces the conventional Celery + Redis pair — `celery` and `redis` appear in `pyproject.toml` but the corresponding services are commented out in `docker-compose.yml` and no module imports them. The trade-off is acceptable because the jobs are coarse-grained (three jobs, each running at most once per day) and benefit from sharing the same connection pool as the API.

3.1.2.5 Authentication primitives. Passwords are hashed with bcrypt through `passlib`, and JSON Web Tokens are signed with HS256 through `python-jose`. Upload endpoints use `python-multipart` for multipart parsing.

3.1.2.6 Document and Excel handling. The Excel import endpoints rely on `openpyxl` for both reading uploaded files and generating downloadable templates. PDF parsing uses PyMuPDF (imported as `fitz`); the `pypdf` dependency declared in `pyproject.toml` is not invoked at runtime. OCR fallback for scanned PDFs uses `pytesseract` with the Pillow image library.

3.1.3 AI / ML Layer

3.1.3.1 XGBoost classifier. The risk-prediction model is an `xgboost.XGBClassifier` (XGBoost 2.0). Training uses 5-fold stratified cross-validation under `scikit-learn 1.4`, with the `StratifiedKFold` and `train_test_split` primitives. The trained model is serialised to disk through `joblib`.

3.1.3.2 Optuna hyperparameter tuning. Hyperparameter search is driven by Optuna with a maximise-F1 objective over a 25-trial study covering depth, learning rate, regularisation, subsampling and `scale_pos_weight` bounds. Monotonic constraints are passed through to XGBoost so the search space respects the protective/risk semantics of each feature.

3.1.3.3 SHAP explainability. Per-prediction explanations come from the SHAP library's `TreeExplainer`. The wrapper in `app/ai/prediction/explainer.py` computes SHAP values, filters out low-impact contributions, normalises the surviving ones to sum to 100%, and attaches Vietnamese labels for student-facing presentation.

3.1.3.4 Google Gemini for chat generation. The RAG chatbot's default chat provider calls Google Gemini through the official `google-generativeai` SDK with native streaming via `generate_content_async(..., stream=True)`. Three Gemini model identifiers are tried in a fallback list (`gemini-2.5-flash`, `gemini-2.0-flash`, `gemini-flash-latest`) to absorb regional model availability differences. The same SDK provides the embedding model `embedding-001` when an API key is configured. The `openai` and `llama-index` dependencies declared in `pyproject.toml` are not imported by the current chatbot pipeline.

3.1.3.5 Provider fallback chain. When the configured chat or embedding provider raises `ProviderConfigError` (missing key, rate limit, network failure), the system degrades gracefully through Hugging Face Inference, then to a local OpenAI-compatible endpoint (such as Ollama), and finally to the deterministic `ExtractiveChatProvider` that composes answers directly from retrieved chunks. The embedding pipeline has an analogous fallback to the in-process `HashEmbeddingProvider`, so document ingestion never blocks waiting for an external API.

3.1.4 DevOps and Tooling

3.1.4.1 Docker Compose orchestration. Local and demo deployment uses Docker Compose with three active services: `db` (the pgvector/postgres image with a health-check), `backend` (the FastAPI container built from `backend/Dockerfile`, depending on the database health-check), and `pgadmin` (web-based DB administration on port 5050). The frontend is also defined as a service in the compose file; in practice it is more often run with `npm run dev` during development for faster iteration.

3.1.4.2 Testing. The backend test suite is `pytest` with `pytest-asyncio` for async fixtures and `pytest-cov` for coverage. Pure-function modules (warning engine, GPA calculator, myBK parser) are covered by unit tests; the API surface has integration tests against an in-memory test database.

3.1.4.3 Code quality tooling. Ruff handles linting and import sorting; mypy is configured with non-strict mode and `ignore_missing_imports` for third-party SDKs. A `pre-commit` hook chain runs both before each commit.

3.2 System Architecture

Choosing an architecture pattern is a load-bearing decision for a project at this scope: a single developer, a moderate-complexity domain augmented with an AI subsystem, and a demo deployment rather than a production rollout. This section analyses four common architectural styles, justifies the chosen combination, and then describes how the three tiers fit together.

3.2.1 Architecture Pattern Analysis

3.2.1.1 Monolithic MVC. The classical Model-View-Controller monolith bundles every layer of the application into one deployable unit. **Pros:** simplest to develop and reason about; one process, one log stream, one connection pool. **Cons:** as the codebase grows the layer boundaries blur, the entire application has to be redeployed for any change, and the rigid MVC contract maps poorly onto frameworks built around dependency injection rather than around controllers. **Verdict:** too rigid for a project with a clearly separable AI subsystem and a JavaScript single-page frontend.

3.2.1.2 Microservices. Each business capability is a separately deployable service with its own database, communicating over the network. **Pros:** independent scaling, language choice per service, failure isolation. **Cons:** significant operational overhead — multiple deployments, distributed tracing, schema-versioned cross-service APIs, eventual-consistency joins; the development complexity is justifiable only when the team and traffic profile demand it. **Verdict:** overengineering at this scale; the entire warning-and-AI workload comfortably fits in one process and one developer cannot reasonably operate a service mesh.

3.2.1.3 Serverless / Functions-as-a-Service. Backend logic is decomposed into stateless functions executed on demand by a cloud provider. **Pros:** no server management; pay-per-invocation cost model; built-in autoscaling. **Cons:** cold-start latency is severe for ML inference (loading the XGBoost classifier plus the SHAP explainer takes seconds at process start); the in-process scheduler used here does not map cleanly to ephemeral functions; vendor lock-in shapes the entire design. **Verdict:** unsuitable for an AI-heavy workload that needs a warm model singleton and a long-running scheduler.

3.2.1.4 Modular Monolith behind a Client-Server frontend. The backend is a single deployable service internally partitioned into well-named modules (API, services, data access, AI, scheduler) with one-way dependencies; the frontend is a separately deployed single-page application talking to the backend over HTTPS. **Pros:** simple to deploy (one Docker Compose file); easy local development; the AI subsystem shares the same database connection pool and Python process as the API; clear module boundaries leave the door open for future microservice extraction if the load profile changes. **Cons:** the entire backend has to be redeployed together; horizontal scaling is coarse-grained. **Verdict:** the best fit for the project's constraints.

3.2.2 Chosen Architecture and Rationale

The system implements a **modular-monolith backend** behind a **client-server frontend**: a Next.js single-page application talks to a FastAPI service over JWT-bearer-authenticated HTTPS, which in turn reads and writes a single PostgreSQL database. The choice is driven by two main project-specific factors.

3.2.2.1 AI-data colocation. Both AI subsystems — the XGBoost risk predictor and the RAG chatbot — need direct access to the same database the API serves. The predictor reads `Student` and `Enrollment` rows during feature extraction; the chatbot reads both the `documents` vector store and the same student record for personalised context. Putting them in a separate service would force every prediction or chat call to make an extra network hop, or it would require a duplicate read replica with non-trivial replication logic.

3.2.2.2 Demo-grade deployment. The acceptance criterion is a working demo on a single host, not a production rollout serving thousands of concurrent users. A modular monolith comfortably handles this load and leaves microservice extraction as a future option once production traffic patterns are available.

3.2.3 Overall Architecture

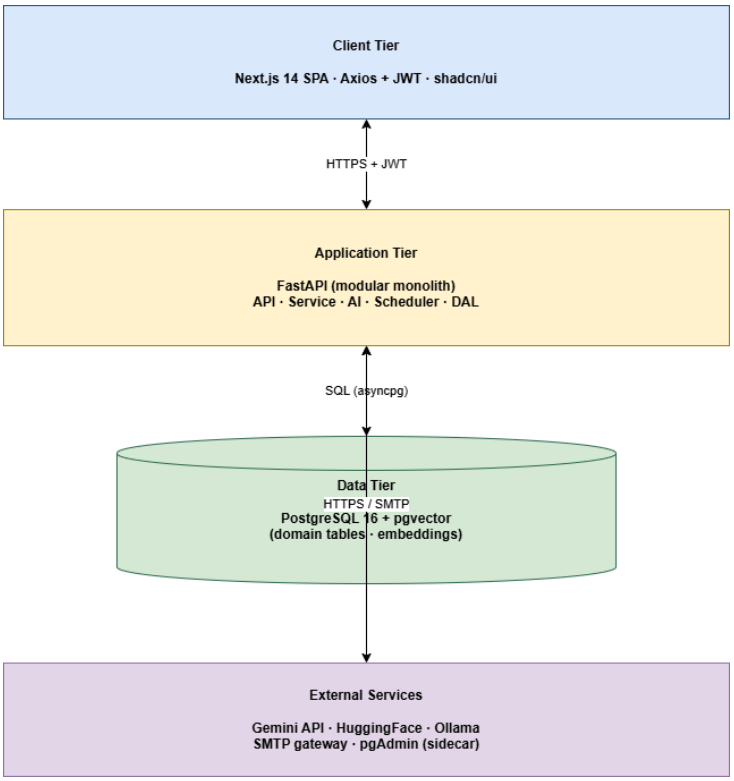

Figure 26: *Overall system architecture — Browser client, API server, database and external providers*

The diagram shows four logical tiers. The **Client tier** is the Next.js 14 single-page application served either by Next's development server or by the production `next start` container. The frontend communicates exclusively with the backend over HTTPS using JWT-bearer authentication; the auth store persists the token to `localStorage` and the Axios interceptor automatically attaches it to every outbound request.

The **Application tier** is the FastAPI process. It hosts the eleven API router groups under the `/api/v1` prefix, the in-process APScheduler with three cron jobs, and the singleton `PredictionService` that loads the XGBoost model and SHAP explainer at start-up. The same process handles the chat-completion pipeline, calling out to external providers (Google Gemini, Hugging Face Inference, optionally a local Ollama endpoint) and SMTP for outbound email.

The **Data tier** is a single PostgreSQL 16 container with the `pgvector` extension enabled. The relational tables (`users`, `students`, `enrollments`, `warnings`, ...) coexist in the same database as the `documents` table whose `embedding` column stores 768-dimensional vectors. The schema is owned by Alembic migrations; the application never executes DDL outside that channel.

External services on the right side of the diagram are dependencies the system can degrade past: Google Gemini for chat and embeddings (fall back to Hugging Face, then local LLM, then extractive provider), and an SMTP server for email notifications (failures are logged but never block API responses). The administrator-facing pgAdmin web interface runs as a separate container for DBA convenience but is not on the request path.

3.2.4 Frontend Architecture

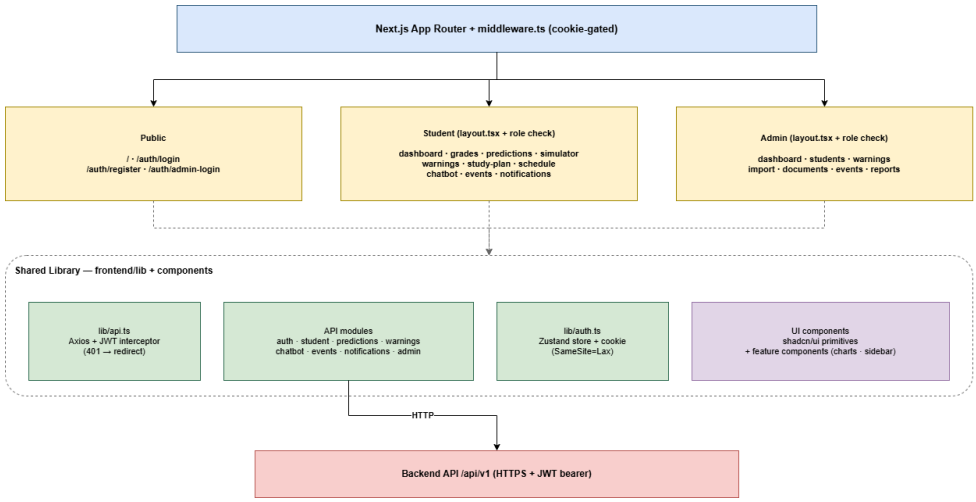

Figure 27: *Frontend architecture — Role-segmented routes and shared library layer*

The application uses the Next.js App Router so each route corresponds to a folder under `frontend/app/`. Routes are grouped by access level. **Public routes** (the landing page `/`, `/auth/login`, `/auth/register`, `/auth/admin-login`) do not require a token. **Student routes** live under `/student/` and cover nine functional pages — `dashboard`, `grades`, `predictions` (with a nested `/simulator` sub-page), `warnings`, `study-plan`, `schedule`, `chatbot`, `events` and `notifications`. **Administrator routes** live under `/admin/` and cover seven pages — `dashboard`, `students`, `warnings`, `import`, `documents`, `events` and `reports`. Each group has its own `layout.tsx` that wraps its pages with the relevant sidebar, top bar and access-control check (the admin layout redirects non-admin sessions to the student dashboard).

The shared library layer at `frontend/lib/` is consumed by every page. The Axios client (`lib/api.ts`) instantiates one `axios` object configured with the `NEXT_PUBLIC_API_URL` base, automatically attaches the JWT from the Zustand auth store, and reacts to `401` responses by clearing the store and redirecting to the login page. API modules (`authApi`, `studentApi`, `predictionsApi`, `warningsApi`, `chatbotApi`, `eventsApi`, `notificationsApi`, `studyPlanApi`, `adminApi`) wrap the raw HTTP calls in typed functions so pages stay free of URL string assembly. The auth store (`lib/auth.ts`) persists to `localStorage` and also writes the JWT to a `SameSite=Lax` cookie named `access_token`. The App Router middleware in `frontend/middleware.ts` reads only this cookie and gates every non-public route on its presence: requests without a token are redirected to `/auth/login`, regardless of role. Role-based separation (an administrator should not land on the student dashboard, and vice versa) is enforced inside the per-area `layout.tsx` files on the client side, where the layout inspects the persisted user object and redirects mismatched roles. This split between cookie-gated middleware and layout-side role check keeps the SSR fast (no database lookup on every request) while still preventing cross-role page renders before any data is fetched.

Two UI primitive layers sit beneath the pages. **Generic primitives** from `components/ui/` are the shadcn/ui components copied into the project (button, card, dialog, input, badge, skeleton, separator, ...); each is a thin wrapper around Radix or Base UI primitives styled with Tailwind. **Feature components** from `components/` include the student and admin sidebars, the notification bell with the unread-count badge, the language toggle, and the onboarding tour. Charts in the dashboard, the prediction page and the history view are built from `recharts` primitives composed inside feature components.

3.2.5 Backend Architecture

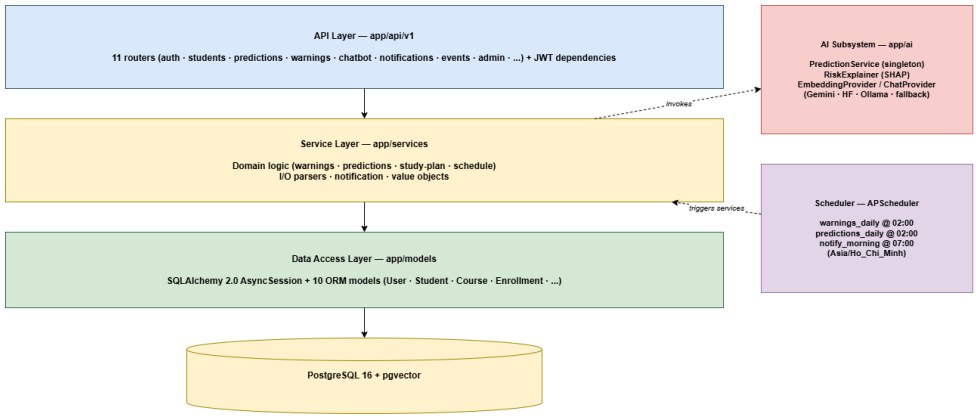

Figure 28: *Backend architecture — Layered FastAPI server with in-process scheduler and AI subsystem*

The backend is organised as four horizontal layers plus a vertical AI subsystem, all running inside the same Python process.

The **API layer** is composed of eleven FastAPI router modules under `app/api/v1/` (`auth`, `students`, `predictions`, `chatbot`, `warnings`, `study_plan`, `events`, `notifications`, `documents`, `courses`, `admin`) plus an `api_router` aggregator. The CORS middleware sits in front and allows only origins declared in `settings.cors_origins`. Authentication is enforced through the `get_current_user` dependency, and administrator-only routes wrap an additional `require_admin` dependency.

The **Service layer** (`app/services/`) contains the business logic. Following Python conventions the services are organised as modules of pure functions plus `@dataclass` value objects, not as injectable classes (see Section 2.6.3 in Chapter 2). The principal modules are `warning_engine` (the HCMUT regulation engine), `gpa_calculator` and `grade_aggregator` (four-point scaling and the highest-wins retake rule), `recommender` and `study_plan` (credit-load and retake suggestions), `notification` (in-app + email dispatch), `email_service` (the fire-and-forget SMTP wrapper), `mybk_parser`, `tkb_parser` and `exam_schedule_parser` (text-to-domain extraction), `import_service` (Excel bulk loaders and templates), and `event_manager` (academic event CRUD with audience filtering).

The **Data access layer** is SQLAlchemy 2.0 with the `AsyncSession` pattern over the `asyncpg` driver. The ten declarative models in `app/models/` expose typed `Mapped[...]` columns and bidirectional relationships; routes obtain a session through the `get_db` dependency that wraps every request in a SAVEPOINT-rolled-back-on-error transaction. Schema changes go through Alembic migrations in `backend/migrations/versions/`.

The **Data tier** is the PostgreSQL 16 database with `pgvector`. The same physical database contains both the transactional tables and the 768-dimensional vector embeddings, so retrieval-augmented chatbot queries do not require a network hop to a separate vector store.

The **AI subsystem** sits orthogonally to the four-layer stack: `PredictionService` (singleton loaded once at lifespan start-up), the `RiskExplainer` it wraps, and the embedding/chat provider hierarchies in `app/ai/chatbot/providers.py`. Services depend on the AI subsystem (the predictions router calls `predict_for_student`; the chatbot router calls `ask_chatbot`); the dependency direction is one-way, so the AI subsystem never holds a session reference of its own — callers pass one in.

The **Scheduler** runs as a fifth top-level component instantiated in the FastAPI lifespan hook. APScheduler with the `Asia/Ho_Chi_Minh` time zone registers three cron jobs: `predictions_batch_daily` at 02:00, `deadline_reminders_daily` at 07:00, and `exam_reminders_daily` also at 07:00. Each job opens its own `AsyncSessionLocal` so it does not interfere with HTTP request sessions. Because the scheduler shares the application process, manual API triggers (such as the administrator's batch-prediction button) reuse the same service functions as the scheduled jobs, ensuring identical behaviour whether a run is automated or operator-driven.

3.3 Machine Learning Pipeline

The risk-prediction subsystem is built around an XGBoost gradient-boosted classifier trained offline on a synthetic dataset and served as a singleton in the FastAPI process. The training pipeline and the inference pipeline share the same feature engineering code but otherwise operate independently — one runs occasionally on the developer's machine via a CLI entry point, the other runs on every `GET /predictions/me` request.

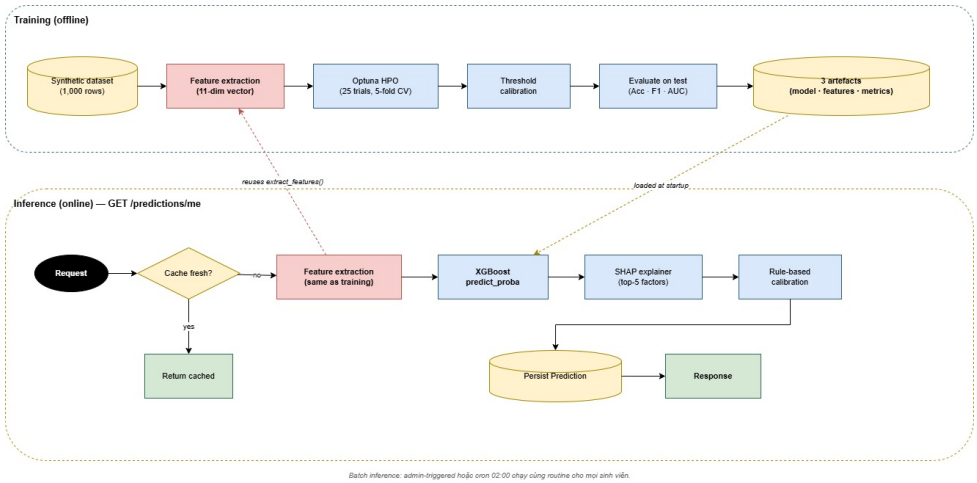

Figure 29: *End-to-end ML pipeline — offline training (top half) and online inference with rule-based calibration (bottom half) sharing the same 11-feature extractor*

3.3.0.1 Feature engineering. The eleven features extracted by `app/ai/prediction/features.py` are framed as *risk signals* rather than raw measurements so that monotonic constraints make sense. Ten signals point in the *higher-is-riskier* direction: the cumulative-GPA deficit, the most-recent-semester deficit, the recent downward-GPA trend, the run length of consecutive sub-2.0 semesters, unresolved-failed-course count, unresolved-failed-last-semester count, retaken-still-failed count, withdrawn count, pass-rate deficit, and attendance risk. One feature is protective: `recovered_failed_courses`. A versioned constant `FEATURE_VERSION` lets the production loader reject any model whose embedded feature list disagrees with the current code, preventing an obsolete artefact from silently producing nonsense predictions.

3.3.0.2 Training (offline). The entry point `python -m app.ai.prediction.train` loads the 1,000-row synthetic dataset (students with MSSV beginning with `"SYN"`), eager-loads enrollments with their courses, and calls `extract_features_batch` to produce the 11-dimensional feature matrix and binary labels (`label = 1` when `warning_level` $\geq$ `1`). The data is split into 70/15/15 train/validation/test slices with stratification on the label. Optuna runs 25 trials over nine XGBoost hyperparameters (`n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `min_child_weight`, `gamma`, `reg_alpha`, `reg_lambda`) with five-fold cross-validated F1 as the objective; monotonic constraints fix the per-feature direction and `scale_pos_weight` addresses the class imbalance. After the best hyperparameters are chosen, the decision threshold is calibrated on the validation slice by sweeping from 0.20 to 0.78 in 0.02 increments and picking the value that maximises validation F1. The model is then retrained on the union of train and validation slices with those hyperparameters, evaluated on the never-seen test slice (accuracy, precision, recall, F1, AUC-ROC, confusion matrix), and three artefacts are written under `backend/data/models/`: `xgboost_v1.pkl` (the serialised model), `feature_names.json` (the ordered FEATURE_NAMES at training time), and `metrics_v1.json` (every metric plus the calibrated threshold, the chosen hyperparameters and timing information).

3.3.0.3 Inference (online). The `GET /predictions/me` endpoint first checks freshness by comparing the most recent `Prediction.created_at` against the maximum `Enrollment.updated_at` for the student: if the cached prediction is newer, it is returned as-is and no inference takes place. Otherwise the pipeline calls `warning_engine.sync_current_warning_level` and `sync_student_stats` to refresh the GPA, credits and warning level before features are extracted, fetches all enrollments with their courses eagerly loaded, and short-circuits with a "cold-start" response if the student has no enrollment history. The same `extract_features` routine used during training builds the 11-dimensional vector, the XGBoost `predict_proba` returns the raw probability, and `RiskExplainer.explain(features, top_k=5)` computes per-feature SHAP values, drops contributions below $|0.01|$, filters out contradictory signals, normalises the surviving absolute values to sum to 100%, and labels each contribution with a Vietnamese phrase. A rule-based calibration layer then post-processes the raw probability: each rule independently checks a domain condition (cumulative GPA below 1.2 forces a floor of 0.78, GPA below 1.6 forces 0.62, three or more unresolved Fs force 0.55, etc.) and the calibrated score is the maximum of the raw model output and every rule floor that fires. The endpoint persists a new `Prediction` row carrying the calibrated risk score, the discrete risk level (low/medium/high/critical), the SHAP factor list and the per-course pass probabilities, and returns the payload to the client. If the model is not loaded at start-up (no compatible artefact, or feature-name mismatch with the current `FEATURE_NAMES`) the endpoint short-circuits with `503 Service Unavailable`.

3.3.0.4 Batch inference. The same `predict_for_student` routine is invoked in a loop by `predict_batch`, which powers both the administrator's manual batch trigger (`POST /predictions/batch-run`) and the APScheduler cron job `predictions_batch_daily` that fires every day at 02:00 Asia/Ho_Chi_Minh. Sharing the routine guarantees byte-identical `Prediction` rows whether a run is automated or operator-driven.

3.4 Retrieval-Augmented Generation Pipeline

The chatbot subsystem implements a Retrieval-Augmented Generation (RAG) pipeline that combines hybrid retrieval over an internal regulation corpus with a large language model for natural-language answer composition. As with the prediction subsystem, an offline-style ingestion path (run when an administrator uploads a document) is cleanly separated from an on-line query path (run on every chatbot request); both paths share the same `documents` table backed by `pgvector`.

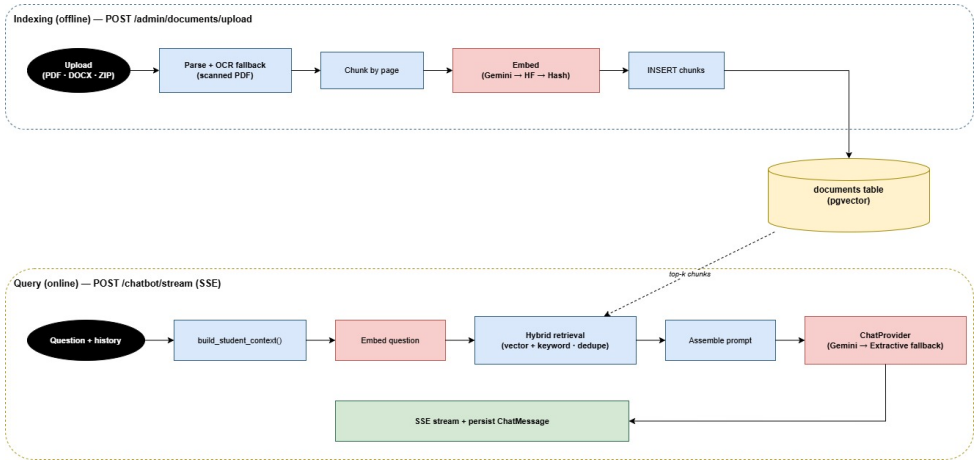

Figure 30: *End-to-end RAG pipeline — offline indexing (top half) and online query with hybrid retrieval and provider fallback (bottom half), sharing the same `documents` pgvector store*

3.4.0.1 Indexing (offline). The administrator-facing upload endpoint accepts PDF, DOCX, TXT, MD, or ZIP archives and forwards the bytes to `ingest_document` in `app/ai/chatbot/vectorstore.py`. PDF parsing uses PyMuPDF to extract text page by page in two modes (`"text"` and `"blocks"`); if the resulting text has fewer than 80 letters across all pages — the heuristic for scanned PDFs without a text layer — the function falls back to Tesseract OCR at `RAG_OCR_DPI` (default 220) with the `vie+eng` language pack, capped at `RAG_OCR_MAX_PAGES` pages (default 120) to bound worst-case cost. DOCX files go through `python-docx`; TXT and MD files use a multi-encoding decode chain (UTF-8, UTF-16, CP1258, Latin-1). The resulting pages are split by `chunk_pages` into overlapping word-windows of `RAG_CHUNK_SIZE` words (default 800, hard floor of 200) with `RAG_CHUNK_OVERLAP` words of overlap (default 120) so contextual references survive a chunk boundary. Each chunk is then embedded by the active `EmbeddingProvider` (Gemini's `embedding-001` if an API key is configured, Hugging Face if that is set instead, or the deterministic 768-bucket hash provider as a never-fail default) via `embed(content, task_type="retrieval_document")`, and a new `Document` row is inserted with the chunk text, the 768-dimensional embedding vector, the chunk index, the page number, a metadata JSON recording which provider produced the vector, and the uploader's user ID.

3.4.0.2 Query (online). The query path is implemented in `app/ai/chatbot/chains.py` as two parallel entry points: `ask_chatbot` (returns a complete response) and `stream_chatbot_response` (yields Server-Sent Events). They share every retrieval and prompt-assembly step. The pipeline first loads the twelve most recent `ChatMessage` rows for conversation context (newest first by `created_at`, then reversed) and assembles a Vietnamese-language profile string from the student record via `build_student_context` (name, MSSV, GPA, warning level, failed-course summary split into unresolved and recovered groups). The question is then embedded through the same provider used during ingestion and `search_documents` runs two retrievals in parallel: **vector retrieval** ordered by `Document.embedding.cosine_distance(query_vector)`, taking the top $4 \times$ `RAG_TOP_K` candidates so that documents with many chunks cannot monopolise the final selection; and **keyword retrieval** that tokenises the question, drops Vietnamese stopwords, matches a curated multi-word phrase list ("điểm trung bình tích lũy", "cảnh báo học vụ", "buộc thôi học", "xếp loại tốt nghiệp", etc.) and scores chunks by phrase and single-token hits with extra weight on the GPA-band thresholds. The two hit lists are concatenated, deduplicated by document ID (higher score wins), labelled `merged` where both retrievals fired, and capped at three chunks per source file for citation diversity. Each surviving hit becomes a `ChatCitation` with a query-centred snippet of up to 520 characters.

3.4.0.3 Prompt assembly and provider fallback. The citations are formatted into a numbered context block, concatenated with the system instructions, the conversation history (included verbatim only when the current question contains an anaphoric reference such as "môn đó" or "cái này"), the personal context and the user's current question, then dispatched to the active `ChatProvider`. The default `GeminiChatProvider` tries the model named in `settings.GEMINI_CHAT_MODEL` first and then walks through a static fallback list (`gemini-2.5-flash`, `gemini-2.0-flash`, `gemini-flash-latest`) so a single regional outage does not break the chatbot. If the primary provider raises `ProviderConfigError` (missing API key, rate limit, network failure, or misconfigured model name), the `ExtractiveChatProvider` is instantiated immediately and runs against the same inputs — its response composes a Vietnamese answer directly from the retrieved chunks with no external LLM, and is prefixed with a short notice so the user knows the answer was extracted rather than generated. The extractive provider is also the universal floor because it has no external dependencies and never raises.

3.4.0.4 Streaming and persistence. For streaming requests each provider token yields a `data: {"type":"delta","content":"..."}` SSE frame; when the stream ends a closing `data: {"type":"done","citations":[...],"provider":"..."}` frame carries the citation array and the name of the provider that actually answered. Both the user message and the assistant message (with citations JSON) are then inserted into `chat_messages` within the same database session, ensuring the citation chips are reproducible from history alone when the student reopens the chatbot page.

4 User Interface Design

The user interface is implemented as a Next.js 14 single-page application styled with Tailwind CSS and built from `shadcn/ui` primitives. The visual language follows the HCMUT brand (deep blue `#003087` as the primary action colour, neutral grey backgrounds, accent reds and greens for risk and positive state) and is applied uniformly across the three user surfaces: the public landing and authentication pages, the student workspace, and the administrator workspace. This section presents the design of every page as it ships in the prototype, in the order a typical user would encounter them. Each screenshot is annotated with the layout regions, primary actions, and the data the page surfaces.

4.1 Public-Facing Pages

4.1.0.1 Landing page. The landing page introduces the project with a sticky navigation bar (HCMUT logo, the bilingual toggle between Vietnamese and English, and a primary action to sign in) and a full-height hero section over a blue gradient backdrop with decorative geometric accents. The hero displays a personalised greeting and a single call-to-action that routes to the login page. The page is purely static, makes no API calls, and adapts cleanly from desktop to mobile widths thanks to the Tailwind responsive grid.

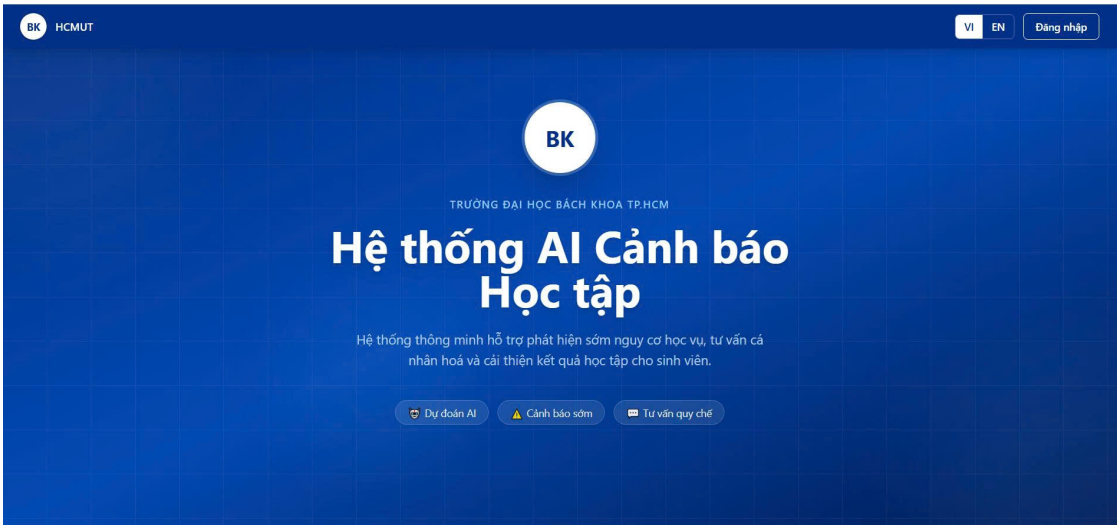

Figure 31: *Landing page with bilingual content and gradient hero*

4.1.0.2 Student login. The student-facing login screen is a centered card containing two text inputs (email and password), a primary submit button, and inline links to the registration page and the language toggle. The card is rendered by a shared `LoginCard` component instantiated in student mode; this same component drives the administrator login below, ensuring identical look-and-feel across the two roles.

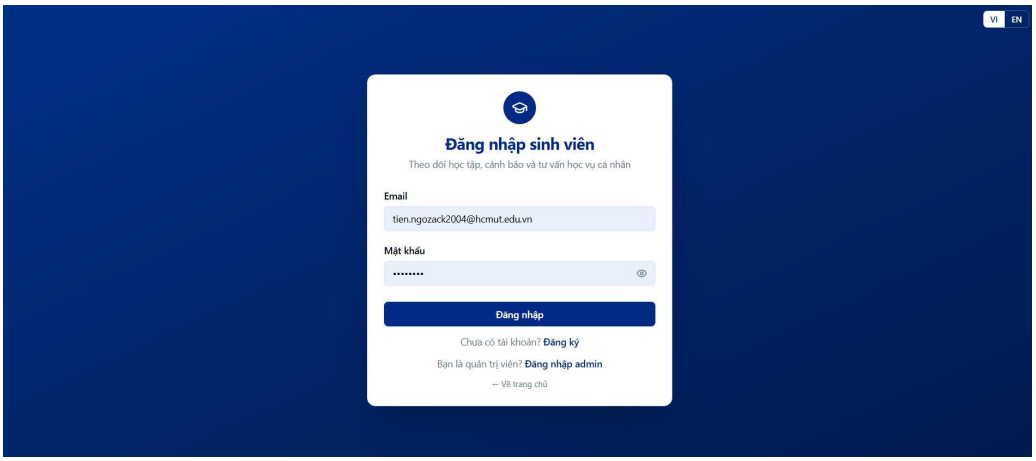

Figure 32: *Student login*

4.1.0.3 Administrator login. The administrator login reuses the same `LoginCard` component but in administrator mode, which changes the card title and the submit-success redirect target. The visual layout is intentionally

identical to the student login, because the only real difference is the role embedded in the resulting JWT.

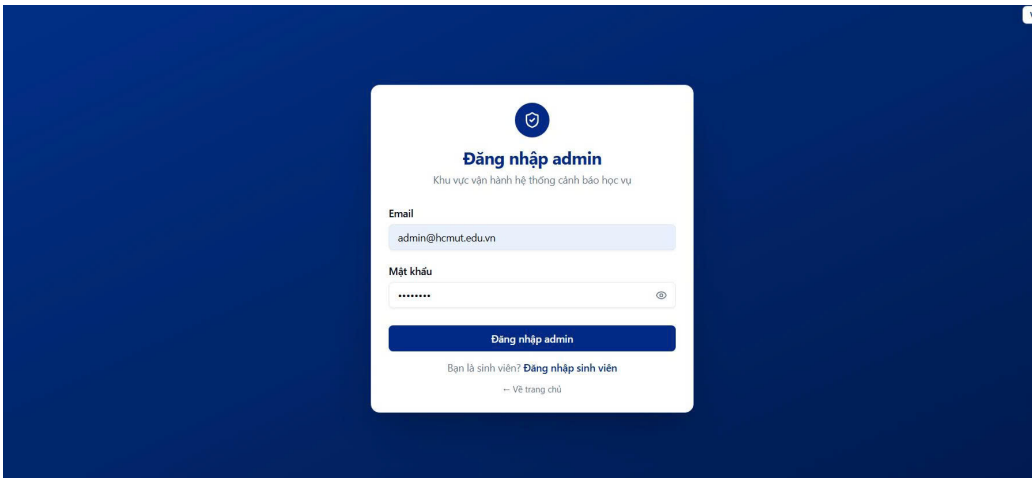

Figure 33: *Administrator login (shared component, different mode)*

4.1.0.4 Student registration. Registration uses a longer card-based form with a header (HCMUT logo, page title, helper text), grouped input fields, a faculty selector dropdown, and live Zod-driven inline validation. After a successful submission the card swaps to a confirmation state that informs the new user the account is ready and links to the login page; validation failures keep the form mounted and show the offending field's error inline.

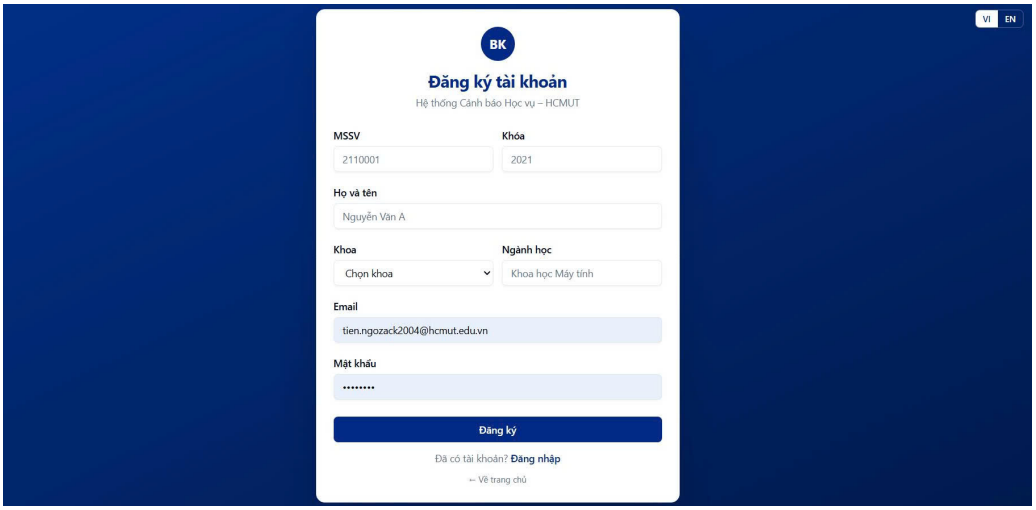

Figure 34: *Student registration with field-level Zod validation*

4.2 Student Workspace

The student workspace shares a two-column layout: a fixed sidebar on the left containing the nine main navigation items and the notification bell, and a fluid main content area on the right. The same `StudentLayout` wraps every student route and is responsible for the access-control redirect.

4.2.0.1 Dashboard. The dashboard opens with a personalised greeting (one of nine tone variants chosen by warning level and GPA), a warning-level badge when applicable, and a one-sentence insight about the most recent semester. Five stat cards display the cumulative GPA, credits earned, credits currently in progress, the count of failed courses, and the most recent AI risk score. Below the cards sit a GPA trend chart (a Recharts area chart with a gradient fill) and a semester-details table. The risk card is clickable and routes to the predictions page; if the AI model has not yet been trained, the dashboard degrades gracefully and the prediction card simply reports "not yet computed."

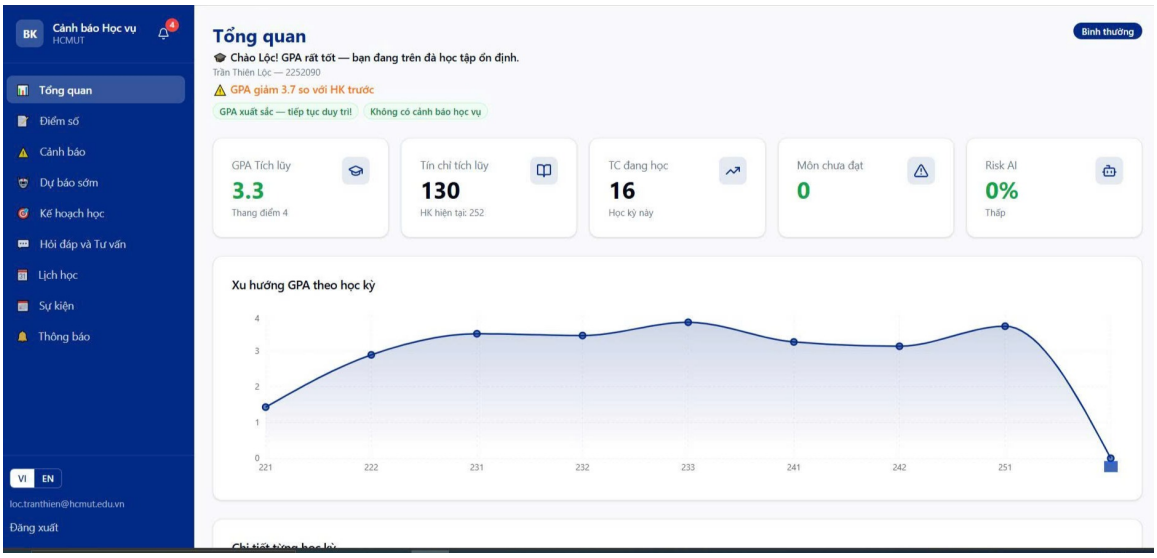

Figure 35: *Student dashboard — personalised greeting, stat cards and GPA trend*

4.2.0.2 Grades management. The grades page is the most interaction-heavy student surface. The header carries three primary actions (add course, import from myBK, delete all). A semester filter band lets the student narrow the listing to a single semester or view every semester grouped under collapsible cards with per-semester GPA badges. Each enrollment row exposes inline editing of the four component scores (midterm, lab, other, final) and the attendance rate. The "Add course" dialog ships with twelve preset weight templates covering the typical HCMUT grading patterns plus a custom weights option. The "Import from myBK" dialog runs as a three-step wizard — the guide, a parsed preview that distinguishes new courses from updated ones, and a final confirmation.

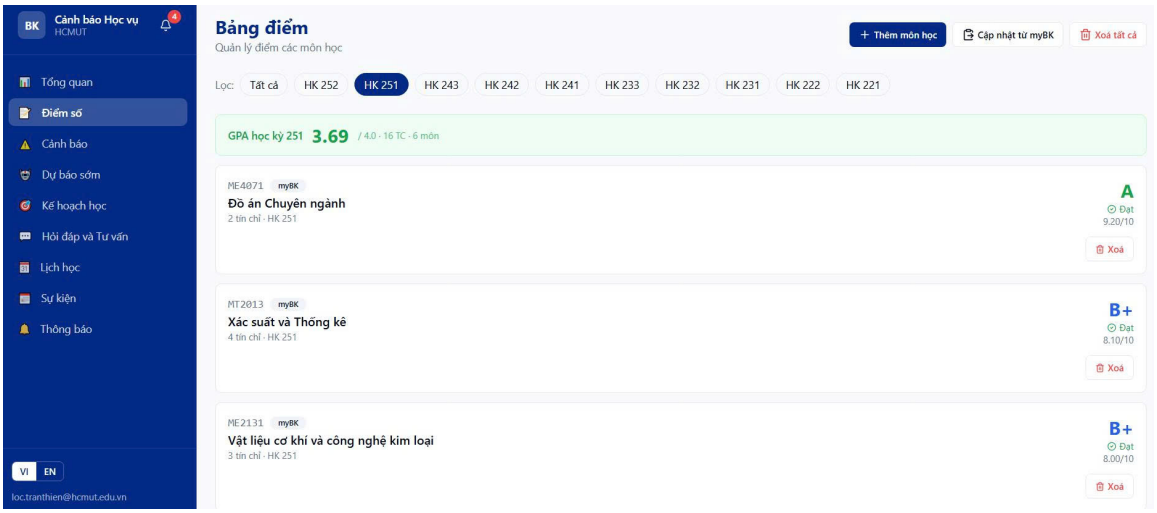

Figure 36: *Grades management — inline editing, semester grouping, and the 3-step myBK import dialog*

4.2.0.3 AI risk predictions. The predictions page is laid out as a three-column grid. The leftmost column renders the risk gauge as a Recharts radial bar chart spanning 225 to −45, with the risk percentage and discrete level (low/medium/high/critical) at the centre. The middle column lists the SHAP-derived contributing factors as horizontal bars colour-coded by direction (red for risk-increasing, green for protective). The right column is a table of currently-enrolled courses with their predicted pass probabilities, where the probability badge changes colour at the 70% and 50% thresholds. A 30-day risk-history area chart sits below the grid, and a refresh button at the top-right forces a fresh inference. When the model is not yet loaded the page surfaces a non-blocking message instead of failing the request.

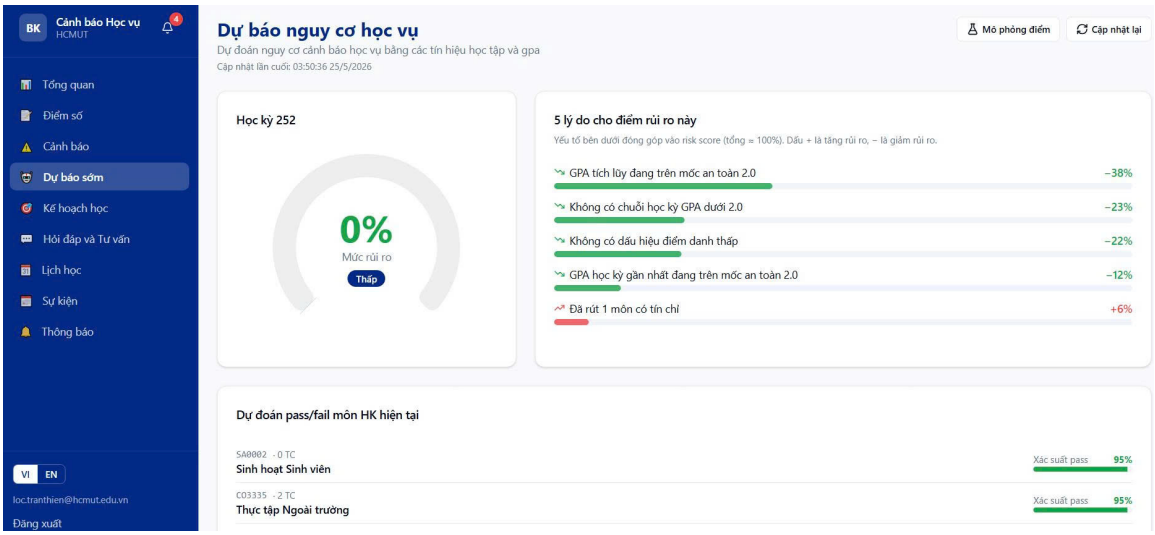

Figure 37: *Predictions page — radial risk gauge, SHAP factors, per-course probabilities, history chart*

4.2.0.4 What-if GPA simulator. The simulator page is a focused single-purpose tool reached from the predictions page. The body lists every currently-enrolled non-finalised course in a grid; each row exposes a number input clamped to the 0–10 range with 0.1 steps and a live grade letter that updates as the value changes. On the right, a result panel compares the baseline risk score against the simulated risk score with a delta indicator and a colour-coded badge. The simulator debounces input changes by 600 ms before submitting them, so the model is not invoked on every keystroke.

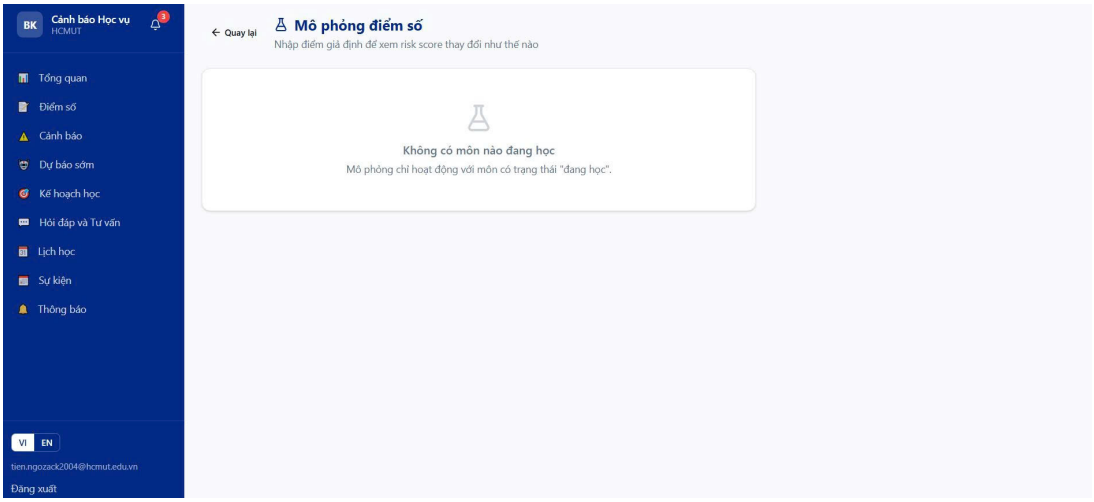

Figure 38: *What-if simulator — per-course score inputs and a baseline-vs-simulated comparison*

4.2.0.5 RAG chatbot. The chatbot is a full-height conversational interface. The header carries the topic indicator and a “clear history” control. The message feed renders user and assistant turns as Markdown via `react-markdown` with custom renderers for code blocks, tables, lists and blockquotes. Below the feed, the input area is a textarea with the conventional Enter-to-send / Shift+Enter-for-newline behaviour. On the very first visit the feed shows five suggestion pills tailored to the student’s current academic situation; clicking one pre-fills the input. Assistant responses include inline citation badges that show the document, page number, and retrieval method (vector, keyword, or merged), and the response streams in token-by-token over Server-Sent Events.

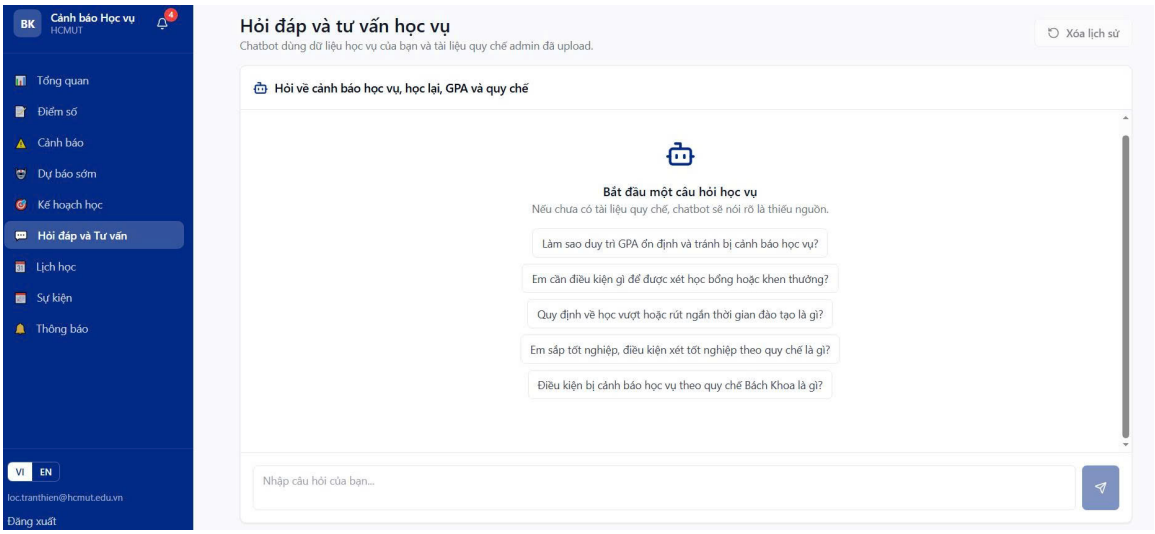

Figure 39: *RAG chatbot — streaming reply with citation badges, suggestion pills, and Markdown rendering*

4.2.0.6 Warnings history. The warnings page begins with a summary band (total, unresolved, per-level counts) and a tab strip that filters between all / unresolved / resolved. Below, each warning is rendered as a card with a coloured left border keyed to the level (yellow / orange / red), the originating semester, the reason text, the GPA snapshot at the moment of the warning, the AI risk score when the warning was AI-suggested, and a created-by badge (system or admin). A single button per card toggles the resolved flag, and resolved warnings are dimmed.

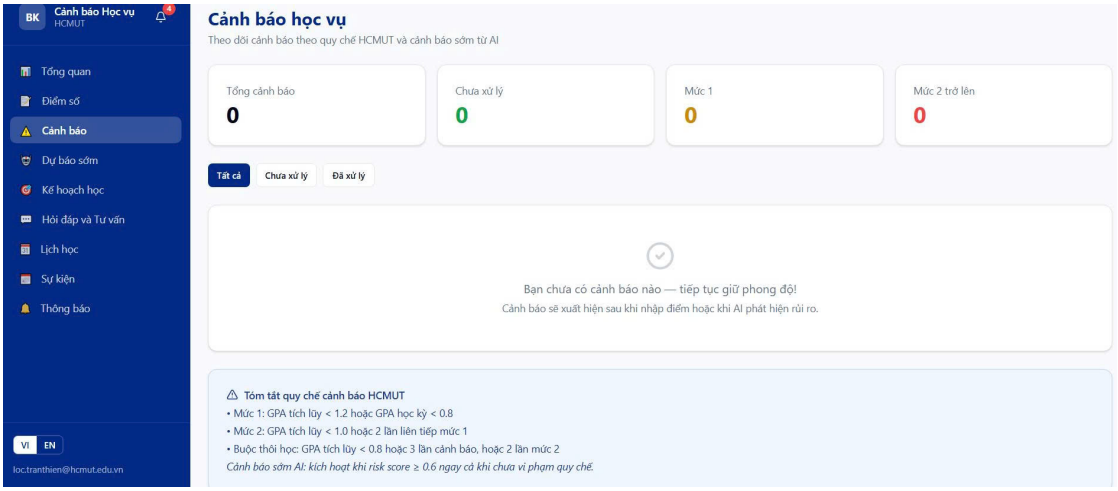

Figure 40: *Warnings history — per-level colour coding, filter tabs and the resolve toggle*

4.2.0.7 Study plan. The study plan page collects three small snapshot cards at the top (cumulative GPA, credits earned, unresolved failed courses) and then three thematic sections. The credit-load card splits into three boxes (minimum, recommended, maximum) with the recommended value emphasised and a rationale paragraph from the recommender. Below it, the retake-priority list shows each suggested course with a coloured priority badge (high / medium / low), the most recent grade, the last attempted semester, and the reason text. A final “suggested courses” section optionally lists electives that would improve the cumulative GPA.

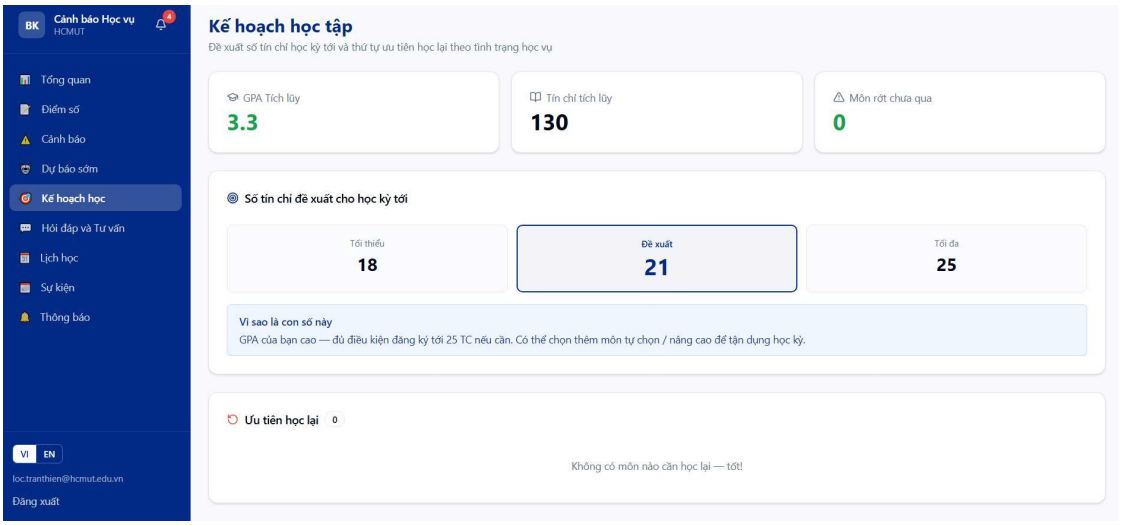

Figure 41: *Study plan — credit-load card with rationale and prioritised retake list*

4.2.0.8 Schedule (timetable + exams + personal events). The schedule page consolidates three calendars behind a tab strip. The week view is a 15-row by 7-column grid (HCMUT periods 1 through 15 on one axis, Monday through Sunday on the other), with course blocks rendered using a hash-stable colour palette so a given course always appears in the same colour. Adjacent periods of the same course are merged into a single multi-period block. The month view is a Recharts-free calendar with coloured dots distinguishing classes, exams, administrator-published events, and personal events. The list view sorts upcoming items chronologically. Three modals support importing the timetable from the myBK paste format, importing the exam schedule, and creating ad-hoc personal events.

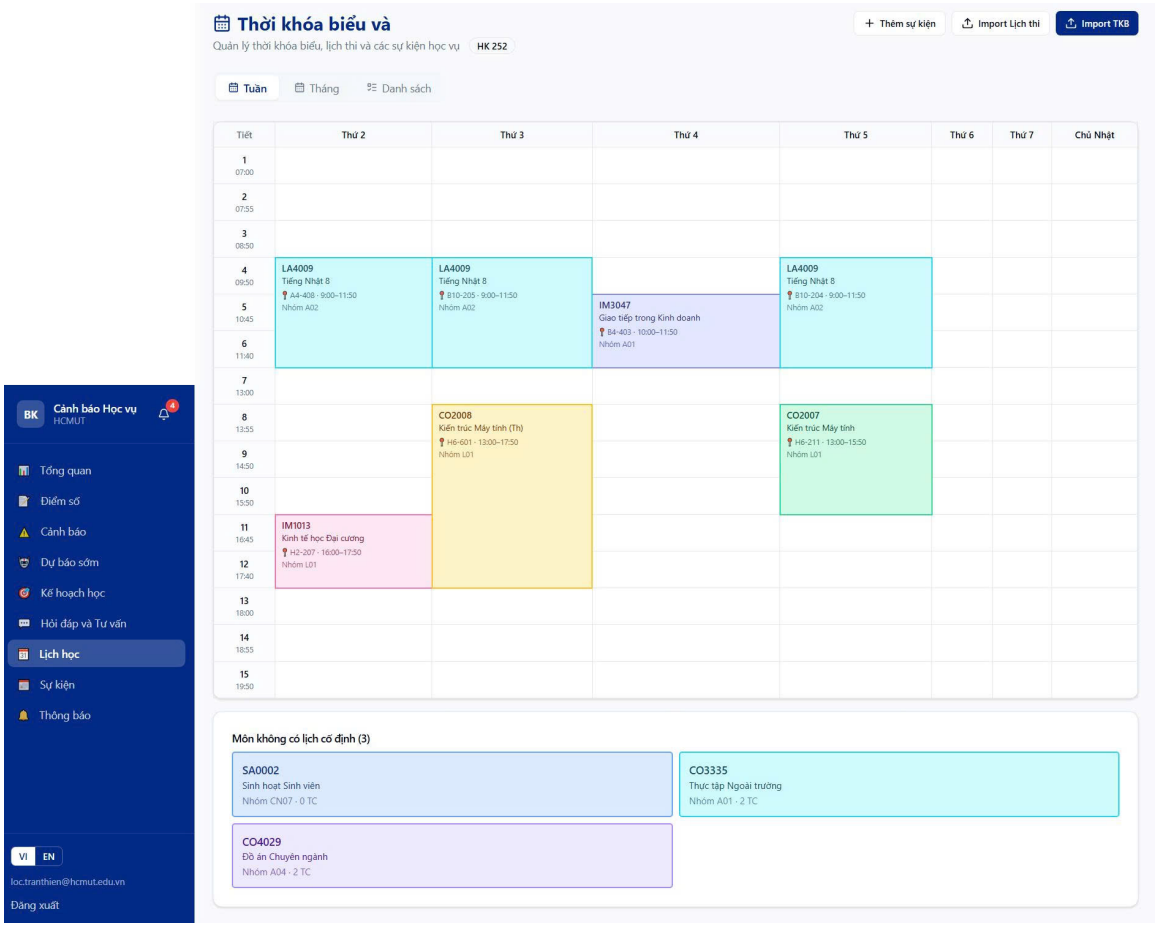

Figure 42: *Schedule — week / month / list tabs with myBK import and personal event modals*

4.2.0.9 Events. The events page lists the academic events the student belongs to (university-wide, faculty-specific, or cohort-specific). A two-tab control switches between “upcoming” (events with a future `start_time`)

and “all”. Each card is iconified by event type (exam, submission, activity, evaluation), shows the date range, and adds a destructive badge when the event is mandatory. Events within the next seven days carry a countdown badge that turns red when the event is within a day.

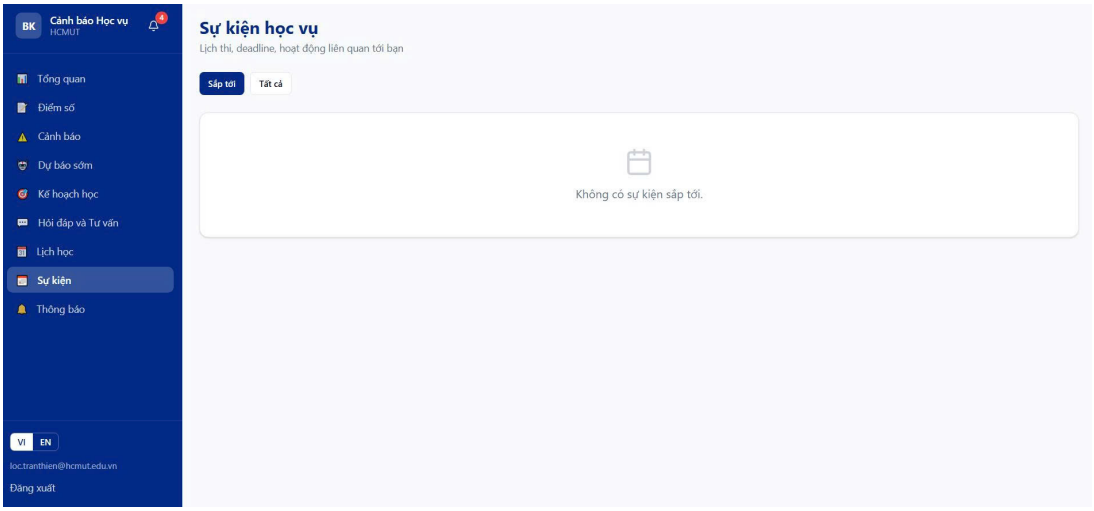

Figure 43: *Events — upcoming/all tabs with type-keyed icons and countdown badges*

4.2.0.10 Notifications. The notifications page leads with a header showing the unread count and a “mark all as read” button. A toggle card at the top controls whether outgoing email notifications are sent. The body lists notifications with a type icon (warning, reminder, event, system), the title and content, the timestamp, and an indicator showing whether the corresponding email has been dispatched. Marking a notification as read is optimistic: the UI updates immediately and reverts only if the API call fails.

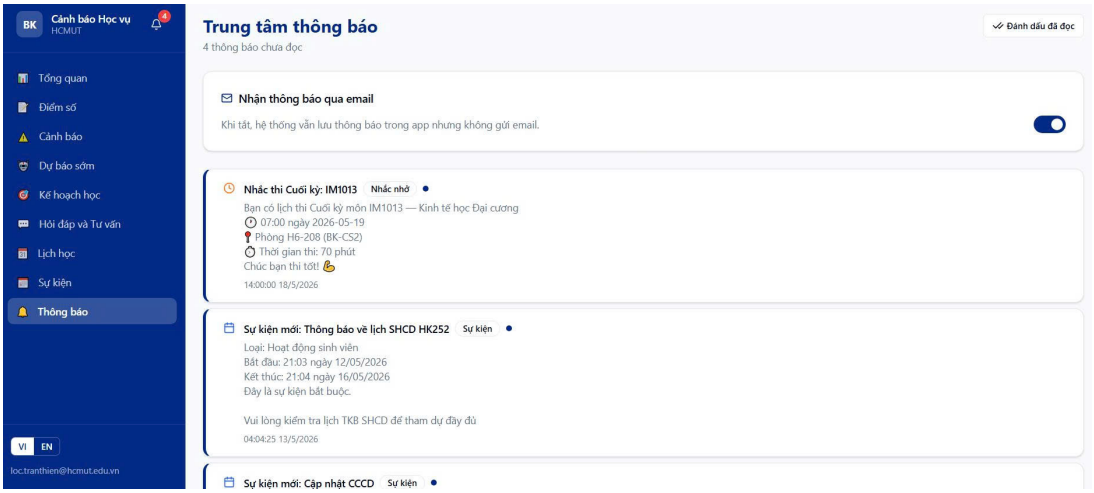

Figure 44: *Notifications — type-keyed icons, optimistic mark-read, email opt-in switch*

4.3 Administrator Workspace

The administrator workspace shares the same sidebar pattern as the student workspace but with seven items keyed to the operational tasks an Academic Affairs Office staff member performs. The administrator root path (`/admin`) is a server-side redirect to the dashboard so the sidebar always lands on a meaningful view.

4.3.0.1 Admin dashboard. The administrator dashboard opens with a row of four top-line counters (total students, total under warning, high-risk students, critical-risk students) and a last-updated timestamp. Two horizontal-bar charts summarise the distribution of students by risk level and the faculties with the most warnings. A ten-row table ranks the highest-risk students with their MSSV, faculty, GPA, warning level, and risk score, with each row linking to the corresponding student-detail page. When the database is empty, an inline call to action invites the operator to run the bulk import.

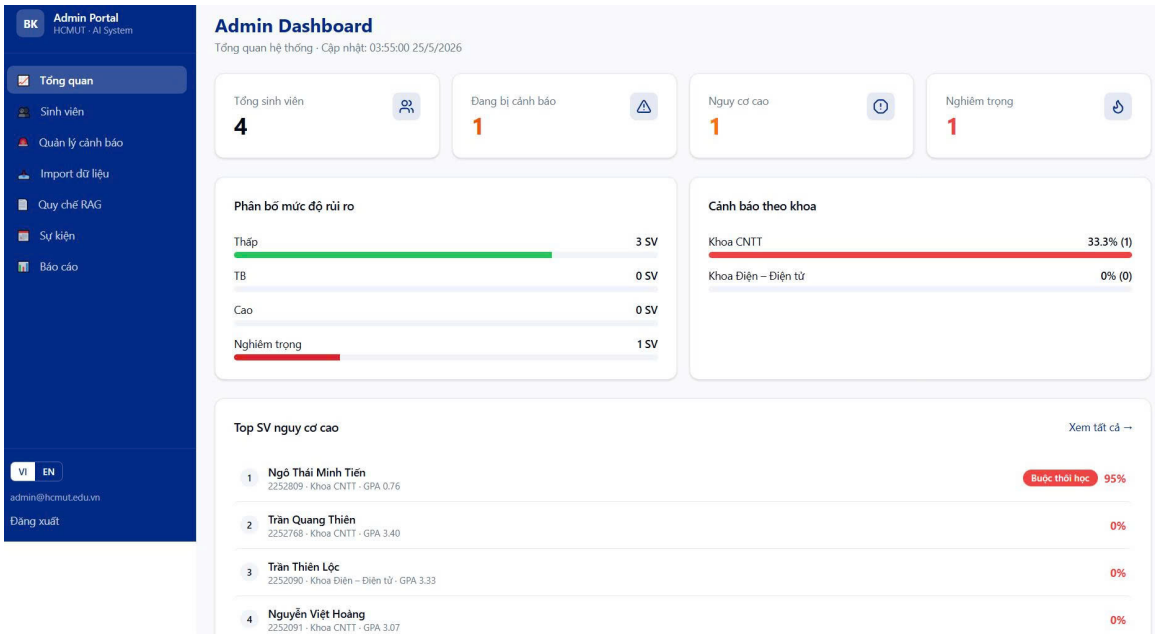

Figure 45: *Administrator dashboard — aggregate counters, risk + faculty bars, top-10 at-risk table*

4.3.0.2 Student list. The student list is a paginated table (twenty rows per page) with a search field and three filter chips (all, on warning, high risk). Each row shows the MSSV, full name, faculty, the cumulative GPA (colour-coded against the safe threshold), the AI risk percentage, the warning-level badge, and a button that opens the student-detail page. The total record count and the current range (e.g., "1–20 / 450") are visible next to the page controls so the operator can gauge the scale of the cohort at a glance.

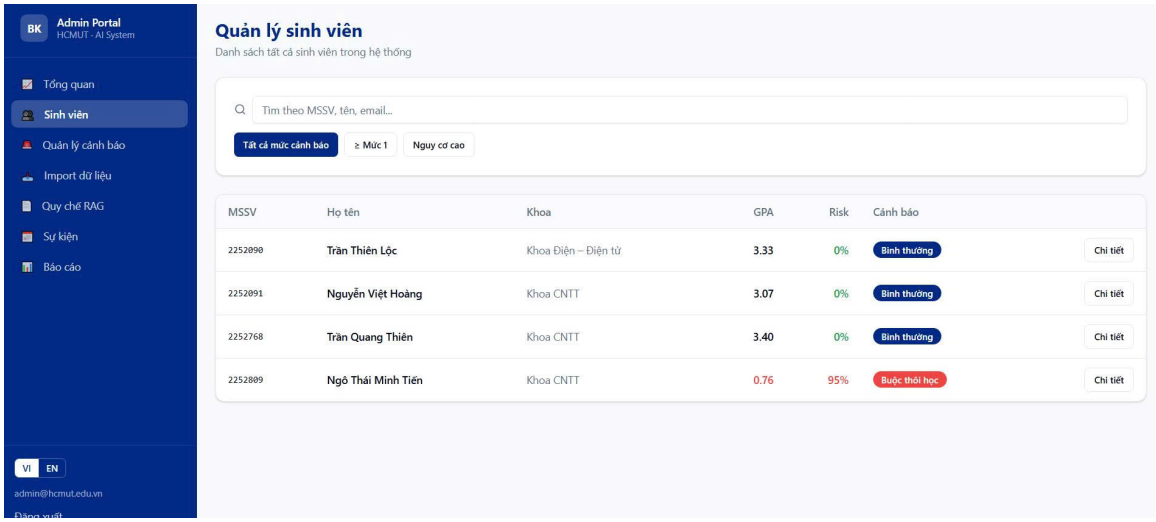

Figure 46: *Student list — search, filter chips, paginated table with GPA colour coding*

4.3.0.3 Student detail. The student-detail page is opened from the table or from the top-10 list on the dashboard. The header presents the student's identity (MSSV, name, faculty, major, cohort) and a primary action to issue a manual warning. The body is a three-column block consisting of a profile card, a four-cell statistics card, and an optional risk-factors panel. Below it sit a GPA trend chart and the full warning history. The manual-warning form is in-page and asks for the level (1–3), the semester, and a free-text reason.

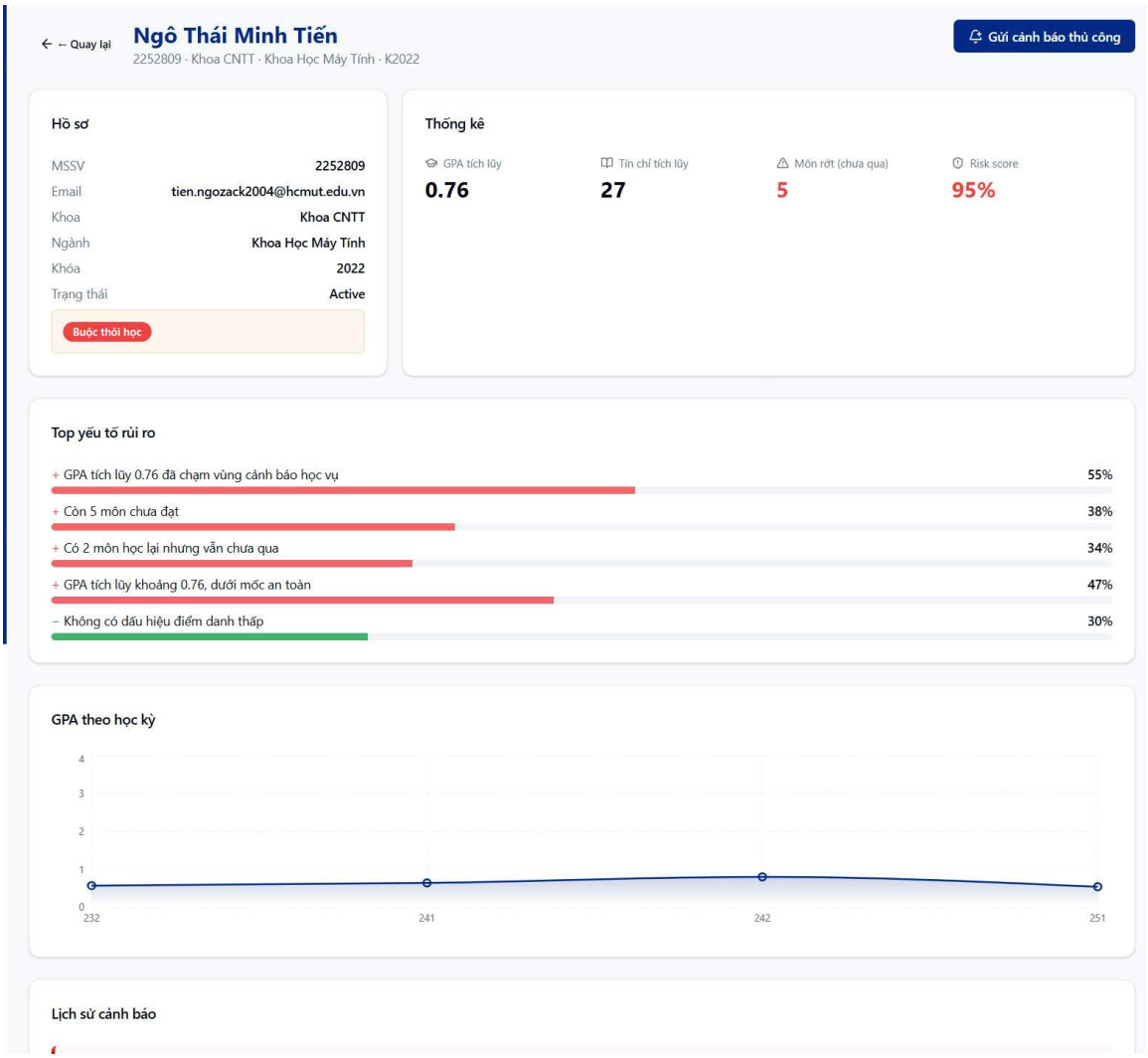

Figure 47: *Student detail — profile, statistics, risk factors, GPA trend and warning history*

4.3.0.4 Warning approval queue. The warning page is built around the queue of warnings the system has suggested but that the administrator has not yet approved. The top of the page exposes the AI suggestion threshold as an editable percentage with a save control, alongside three counters (pending, sent this semester, last batch timestamp) and a primary "run batch" button that triggers a sequential predictions-then-warnings pass. Below the controls, each pending entry shows the student, the proposed level, the faculty, the AI risk score, the reason, and three actions: open the detail modal, approve and send, or dismiss without sending.

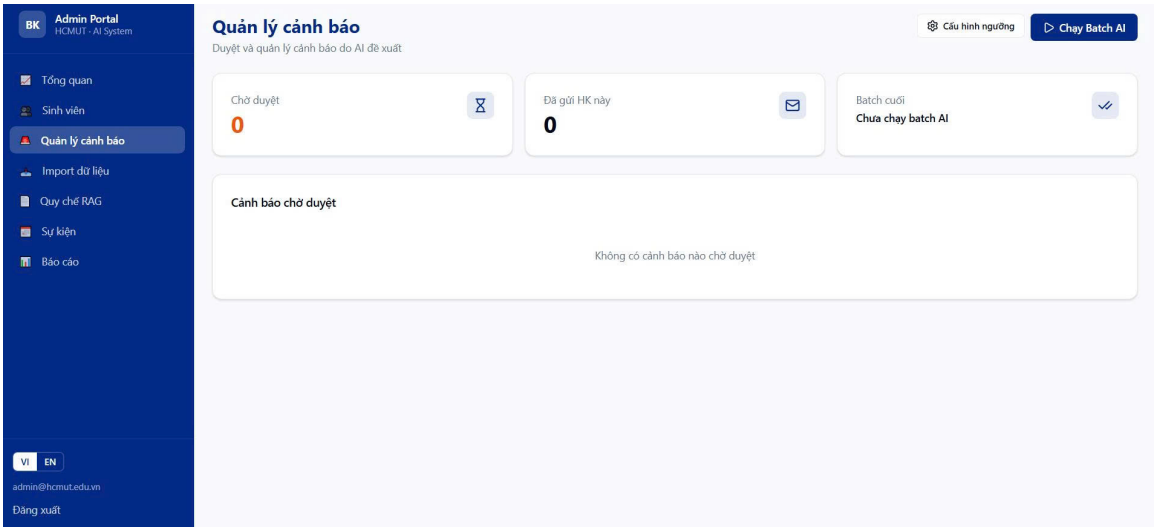

Figure 48: *Warning approval — threshold editor, batch trigger, pending queue with per-item actions*

4.3.0.5 Bulk import. The bulk-import page presents two upload boxes (one for students, one for grades), each accompanied by a template-download button. Following an upload, an import-result card surfaces the created/updated/failed counts and exposes a togglable table of row-level errors that names the offending row, column, and reason. A history table below the boxes lists the previous import runs so the operator can audit what was loaded when.

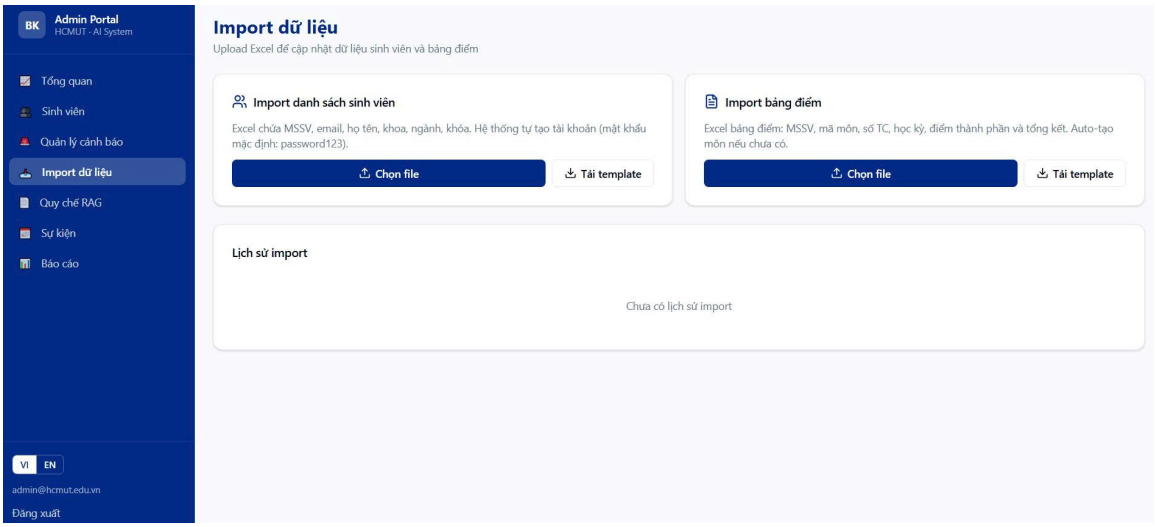

Figure 49: *Bulk import — upload boxes, template downloads, result card with error details, run history*

4.3.0.6 RAG document repository. The documents page manages the corpus the chatbot retrieves from. A batch-upload button accepts multiple PDF, DOCX, TXT, MD files (and ZIP archives that are unpacked at the API). Each row in the table shows the filename, the chunk and page counts, an active/inactive toggle, the upload timestamp, and a delete button. After an upload, a summary card reports per-file success or failure with toast notifications for the slower ingestion path.

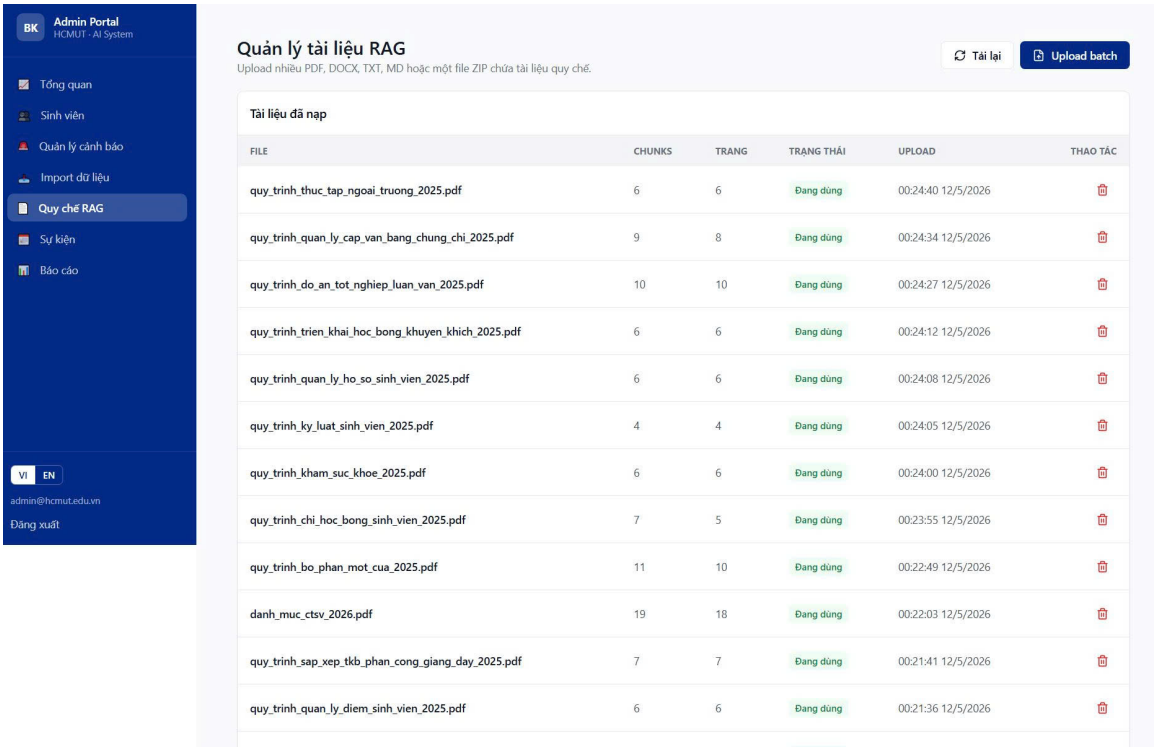

Figure 50: *Document repository — batch upload, per-file status, active toggle and chunk counts*

4.3.0.7 Event publishing. The events page lets the administrator create academic events with a type (exam, submission, activity, evaluation), a start and end time, an audience selector (all students, a particular faculty, or a particular cohort), an optional audience value, a description, and a mandatory flag. The form is rendered inline rather than in a dialog and pre-fills cleanly when editing an existing event. The list below the form mirrors the

cards the student sees on their own events page, with edit and delete controls visible only to administrators.

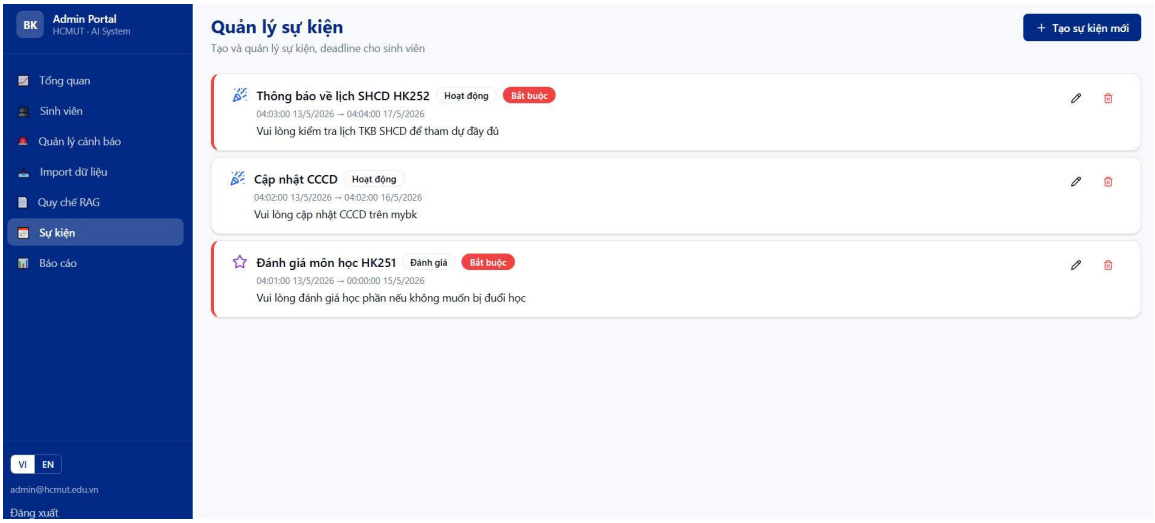

Figure 51: *Event publishing — inline form, audience selector, list of published events*

4.3.0.8 Reports and exports. The reports page collects the institutional-level statistics into four top-line stat cards (average GPA, warning rate, improvement rate, pass rate) and four chart cards (warnings by semester as a bar chart, GPA distribution as a colour-coded histogram, risk distribution as a donut, and faculty warning rates as horizontal bars). The bottom of the page exposes four export panels (warnings report, GPA report, AI report, warned-students list), each offering a PDF and an Excel download. The downloads stream the file directly from the backend and respect the `Content-Disposition` filename returned by the server.

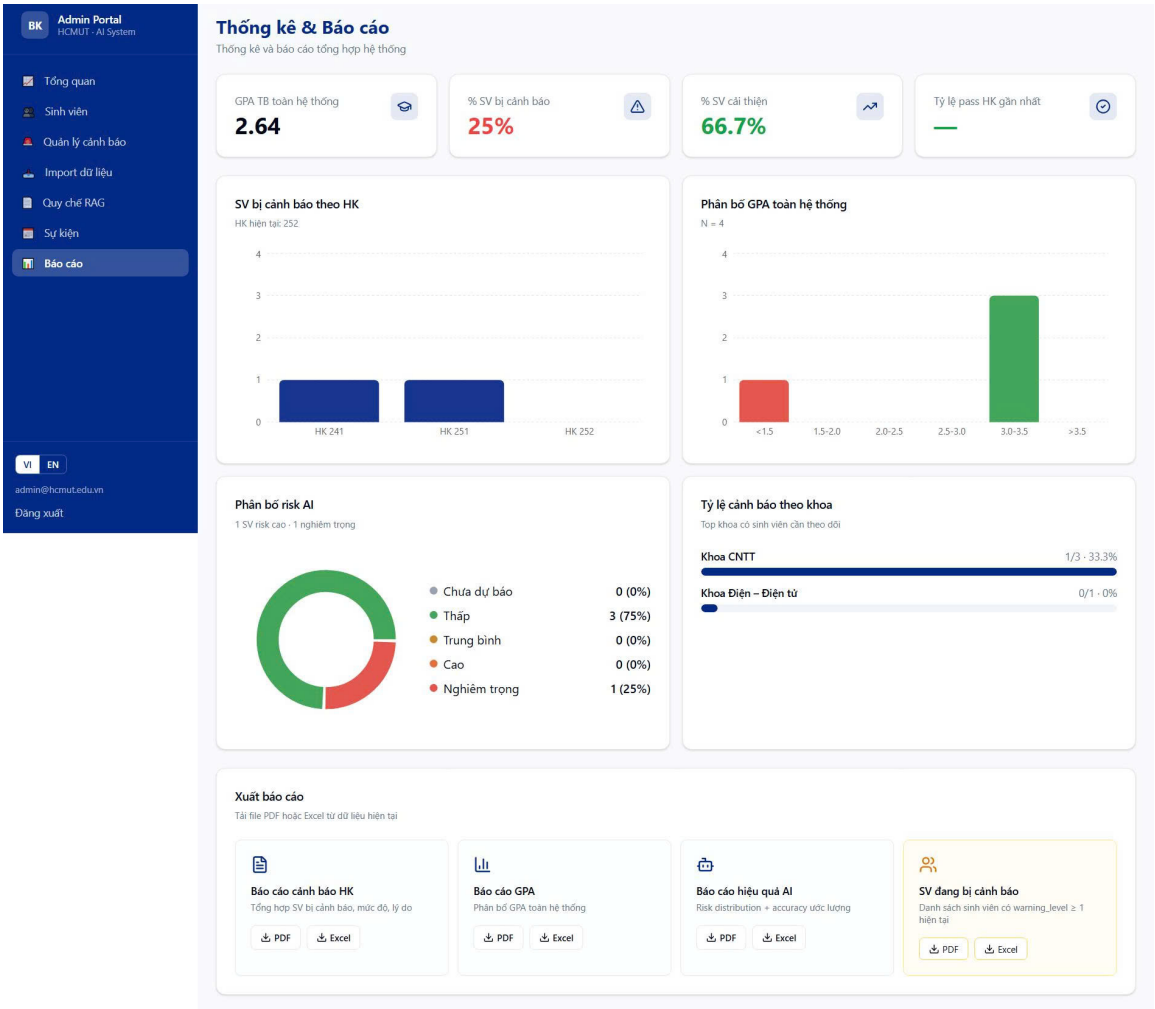

Figure 52: *Reports — four statistics cards, four charts and export panels in PDF and Excel*

4.4 Cross-cutting Design Elements

Several design elements recur across every page and warrant a brief consolidated description. The colour system reserves deep blue (`#003087`) for primary actions and the brand header, neutral greys (`slate-50` through `slate-900` from the Tailwind palette) for backgrounds and body text, red for risk and destructive states, green for protective signals and successful operations, and amber for cautionary states. The component library is `shadcn/ui` on top of Radix and Base UI primitives, providing accessible card, button, input, badge, dialog, skeleton, and separator components copied directly into `frontend/components/ui/`. Feature components in `frontend/components/` include the two sidebars, the notification bell with its unread count, the language toggle, the login card, and the onboarding tour. Charts are produced through Recharts (area, radial-bar, bar, donut, and pie variants); icons are from `lucide-react`; toast notifications use `sonner` for non-blocking feedback; and skeleton loaders are shown during data fetches to communicate that the page is alive while the backend responds.

5 System Implementation

This chapter walks through the implementation of the user-visible workflows in the order a real student or administrator typically experiences them. Each section pairs the relevant screenshots with the concrete code paths — the API endpoints, the service functions, the database operations, and the AI subsystem invocations — that the workflow exercises, so the reader can trace any feature back to the source files described in Chapter 3.

5.1 Account Creation and Authentication

The authentication flow connects three frontend pages (registration, student login, administrator login) to two backend endpoints (`POST /api/v1/auth/register` and `POST /api/v1/auth/login`) and the JWT session handling code.

5.1.0.1 Registration. The student opens the registration page and completes a single card-based form covering email, password, MSSV, full name, faculty, major and cohort. The form runs Zod validation on the client (`frontend/app/auth/register/page.tsx`) before submitting; on success it calls `authApi.register(payload)` from `frontend/lib/api.ts`, which POSTs the payload to `/auth/register`. The endpoint in `backend/app/api/v1/auth.py` checks that the email and MSSV are not already taken (returning `400 Bad Request` otherwise), hashes the password with bcrypt through `hash_password`, persists the `User` row, flushes to obtain the new ID, persists the matched `Student` row referencing that ID, commits the transaction, and returns a freshly signed JWT. The frontend stores the token in the Zustand auth store — which writes it both to `localStorage` (for the Axios interceptor) and to the `access_token` cookie (for the App Router middleware) — and shows the registration-complete card.

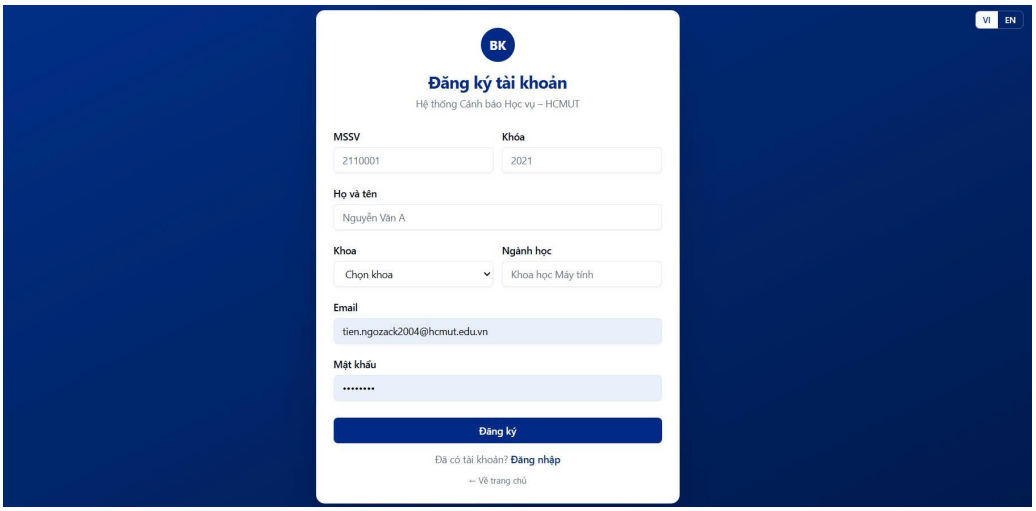

Figure 53: *Registration card with field-level Zod validation and the success state*

5.1.0.2 Login. The student or administrator opens their respective login page, both of which render the shared `LoginCard` component but in different modes. On submit, `authApi.login(payload)` POSTs to `/auth/login`; the endpoint looks up the user by email, verifies the password against the bcrypt hash through `verify_password`, rejects with a deliberately identical `401 Unauthorized` message for both unknown emails and wrong passwords (to defeat account enumeration), and rejects disabled accounts with `403 Forbidden`. On success the token is returned and stored, and the layout that the user lands on (student or administrator) is selected from the role embedded in the JWT payload.

5.2 Personal Dashboard

After authentication the student is routed to `/student/dashboard`, which loads through `frontend/app/student/dashboard/page.tsx`. The page issues three parallel API calls on mount — `studentApi.dashboard()`, `studentApi.gpaHistory()` and `predictionsApi.me()` — and renders a skeleton placeholder until the three resolve. The dashboard endpoint `GET /api/v1/students/me/dashboard` consults `warning_engine.sync_current_warning_level` to refresh the warning level, then assembles a snapshot of the cumulative GPA, the credits earned and in progress, and the failed-course count.

The visible layout consists of a personalised greeting (one of nine tone variants selected by the warning level and GPA), five summary stat cards, a Recharts area chart of semester GPA, and a one-row table of semester details.

The prediction card is rendered defensively: if the AI service responds with `503` (model not yet trained) the card simply displays "not yet computed" instead of failing the page, so a freshly bootstrapped environment is still usable. Clicking the prediction card navigates to `/student/predictions`, where the deeper inference flow described in Section 5.4 runs.

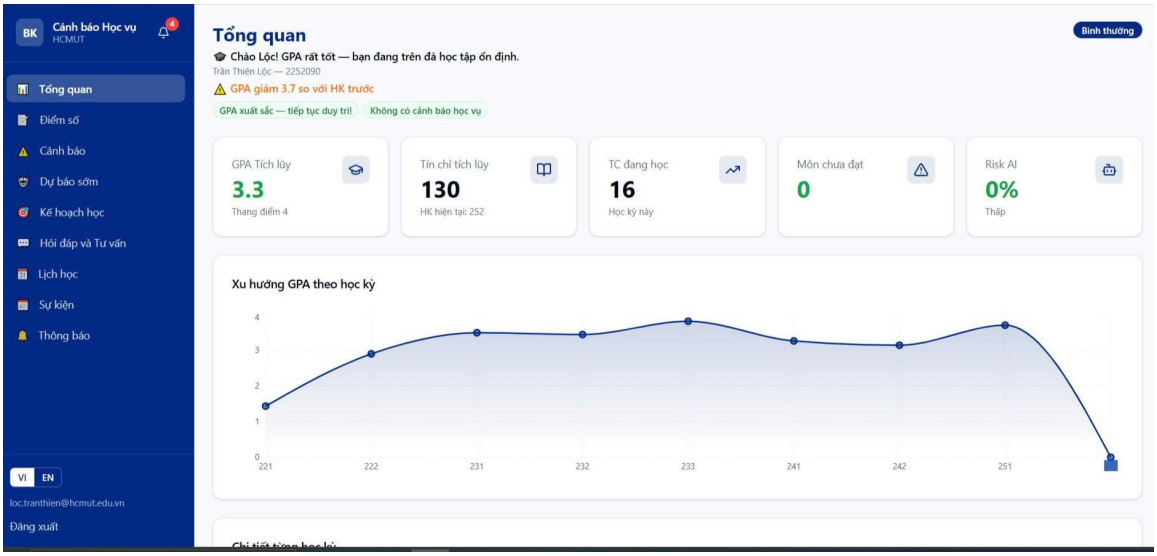

Figure 54: *Student dashboard — summary cards, GPA trend chart, semester table and AI risk card*

5.3 Grade Management and myBK Import

The grades workflow is by far the most interactive student-facing surface, because it is the single source of truth for everything downstream — the cumulative GPA, the warning-level calculation, the AI risk score, and the study-plan recommender all consume `Enrollment` rows that originate here.

5.3.0.1 Browse and inline edit. The grades page (`frontend/app/student/grades/page.tsx`) issues `studentApi.enrollments()` on mount and groups the returned rows by semester. The semester filter band at the top either narrows to a single semester or shows every semester as a collapsible card with its own GPA badge derived from `studentApi.gpaHistory()`. Inside each card, every enrollment row exposes the four component scores (midterm, lab, other, final) and the attendance rate as inline-editable fields. On save, the row calls `studentApi.updateGrades(enrollment_id, payload)` which targets `PUT /api/v1/students/me/enrollments/{id}/grades`. The endpoint validates that the four weights sum exactly to 1.0 in the Pydantic schema, recomputes the weighted total and derives the grade letter via `gpa_calculator.score_to_grade_letter`, updates the row, synchronises the student-level GPA, credits and warning level via `grade_aggregator.sync_student_stats` and `warning_engine.sync_current_warning_level`, and invalidates any existing `Prediction` record so the next visit to the predictions page recomputes the risk score.

5.3.0.2 Manual course addition. The "Add course" dialog is a small wizard. The first screen offers twelve preset weight templates that cover the typical HCMUT grading patterns (for example 30% midterm +70% final, or 20% midterm +30% lab +50% final) plus a custom-weights option. After the operator picks a template, the dialog reveals the course-code, name, semester and component-score inputs. On submit the dialog POSTs to `/api/v1/students/me/enrollments/manual` which finds or creates the matching `Course` record (so unseen course codes do not break the workflow), persists the new `Enrollment` with `source = "manual"`, and runs the same post-save synchronisation as the inline edit path.

5.3.0.3 myBK paste import. The most distinctive feature of the grades page is the three-step myBK import wizard, which lets the student copy their transcript from the HCMUT `myBK` portal directly into the system. The first step is a static guide showing the operator how to copy from `myBK`; the second step accepts the pasted text and runs the import as a dry-run first; the third step shows the parsed preview — distinguishing newly-created courses from updated ones — and confirms the commit. The parsing pipeline is implemented in `app/services/mybk_parser.py`: a regular expression detects semester headers such as "Năm học 2025-2026 / Học kỳ 1" and normalises them into the canonical `251` code, then each tab-separated row is validated against the course-code regex and routed into a `ParsedCourse` dataclass. After confirmation, the endpoint upserts the `Course` and `Enrollment` rows inside a single transaction, marks each row with `source = "mybk_paste"` and

`is_finalized = true`, and triggers the same post-save synchronisation as the manual flow. Finalised rows are then locked — subsequent attempts to edit them through the inline path return `409 Conflict`.

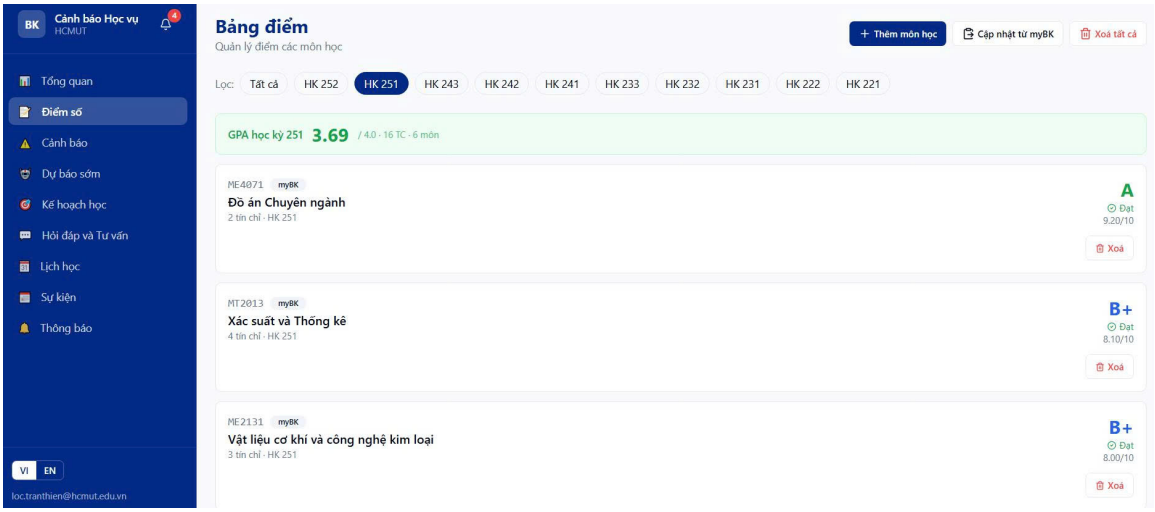

Figure 55: *Grades page — inline editing, semester grouping, weight template picker and the myBK import wizard*

5.3.0.4 What-if calculator. Each row also exposes a small per-course what-if helper that asks "what score on the remaining component do I need to achieve grade letter X?". This is purely a client-side helper (no API round-trip) and uses the same weighted formula as the backend so that the answer is consistent with what the system will compute when the score is actually entered.

5.4 AI Risk Prediction and What-if Simulation

The prediction and simulator pages share the same in-process inference pipeline described in Section **??**; the difference is that the prediction page reads through a freshness-aware cache, while the simulator never persists.

5.4.0.1 Prediction page. When the student opens `/student/predictions` the page (`frontend/app/student/predictions/page.tsx`) issues `predictionsApi.me()` which targets `GET /api/v1/predictions/me`. The endpoint in `backend/app/api/v1/predictions.py` first guards on `prediction_service.is_loaded` — if the model file has not been produced yet (or its embedded feature list disagrees with the current `FEATURE_NAMES`) it returns `503 Service Unavailable` and the frontend renders a friendly "model not ready" state instead of failing. Otherwise the endpoint runs a multi-step freshness check: it looks up the most recent `Prediction` row, compares its `created_at` against the maximum `Enrollment.updated_at` for the student, checks that the `feature_version` stored in the prediction's metadata still matches the current code, and finally re-extracts the feature vector to confirm that no field has changed even within a single timestamp. Only if every check passes is the cached row returned. Otherwise the endpoint invokes `prediction_service.predict_for_student(student, db, save=True)`, which synchronises the warning level and student statistics, calls `extract_features` for the eleven-dimensional feature vector, runs XGBoost's `predict_proba`, asks `RiskExplainer.explain` for the top-five normalised SHAP contributions with their Vietnamese labels, applies the rule-based early-warning calibration layer that may raise the score, and persists a fresh `Prediction` row.

The page renders the response as three columns. The leftmost is a Recharts `RadialBarChart` spanning 225 to −45 that displays the risk percentage centred over the discrete risk level. The middle is a bar list of contributing factors — positive factors in red, protective factors in green, each labelled in Vietnamese ("GPA tích lũy khoảng 1.80 dưới mốc an toàn" rather than `gpa_cumulative_deficit = 0.20`). The right is a table of currently-enrolled courses with the predicted pass probability for each, colour-coded green / yellow / red at the 70% and 50% thresholds. A 30-day risk-history area chart sits below the columns, fetched by `predictionsApi.history(30)`, and a refresh button at the top-right invokes `POST /predictions/me/refresh` to force a fresh inference irrespective of the freshness check.

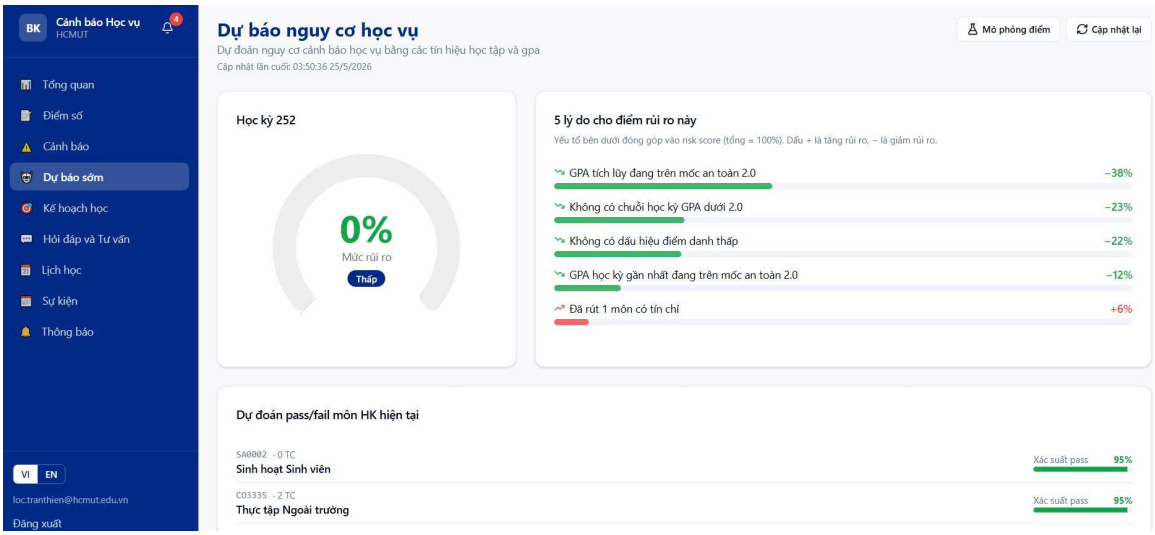

Figure 56: *Predictions page — radial gauge, contributing factors and per-course probabilities*

5.4.0.2 What-if simulator. The simulator (`frontend/app/student/predictions/simulator/page.tsx`) is reached from the link at the top of the predictions page. It fetches the student's enrollments on mount, filters to the courses still in the `enrolled` status and not yet finalised, and renders each one as a row with a number input clamped to the $[0, 10]$ range with 0.1 step. Each row also shows a live grade letter that recomputes from the input via the same scale used by the backend (e.g., $\geq 8.5 \rightarrow$ A, $\geq 7.0 \rightarrow$ B, ...) so the student can see immediately what score corresponds to which grade.

The simulator debounces input changes by 600 ms before issuing `predictionsApi.simulate([{enrollment_id, hypothetical_score}])`, which targets `POST /api/v1/predictions/me/simulate`. The endpoint loads the student's current state, merges the hypothetical scores into a transient in-memory view of the enrollments, recomputes the cumulative GPA and the warning level on that merged view, re-extracts the feature vector, and runs the same XGBoost + SHAP pipeline as the production prediction — but it does not write a `Prediction` row. The response carries both the baseline and simulated risk scores; the page renders them side-by-side with a delta indicator that shows an upward red arrow if the score increased, a downward green arrow if it decreased, and a neutral grey indicator otherwise. The student can iterate through hypothetical exam outcomes quickly without affecting any persisted state.

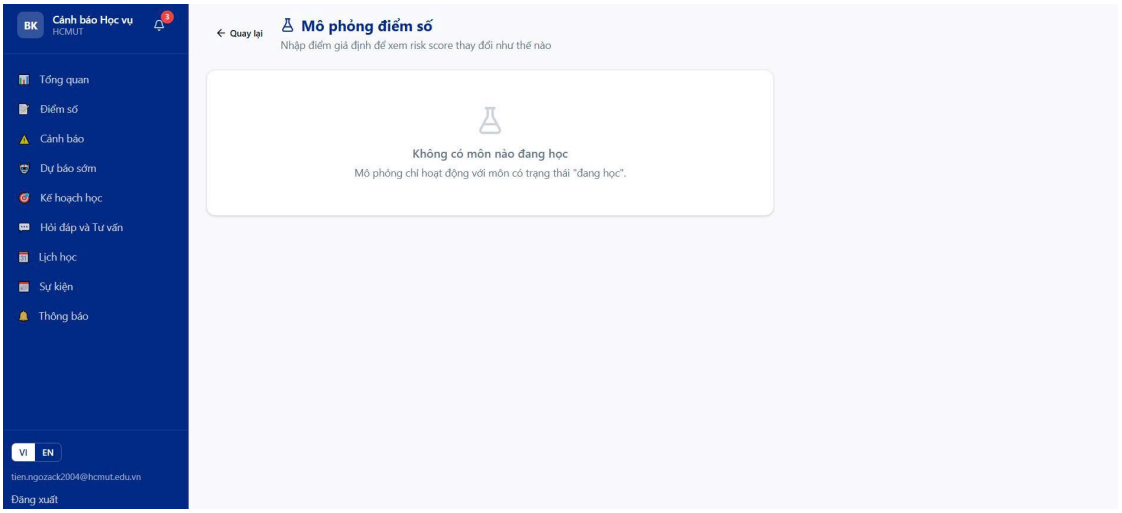

Figure 57: *Simulator — per-course score inputs and the baseline-vs-simulated risk comparison*

5.5 RAG Chatbot

The chatbot workflow connects the page at `frontend/app/student/chatbot/page.tsx` to the dual-mode endpoint pair `POST /api/v1/chatbot/ask` (non-streaming) and `POST /api/v1/chatbot/ask/stream` (Server-Sent Events). The implementation reuses every retrieval and prompt-assembly step described in Section **??**; this section focuses on the page-level integration.

5.5.0.1 Initial state. On mount the page issues two parallel calls. `chatbotApi.history()` returns the last fifty `ChatMessage` rows so the conversation continues where the student left it, and `chatbotApi.suggestions()` returns five context-aware suggestion strings that the backend builds from the student's warning level, GPA and credits. The suggestions are rendered as clickable pills below an empty feed so a first-time user has a guided starting point; clicking a pill pre-fills the textarea.

5.5.0.2 Sending a message. The input area is a textarea with the conventional Enter-to-send / Shift+Enter-for-newline behaviour. On submit the page first tries the streaming endpoint: it opens an `EventSource`-style stream against `/chatbot/ask/stream` with the JWT attached as a Bearer token. The endpoint runs `stream_chatbot_response` in `app/ai/chatbot/chains.py`, which loads the recent history, builds the personal-context string, runs the hybrid retrieval against the `documents` table, assembles the prompt, and forwards each token emitted by the active `ChatProvider` as a `data: {"type":"delta","content":"..."}` frame. The frontend buffers these deltas into the assistant message bubble so the answer appears token-by-token. When the stream ends the backend sends a closing `data: {"type":"done","citations":[...],"provider":"..."}` frame; the page reads the citation array and renders the citation chips beneath the assistant turn.

If the stream itself fails to open — for example because of a network glitch or because a corporate proxy strips Server-Sent Events — the page falls back to `chatbotApi.ask(text)`, which targets the non-streaming `/chatbot/ask` endpoint. The non-streaming path runs `ask_chatbot`, which is logically identical to the streaming path but waits for the full provider response and returns the complete `ChatResponse` object in one go. Either path persists the user and assistant turns (the assistant row carrying the citations JSON) so the conversation survives a refresh.

5.5.0.3 Citations and rendering. Each citation chip carries the source filename, the page number, the chunk index, the similarity score, and a match-type label (vector, keyword, or merged). The match-type label uses three colours so the student can tell whether the retrieval was semantic, lexical, or both — a small but useful signal when judging answer credibility. Assistant messages are rendered through `react-markdown` with custom renderers for code blocks, tables, lists, and blockquotes, because the underlying LLM frequently formats regulation excerpts as Markdown tables.

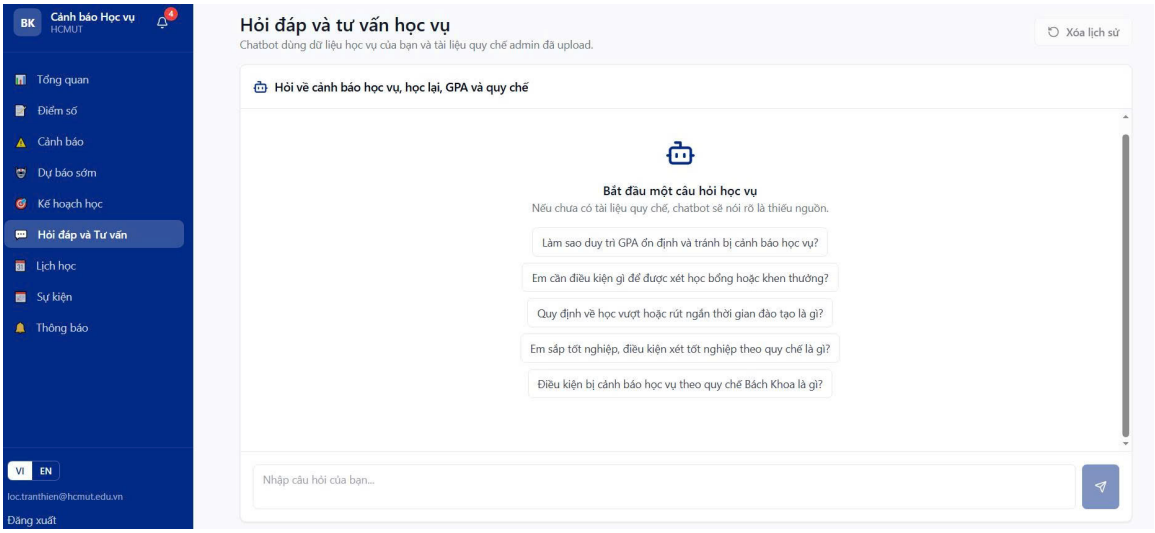

Figure 58: *Chatbot — streamed assistant reply with citation chips and a context-aware suggestion strip*

5.5.0.4 Provider degradation. The page itself is unaware of which chat provider answered. The selection happens server-side in `get_chat_provider()`, which reads the `CHAT_PROVIDER` setting and returns the active class. If the configured provider raises `ProviderConfigError` during the call — the typical cause is a missing or expired Gemini API key in the deployment environment — the chains module catches the exception and substitutes `ExtractiveChatProvider`, which composes a deterministic answer from the retrieved chunks alone. The user-visible effect is that the reply prefix briefly explains that the LLM is unavailable, but the answer still contains the retrieved citations and remains useful for a regulation lookup. The same fall-back applies to the embedding side of the pipeline via `_embed_with_fallback`, so a chat session never dies because an external API has gone down.

5.6 Warnings, Study Plan and Notifications

The warning, study-plan and notification workflows form the student-facing portion of the warning lifecycle: a warning is computed by the engine described in Section **??** (or seeded manually by an administrator), surfaced

to the student here, complemented by a personalised study plan, and announced via the notifications channel.

5.6.0.1 Warnings page. The warnings page (`frontend/app/student/warnings/page.tsx`) issues `warningsApi.list()` on mount, which targets `GET /api/v1/warnings/me` and returns up to fifty rows ordered newest-first. The page derives a summary band from the response (total count, unresolved count, per-level counts) and offers three filter tabs (all, unresolved, resolved) backed by a client-side `useMemo` so switching tabs is instantaneous. Each warning card uses a left-border colour keyed to the level (yellow for level 1, orange for level 2, red for level 3), and surfaces the originating semester, the reason text from the regulation or AI engine, the GPA snapshot at the moment the warning was issued, the AI risk score when applicable, and a badge indicating whether the warning was created by the system or by an administrator. A single button toggles the `is_resolved` state via `PATCH /api/v1/warnings/me/{id}/resolve`; because the endpoint accepts an arbitrary boolean, the same button transparently supports unmarking a previously-resolved warning. Resolved cards are visually dimmed with opacity to keep the unresolved set in focus.

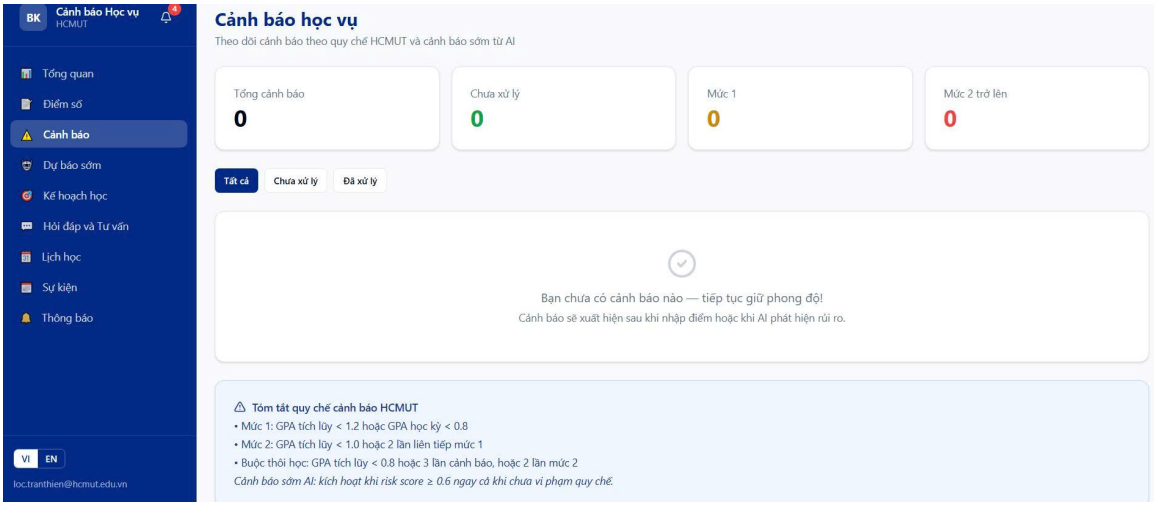

Figure 59: *Warnings page — summary band, filter tabs and per-level colour-coded cards*

5.6.0.2 Study plan. The study-plan page (`frontend/app/student/study-plan/page.tsx`) issues a single call to `studyPlanApi.me()` which targets `GET /api/v1/study-plan/me`. The endpoint orchestrates the recommender by loading the student's enrollments, applying the highest-wins retake rule via `grade_aggregator.effective_enrollments_per_course`, splitting the courses into unresolved-failed and low-grade-passed sets, and then composing the response through `study_plan.build_study_plan`. The composed payload contains a `CreditLoad` dataclass with the minimum, recommended and maximum credit counts plus a rationale string keyed to the student's warning level and GPA, and a list of `RetakeSuggestion` dataclasses ordered by priority.

The page renders three snapshot stat cards at the top (cumulative GPA, credits earned, unresolved failed count) so the student understands the basis for the recommendation. Below it sits the credit-load card with three boxes (minimum / recommended / maximum) and the rationale paragraph rendered verbatim. The retake list is rendered as a sequence of priority-badged rows — priority 1 in red for unresolved failed courses with high credit weight, priority 2 in orange for low-credit unresolved failures, priority 3 in yellow for grade-improvement candidates — each showing the last grade letter, the last total score, the last attempted semester, and the reason text from the recommender. An optional "suggested courses" section lists electives the recommender thinks would lift the cumulative GPA.

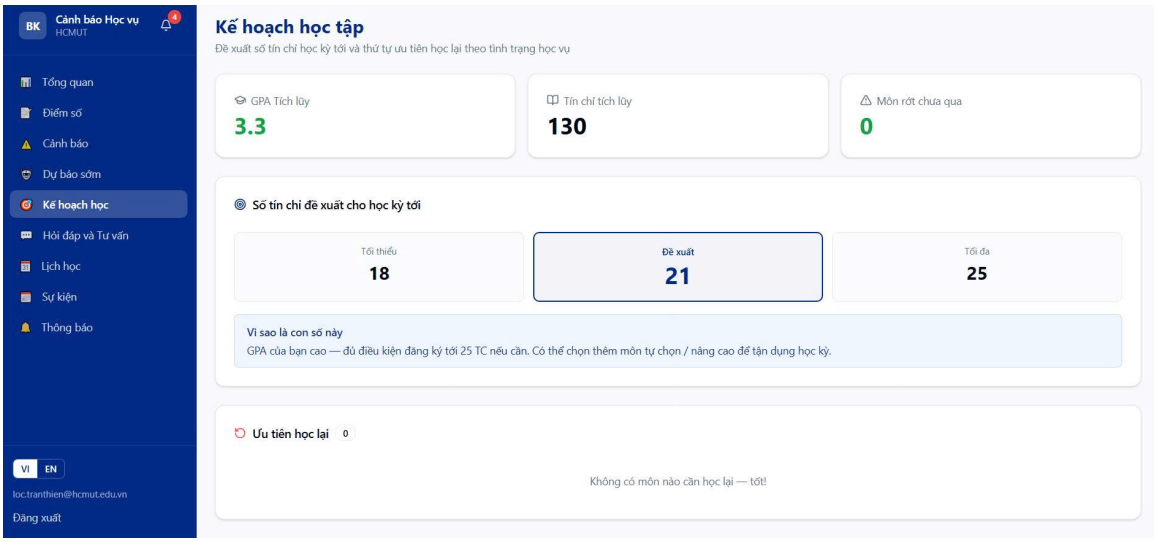

Figure 60: *Study plan — snapshot, credit-load card with rationale, prioritised retake list and suggested courses*

5.6.0.3 Notifications. The notifications page (`frontend/app/student/notifications/page.tsx`) issues `notificationsApi.list(false, 100)` for the most recent hundred notifications and `notificationsApi.getPreferences()` for the email opt-in flag. The page renders the unread count and a "mark all as read" button in the header, followed by a small preference card with the email-on/email-off switch backed by `notificationsApi.updatePreferences(enabled)`. Each notification card carries an icon keyed to the four `NotificationType` values (warning, reminder, event, system), the title and content, the original send timestamp, and an icon indicating whether the corresponding email has been dispatched (this is the same `Notification.email_sent_at` column populated by `email_service.fire_and_forget`). Marking a notification as read is optimistic: the dot disappears immediately, and the UI reverts only if `notificationsApi.markRead(id)` returns an error.

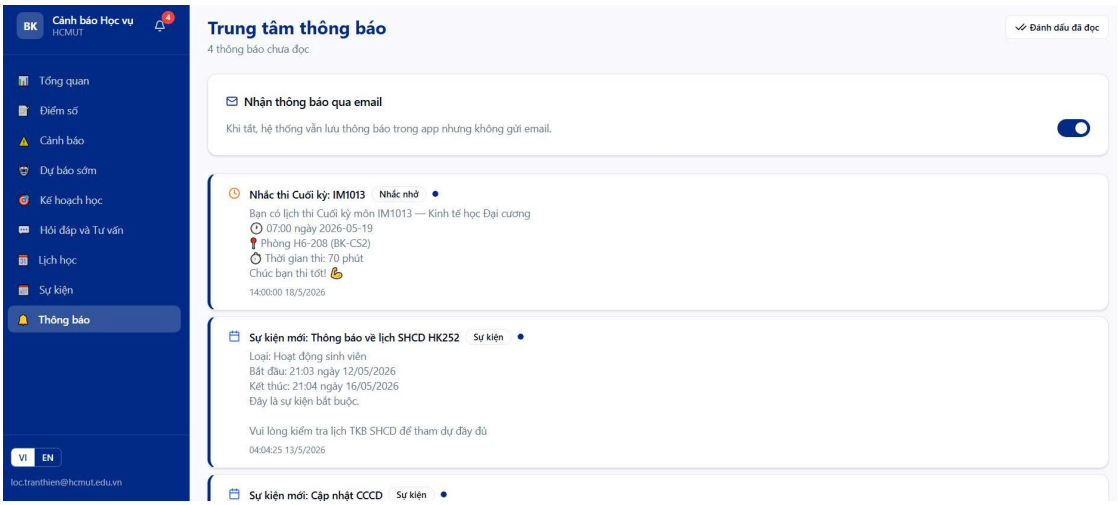

Figure 61: *Notifications — type icons, email opt-in switch and optimistic mark-read interaction*

5.7 Schedule, Exams and Events

The scheduling workflow consolidates three distinct kinds of academic calendar entry behind a single page so that the student does not have to juggle separate views for class timetables, exam dates, and university-wide announcements. All three sources are persisted on the `Student` record as JSONB columns (`timetable_json`, `exam_schedule_json`, `personal_events_json`) introduced by the M9 milestone, plus the relational `Event` table that the administrator populates.

5.7.0.1 Loading the calendar. The page (`frontend/app/student/schedule/page.tsx`) issues four parallel calls on mount: `studentApi.getTimetable()`, `studentApi.getExamSchedule()`, `studentApi.listPersonalEvents()`, and `eventsApi.myUpcoming(50)`. The first three return the JSONB payloads as-is so the frontend can render them without further normalisation; the last invokes the events router

with audience filtering so the student only sees events targeted at all students, their own faculty, or their own cohort. A tab strip at the top of the page switches between the week, month, and list views; each view renders the same merged data set with a different presentation.

5.7.0.2 Week view. The week view is a 15-row by 7-column grid where the rows correspond to HCMUT teaching periods 1 through 15 and the columns to Monday through Sunday. Each course session is rendered as a coloured block; the colour is computed from the course code through a hash function so that the same course always appears in the same colour throughout the week. Adjacent periods belonging to the same course session are merged into a single multi-period block by computing a row-span, which keeps the visual layout clean for two- and three-period blocks. Sessions whose day-of-week is unknown (some myBK exports omit this field) are rendered in a side panel so the student is still aware they exist. Hovering a block reveals a delete button that opens a confirmation dialog with two options — delete only this session or delete every session for the same course code.

5.7.0.3 Month view and list view. The month view is a Recharts-free calendar grid with coloured dots distinguishing the four entry kinds: blue for classes, red for exams, purple for administrator-published events, and green for personal events. Clicking a date opens a day-detail dialog listing every entry on that day. The list view sorts every upcoming exam and event chronologically and renders them as a simple stacked list, which works well for students who prefer a linear agenda.

5.7.0.4 Imports and personal events. Three modals support data entry. The TKB import modal accepts the tab-separated text pasted from `myBK` and routes it through `services/tkb_parser.py`, which extracts course code, day of week, period, room, group, and lecturer into a `ParsedTimetable` dataclass; the modal offers a "merge" option so that re-importing for the same semester adds to the existing schedule instead of replacing it. The exam-schedule modal does the same for exam dates via `exam_schedule_parser.py`, distinguishing mid-term (GK) from final (CK) entries automatically. The personal-event modal lets the student create ad-hoc reminders that are appended to `personal_events_json` without going through any parser. Every import overwrites the relevant JSONB column atomically in a single `PATCH` so the calendar view never lands in a half-updated state.

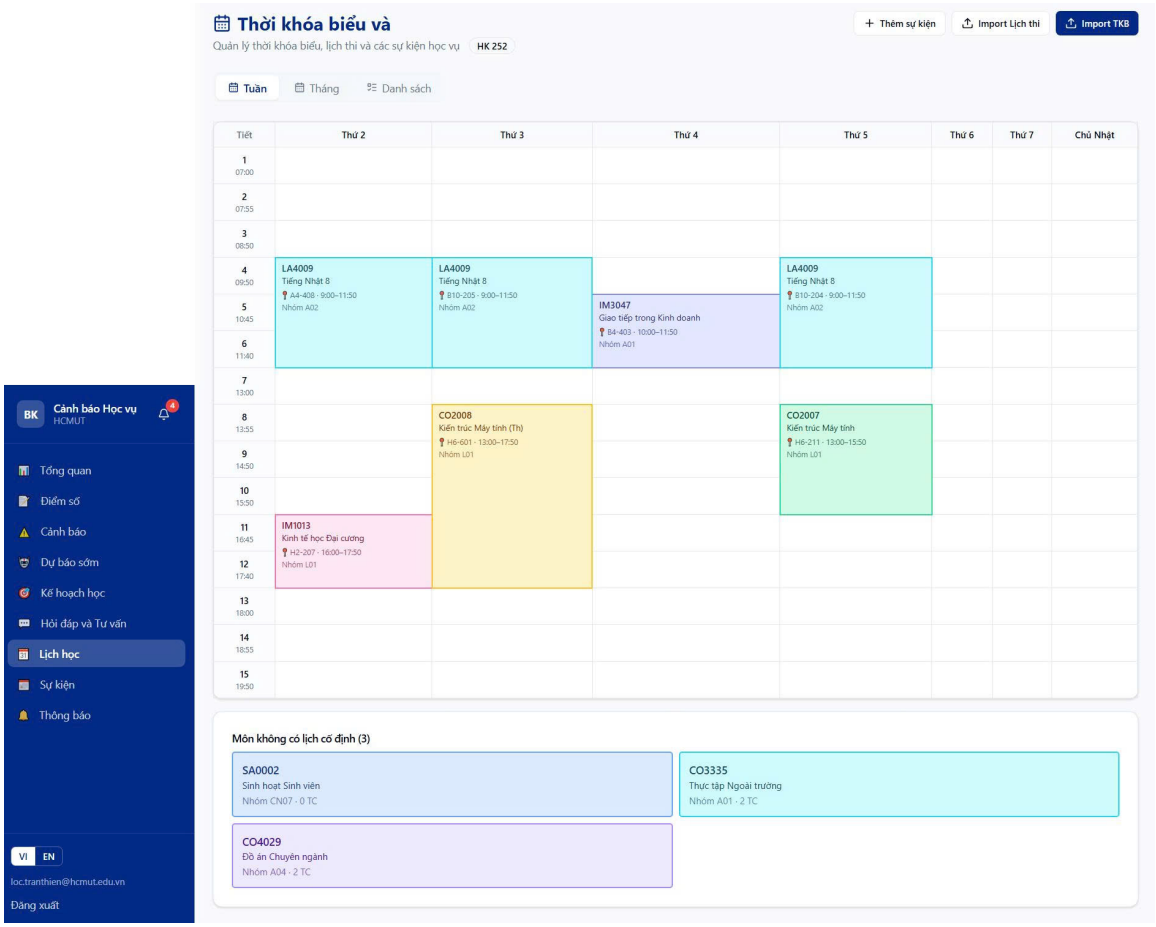

Figure 62: *Schedule — week, month and list views with myBK timetable import and exam schedule*

5.7.0.5 Events page. The dedicated events page (`frontend/app/student/events/page.tsx`) is a thinner view of the same `Event` table. The page has a two-tab control switching between `eventsApi.myUpcoming(50)` (events with a future `start_time`) and `eventsApi.myEvents(100)` (every event the student belongs to). Each card is iconified by the four `EventType` values, shows the date range, and adds a destructive badge when the event is mandatory — mandatory events are exactly those that the `deadline_reminders_daily` scheduler job described in Section 3.2.5 will broadcast as reminder notifications twenty-four hours before they start. Events within the next seven days carry a countdown badge that turns red when the event is within a day, providing a last-minute prompt that complements the email reminder.

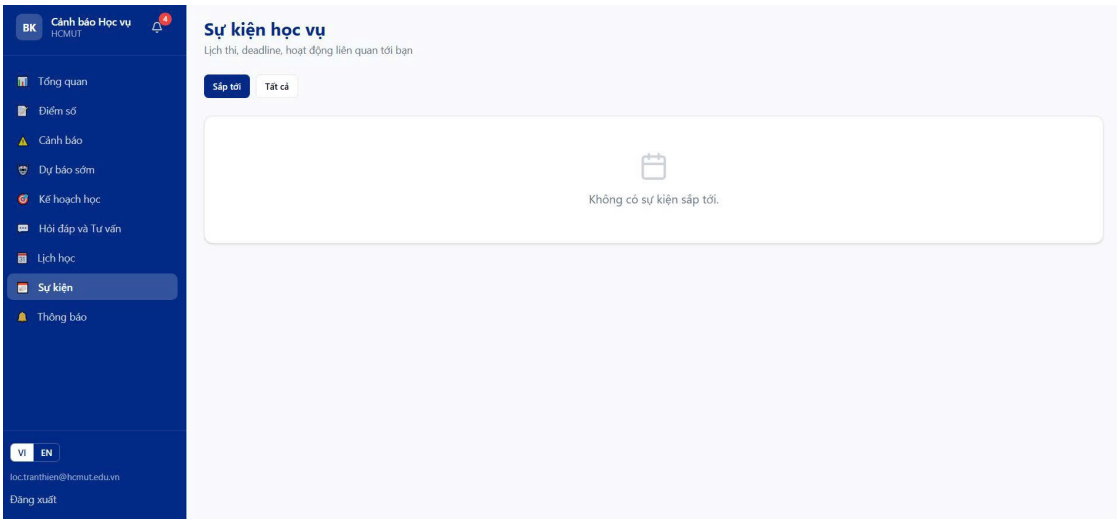

Figure 63: *Events page — upcoming/all tabs, type-keyed icons, mandatory badges and countdown chips*

5.8 Administrator Workflows

The administrator workspace covers the operational tasks an Academic Affairs Office staff member performs — bulk data ingestion, batch risk evaluation, warning approval, document curation, event publishing, and statistical reporting. Every administrator route lives under `/admin/`; the layout file inspects the persisted user object on mount and redirects mismatched roles back to the student dashboard, so a student following an `/admin/...` link will never see the administrator interface.

5.8.0.1 Dashboard and student list. The administrator dashboard (`frontend/app/admin/dashboard/page.tsx`) issues a single call to `adminApi.dashboard()` which targets `GET /api/v1/admin/dashboard`. The endpoint returns four top-line counters (total students, total under warning, high-risk, critical-risk), a risk-level distribution, a per-faculty distribution, and the top ten at-risk students. The page renders the counters as stat cards, the distributions as horizontal-bar widgets, and the top-ten as a table where each row is a link to the student-detail page. The student list page (`/admin/students`) is a paginated table backed by `adminApi.listStudents(page, size, q?, warning_level?, high_risk?)` with twenty rows per page, a search field that debounces input through the URL query parameters, and three filter chips (all, on warning, high risk). Selecting a row opens the student-detail page (`/admin/students/[id]/page.tsx`) which renders the full profile, the GPA trend, the warning history and the AI risk factors, plus an inline form for issuing a manual warning via `POST /api/v1/admin/warnings/manual`.

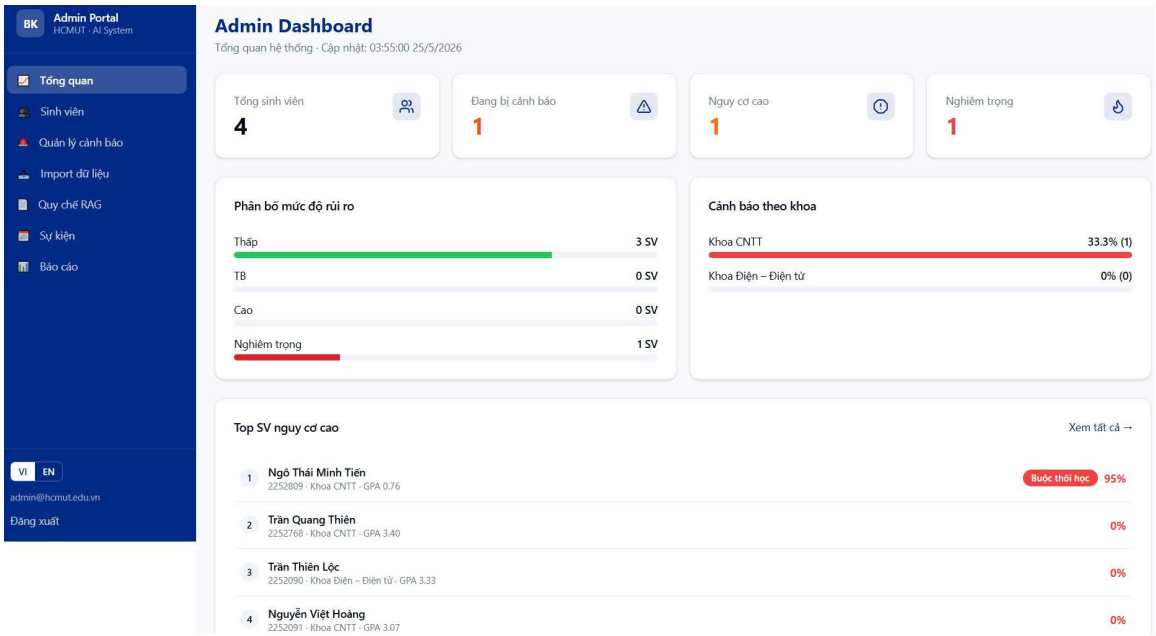

Figure 64: *Administrator dashboard — counters, distributions and top-ten at-risk table*

5.8.0.2 Warning approval queue. The warning page (`frontend/app/admin/warnings/page.tsx`) is the operational core of the administrator workflow. The top of the page exposes the AI suggestion threshold as an editable percentage backed by `adminApi.updateThreshold(value)` which writes to the configuration value consulted by `warning_engine.check_ai_early_warning`. A primary action runs the batch routine in two stages — first `adminApi.runBatchPredictions()` which calls `predict_batch` on every active student, then `adminApi.runBatchWarnings()` which sweeps the resulting predictions through `warning_engine.batch_check_warnings` to materialise the pending queue. Three stat cards show the pending count, the count sent this semester and the last batch timestamp. The pending queue itself is the response of `adminApi.pendingWarnings()`; each entry shows the student, the proposed level, the AI risk score, the reason text, and three actions: open a detail modal, approve (which calls `adminApi.approveWarning(...)` and triggers `notification.create(...)` server-side which fires off the email through `email_service.fire_and_forget`), or dismiss (which simply removes the suggestion without sending anything). Approving immediately removes the entry from the queue with an optimistic UI update.

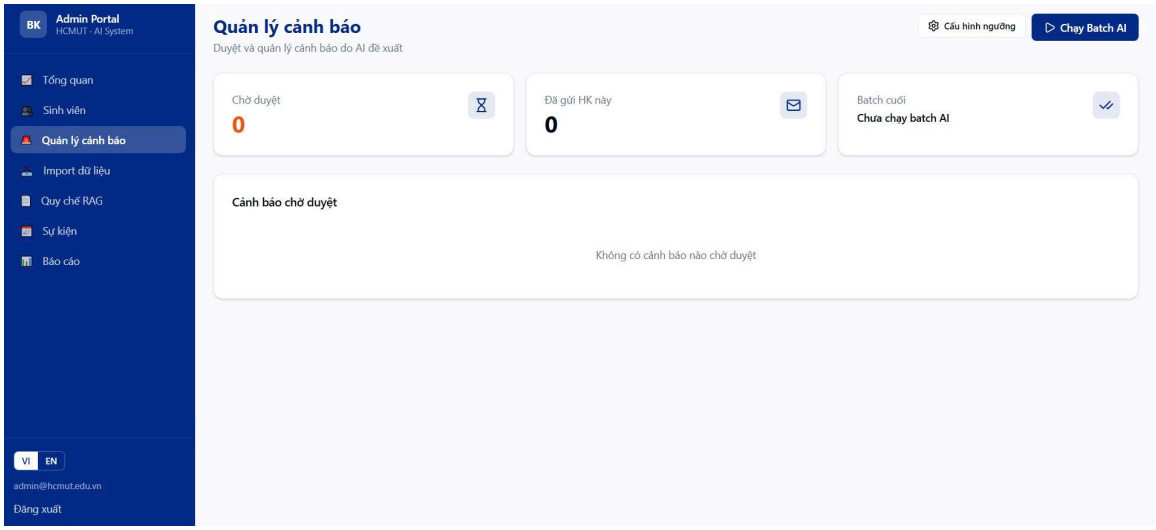

Figure 65: *Warning approval queue — threshold editor, batch trigger and per-item approve/dismiss controls*

5.8.0.3 Bulk import. The import page (`frontend/app/admin/import/page.tsx`) presents two upload boxes (students, grades) and the corresponding template-download buttons. The upload boxes accept the workbook directly and POST it to `POST /api/v1/admin/import/students` or `/import/grades`, which delegate to `services/import_service.py`. The service parses every row, validates against the schema (required columns, MSSV/email format, uniqueness), and either commits the valid rows in a single transaction or returns row-level

errors that the page renders as an expandable table beneath the result card. The administrator may then optionally re-trigger the batch warning-evaluation endpoint described above to refresh the warning state using the freshly-imported grades — the import flow itself does not run that check implicitly. A history table below the upload boxes records each previous run with its filename, the rows seen, and the breakdown into created / updated / failed counts.

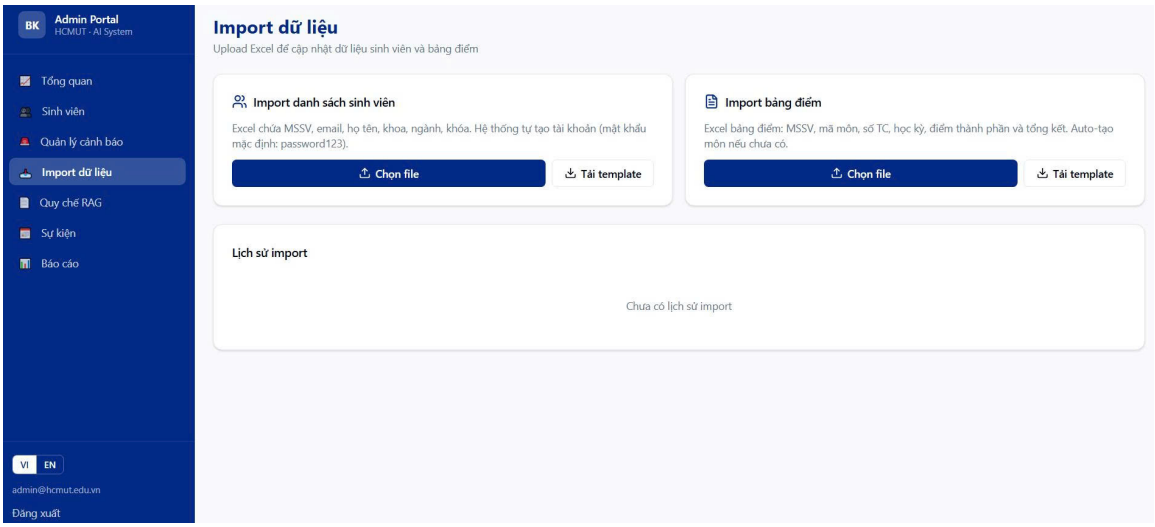

Figure 66: *Bulk import — upload boxes, template downloads, result card with row-level errors, run history*

5.8.0.4 RAG document repository. The documents page (`frontend/app/admin/documents/page.tsx`) manages the corpus the chatbot retrieves from. A batch-upload action accepts multiple files in any of the supported formats (PDF, DOCX, TXT, MD; ZIP archives are unpacked at the API), routes each through `POST /api/v1/documents/upload-batch`, and shows a per-file success or failure status afterwards. The endpoint delegates to `ingest_document` in `vectorstore.py`, which runs the full indexing pipeline described in Section **??**: parsing through PyMuPDF (with the Tesseract OCR fallback for scanned PDFs), word-based chunking with overlap, embedding through the active provider with a hash-based fallback, and bulk insertion of `Document` rows. The page table shows the per-file chunk count and page count, an active/inactive toggle that flips `Document.is_active` without deleting any row (so toggling inactive merely excludes the document from retrieval), and a delete button that permanently removes every chunk for that source.

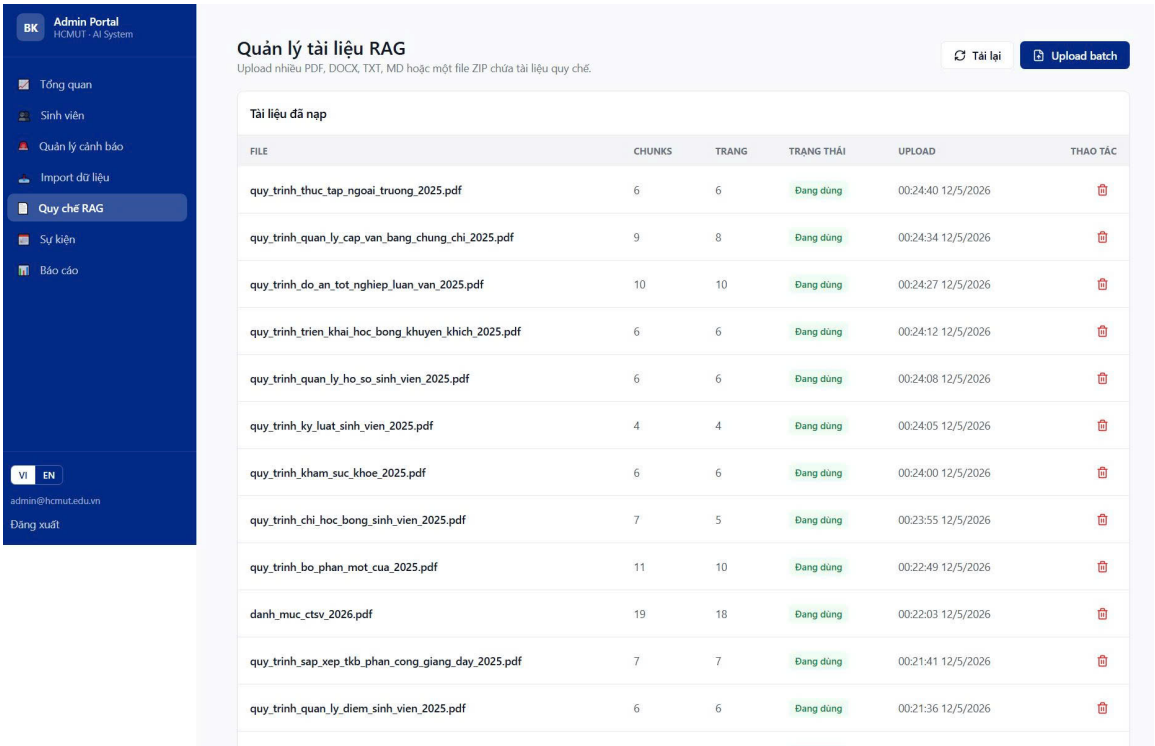

Figure 67: *Document repository — batch upload, per-file status, active toggle and per-source chunk counts*

5.8.0.5 Events and reports. The events page (`frontend/app/admin/events/page.tsx`) renders an inline create / edit form covering every `Event` attribute (title, type, start / end time, target audience and value, description, mandatory flag) and a list of previously-published events with edit and delete controls. The reports page (`frontend/app/admin/reports/page.tsx`) consumes `adminApi.statistics()` to populate four institution-level stat cards and four Recharts widgets (warnings by semester as a bar chart, GPA distribution as a colour-coded histogram, risk distribution as a donut chart, and faculty warning rates as horizontal bars). Four export panels at the bottom let the administrator download the warnings report, the GPA report, the AI report, and the warned-students list in either PDF or Excel; the downloads stream the file from the backend and respect the `Content-Disposition` filename returned by the server.

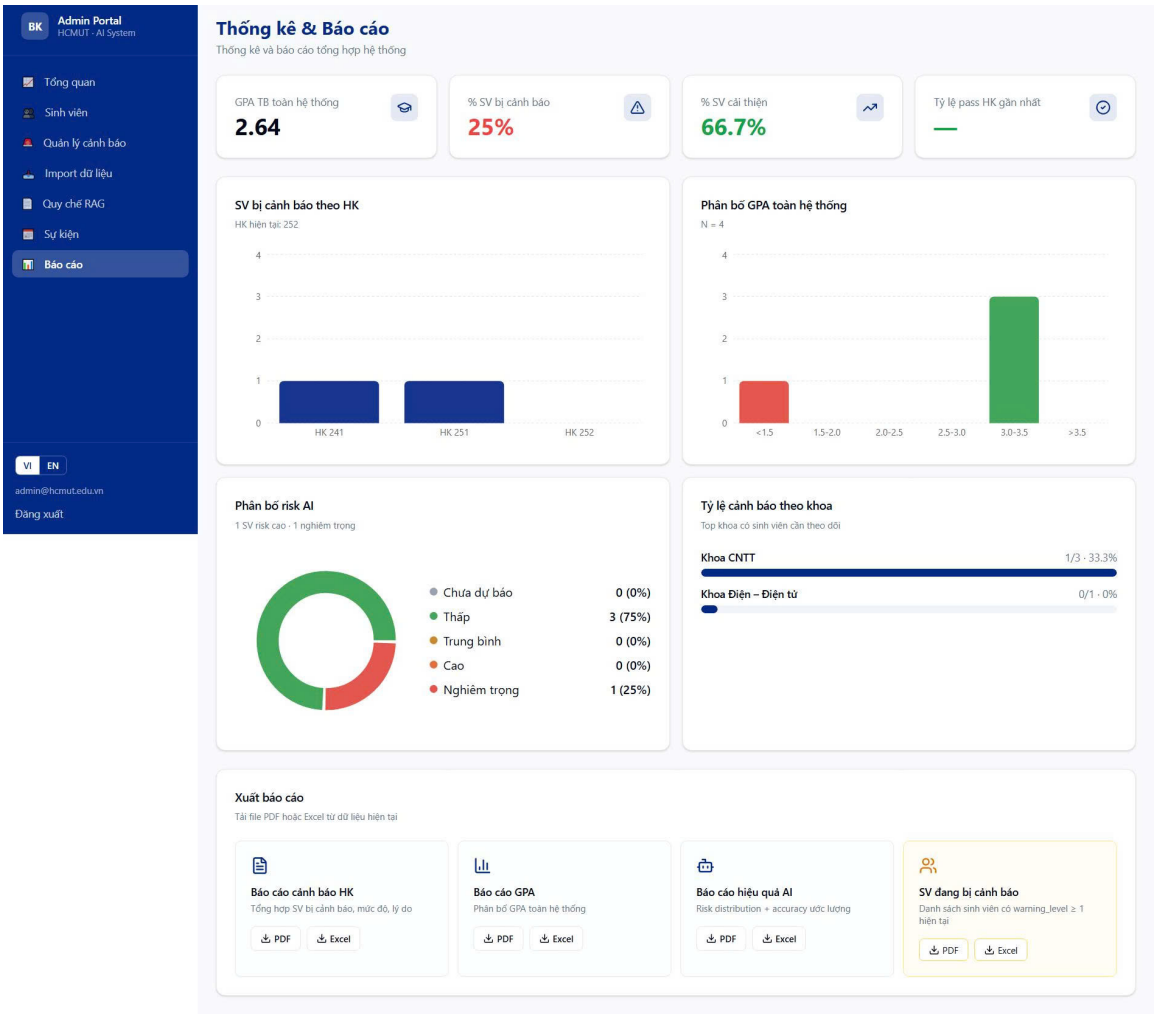

Figure 68: *Reports — statistics cards, charts and export panels in PDF and Excel*

6 Future Plan and Roadmap

This chapter describes the remaining work after the milestones already completed and the longer-horizon directions in which the prototype can grow. The work is organised into three sequential phases that cover the gap from the current production-ready prototype to a defensible final submission and beyond. The accompanying Gantt chart in Figure 69 visualises the phase boundaries and the major tasks within each phase.

6.1 Gantt Chart and Phase Analysis

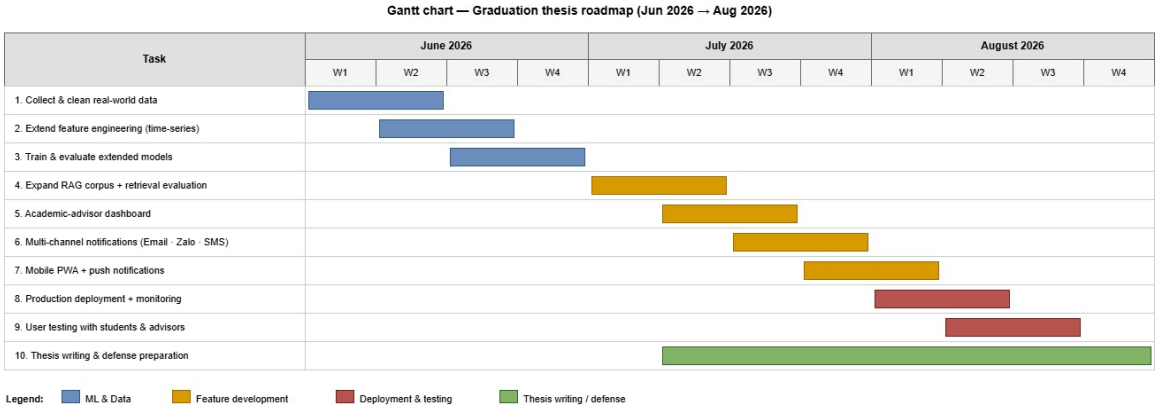

Figure 69: *Gantt chart for the remaining three phases*

6.1.0.1 Phase 1 — Feature completion and polish. The first phase closes the gap between the current state and a feature-complete prototype. The two pieces of work still open are the schedule features and the production-grade sample data for the RAG corpus. The schedule feature set — timetable, exam-schedule and personal-event JSONB columns plus the parsers — is functionally in place, but the week-view rendering needs a final pass on multi-period merging edge cases and the personal-event reminder still has to be wired into the cron job. On the RAG side, the chatbot already works end-to-end against any uploaded document, but the demonstration corpus needs a curated set of HCMUT regulation files (including the consolidated training regulation published by the Academic Affairs Office) so that the live demo retrieves meaningful citations rather than generic placeholder content. A small amount of bug-fix follow-up on the administrator import path completes the phase.

6.1.0.2 Phase 2 — Testing and validation. The second phase widens the existing automated test suite and runs the validation workload that confirms the system meets the non-functional targets set in Section 1.4.2. The automated tests cover the warning engine, the GPA calculator and the myBK parser as pure-function modules; the next priority is integration coverage for the prediction endpoint, the chatbot streaming endpoint, and the administrator approval flow, because those are the highest-impact code paths if they regress. In parallel the validation workload re-trains the XGBoost model on the synthetic dataset, captures the resulting metrics, runs a load test against the API surface to confirm that the dashboard load time stays under three seconds and the batch prediction for a thousand students completes inside the sixty-second budget, and walks each user-facing workflow manually to validate the screenshots that populate Chapter 4 and Chapter 5. Issues uncovered during this phase feed into a small bug-fix backlog that is closed before Phase 3 begins.

6.1.0.3 Phase 3 — Report and defence preparation. The third phase consolidates the deliverables for the thesis defence. The narrative chapters are already drafted (this document); the remaining work is to finalise every diagram referenced as a placeholder in Chapters 2 and 3, capture the live screenshots that fill the placeholders in Chapters 4 and 5, and produce the supporting presentation slides. The slide deck mirrors the report structure but compresses each chapter into a small number of decision-oriented slides: a problem statement, the architecture diagram, the prediction pipeline, the chatbot pipeline, a live demo, and the validation metrics. The final activity is a rehearsal pass to time the spoken delivery and field anticipated questions on the AI calibration layer, the choice of XGBoost over neural network baselines, and the fallback strategy for the chatbot.

6.2 Long-Term Roadmap

Beyond the immediate three-phase plan, the prototype admits several directions of growth that are out of scope for the current submission but worth noting for the defence and for follow-up work.

The most impactful next step is to **train the XGBoost model on real HCMUT data**. The current model is trained on a thousand-row synthetic dataset whose labels follow plausible but ultimately fabricated distributions;

replacing it with anonymised real student records (subject to the appropriate data-protection approvals) would produce risk scores whose absolute values are directly comparable to the Academic Affairs Office's own end-of-semester review. A related enhancement is to expand the feature set with attendance-rate signals, the credit ratio per semester, and per-faculty calibration so the model accounts for the different grading distributions across the engineering faculties.

A second direction is to **integrate the chatbot corpus with the official regulation publishing pipeline** so that document updates flow automatically into the vector store. The current administrator-upload workflow is manual and version-unaware; a small ingestion daemon that polls the Academic Affairs Office's document folder, computes content hashes, and re-ingests changed files would keep the chatbot answers continuously aligned with the latest published regulation without operator intervention.

A third direction is the **deployment hardening** required for a real production rollout: HTTPS termination through a reverse proxy, separate read and write database users, daily backups with point-in-time recovery, a rotating-token scheme for the JWT, structured logging shipped to a central log store, and an uptime monitor with alerting. None of these are technically interesting but all of them are prerequisites for moving the system out of the demo environment.

Finally the prototype is straightforwardly **extensible to other HCMUT-affiliated institutions and beyond**. The warning engine encodes only the HCMUT regulations through a configurable rules layer, so adapting to a different university's regulation set is a matter of changing the thresholds and re-training the model on their dataset rather than rewriting the code. The architecture (the modular monolith, the in-process scheduler, the asynchronous backend, the hybrid-retrieval chatbot, the SHAP-explained prediction) is generic enough that the same artefact can serve any institution that needs early academic warning, with the institution-specific parts contained in the configuration and the document corpus.

7 Workload Distribution

7.1 Process Retrospective

During the project implementation process, the team worked actively and with discipline to ensure that the planned milestones were completed on time. Each member followed the agreed schedule, and the team maintained regular meetings with the supervisor to report progress, discuss difficulties, and receive timely feedback. Task allocation was carried out fairly based on each member's technical strengths. In addition, whenever a member encountered difficulties, the other members were always willing to provide support, helping the tasks be completed according to requirements and improving overall consistency within the team.

Besides the achieved results, the team also recognized several limitations, such as the pressure of balancing individual academic workload with project progress, insufficient anticipation of some technical dependencies between tasks, which led to delays in certain stages, and the need to improve the use of task management tools to track dependencies more clearly. From these experiences, the team learned valuable lessons about weekly detailed planning, defining clear deliverables for each task, strengthening communication during the integration and testing phases, and applying project management tools to monitor progress and assign tasks more effectively.

Overall, the implementation process demonstrated the strong spirit of collaboration and responsibility among team members. The proposed improvements are expected to help optimize working efficiency and enhance product quality in the subsequent stages of the project.

7.2 Distribution

The work distribution table consists of a single row describing the responsibilities the author carried.

Table 17: *Work distribution*

Name	Student ID	Responsibilities	Completion
Ngo Thai Minh Tien	2252809	End-to-end ownership: requirements analysis, system and database design, full-stack implementation (frontend, backend, AI subsystem), Docker deployment configuration, automated testing, documentation, and report writing	100%
Nguyen Viet Hoang	2252235	Documents, Reports, Diagram, Model, Architecture Design. Testing sytem and Business Analysis	100%

References

[1] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. https://doi.org/10.1145/2939672.2939785

[2] Ho Chi Minh City University of Technology, Vietnam National University HCMC. (n.d.). *Undergraduate training regulations*. Retrieved May 25, 2026, from https://hcmut.edu.vn

[3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. https://arxiv.org/abs/2005.11401

[4] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates, Inc. https://arxiv.org/abs/1705.07874

[5] Tiggeloven, T., Pfeiffer, S., Matanó, A., van den Homberg, M., Thalheimer, L., Reichstein, M., & Torresan, S. (2025). The role of artificial intelligence for early warning systems: Status, applicability, guardrails, and ways forward. *iScience*, 28(11), 113689. https://doi.org/10.1016/j.isci.2025.113689